

โครงการเทคโนโลยีสารสนเทศเพื่อธุรกิจ
Senior Project in Information Technology for Business

ภาควิชาสถิติ คณะพาณิชยศาสตร์และการบัญชี
จุฬาลงกรณ์มหาวิทยาลัย

เรื่อง
68B21a

การออกแบบและพัฒนา AI Chatbot ด้วยเทคนิค
Retrieval-Augmented Generation
สำหรับการตอบคำถามบุคลากรในจุฬาลงกรณ์
Design and Development of an AI Chatbot
using Retrieval-Augmented Generation
for Answering Inquiries from Chulalongkorn University Personnel

โดย
6542095626 นายภาคิน ภูโธจนวงศ์

อาจารย์ที่ปรึกษา

ศ.ดร.วชนนวิริย สิทธาวัฒน์

คณะกรรมการ

รศ.ดร.อัษฎาพร ทรัพย์สมบูรณ์

ผศ.ดร.ภูริพันธ์ รุจิขจร

ปีการศึกษา 2568

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาระบบช่วยค้นและตอบคำถามจากเอกสารภายในมหาวิทยาลัย โดยผสาน “การค้นคืนข้อมูล” เข้ากับ “การสรุปคำตอบพร้อมอ้างอิงต้นฉบับ” เพื่อให้บุคลากรและนิสิตเข้าถึงข้อมูลที่ถูกต้อง ทันสมัย และตรวจสอบย้อนกลับได้ ระบบรองรับทั้งภาษาไทยและอังกฤษ ใช้งานกับเอกสารหลายรูปแบบ เช่น หน้าเว็บและไฟล์ PDF และออกแบบให้เชื่อมโยงไปยังต้นฉบับผ่านลิงก์อ้างอิง เพื่อความโปร่งใสและน่าเชื่อถือของคำตอบ

ผู้จัดทำได้ออกแบบและพัฒนาต้นแบบระบบในลักษณะสถาปัตยกรรมเป็นขั้นตอน ตั้งแต่รับข้อมูล สกัดและทำความสะอาด แบ่งชิ้นข้อมูลที่เหมาะสม ไปจนถึงค้นคืนและสังเคราะห์คำตอบ พร้อมความสามารถดูแลข้อมูลภายหลัง เช่น อัปเดตหรือลบรายการตาม URL ที่มา จากการทดสอบใช้งานกับกรณีใช้จริง พบว่าสามารถลดเวลาค้นหา ลดความคลาดเคลื่อนจากข้อมูลล้าสมัย และยกระดับประสิทธิภาพการสื่อสารภายในหน่วยงานได้อย่างชัดเจน ทั้งนี้ได้สรุปข้อจำกัดและข้อเสนอแนะสำหรับการพัฒนาต่อ เช่น การขยายโดเมนข้อมูล การกำกับคุณภาพข้อมูลต่อเนื่อง รวมถึงระบบสิทธิ์เข้าถึงและการติดตามการใช้งาน



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

กิตติกรรมประกาศ

โครงการ“ระบบช่วยค้นและตอบคำถามจากเอกสารของมหาวิทยาลัยด้วยแนวทาง Retrieval-Augmented Generation (RAG)” ฉบับนี้ สำเร็จลุล่วงได้ด้วยความสำเร็จและการสนับสนุนจากหลายฝ่าย ผู้จัดทำขอกราบขอบพระคุณอาจารย์ที่ปรึกษาและคณะกรรมการทุกท่าน ที่กรุณาให้คำแนะนำอย่างใกล้ชิด ชี้แนะแนวทางวิชาการ ตรวจสอบรายละเอียด และผลักดันให้ผู้จัดทำพัฒนาผลงานให้มีความถูกต้อง เที่ยงตรง และพร้อมใช้งานเชิงปฏิบัติ

ขอขอบคุณหน่วยงานภายในมหาวิทยาลัยที่เอื้อเฟื้อข้อมูลสาธารณะและแนวปฏิบัติเกี่ยวกับงานทะเบียน งานวิชาการ และบริการนิสิต ตลอดจนเจ้าหน้าที่ที่ให้ข้อมูลประกอบการทดสอบระบบอย่างเป็นกันเอง ทั้งนี้ผู้จัดทำเคารพในนโยบายการใช้ข้อมูลและความเป็นส่วนตัวของทุกหน่วยงานเสมอ

ผู้จัดทำขอขอบคุณชุมชนโอเพ่นซอร์สและนักพัฒนาซอฟต์แวร์ที่เผยแพร่เครื่องมือและไลบรารีคุณภาพ ซึ่งเป็นรากฐานสำคัญของงานวิจัยและพัฒนาในครั้งนี้ ตลอดจนเพื่อนร่วมชั้น/ทีมงานที่ช่วยแลกเปลี่ยนความคิดเห็น ทดลองใช้งาน ให้ข้อสังเกตเชิงเทคนิค รวมถึงสนับสนุนด้านการจัดทำรายงานและสื่อประกอบ

สุดท้ายนี้ขอขอบคุณครอบครัวและมิตรสหายที่คอยเป็นกำลังใจ เข้าใจ และอยู่เคียงข้างในระหว่างการดำเนินโครงการ จนทำให้งานชิ้นนี้เสร็จสมบูรณ์ตามเป้าหมาย หากมีข้อบกพร่องประการใด ผู้จัดทำขอน้อมรับไว้เพื่อปรับปรุงในโอกาสต่อไป

CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

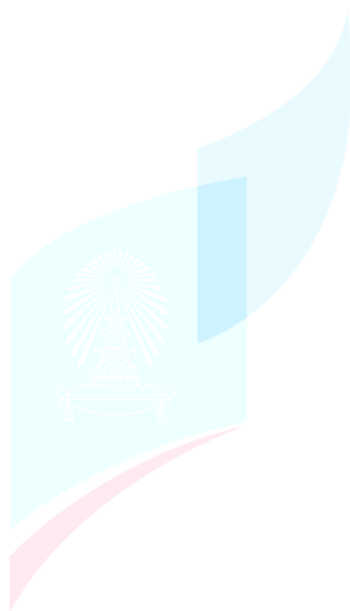
สารบัญ

บทที่ 1 บทนำ.....	1
1.1 ภูมิหลังและที่มาของปัญหา.....	1
1.2 วัตถุประสงค์ของโครงการ.....	1
1.3 ขอบเขตของระบบ.....	2
1.4 วิธีดำเนินงาน	2
1.5 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 งานที่เกี่ยวข้องและเทคโนโลยีที่ใช้	3
2.1 ภูมิทัศน์ระบบสารสนเทศของจุฬาลงกรณ์มหาวิทยาลัย	3
2.1.1 แผนที่โดเมนและบทบาทหลัก.....	3
2.1.2 ชนิดข้อมูลที่พบโดยทั่วไป.....	4
2.2 แนวทางการทำ RAG ทั่วไป.....	4
2.2.1 ความหมายและจุดประสงค์ของ RAG	5
2.2.2 โครงสร้าง และ โมดูลทั่วไปของระบบ RAG	5
2.2.3 ตัวเลือกในการออกแบบ (Design Choices).....	6
2.2.4 ประเภทของ RAG และ รูปแบบสถาปัตยกรรมที่หลากหลาย.....	7
2.2.5 ประเด็นที่ต้องพิจารณา ความท้าทาย และ ข้อจำกัด.....	7
2.2.6 แนวทางปฏิบัติที่ดี.....	8
2.3 เครื่องมือและไลบรารีหลัก.....	9
2.3.1 Playwright	9
2.3.2 Trafilatura.....	10
2.3.3 PyMuPDF (fitz).....	10
2.3.4 pymupdf4llm	10
2.3.5 EasyOCR	11
2.3.6 Tiktoken	11
2.3.7 sentence-transformers / BGE-M3 embedding	11
2.3.8 FlagEmbedding	12
2.3.9 ChromaDB.....	12
2.3.10 OpenAI	13
บทที่ 3 การวิเคราะห์ระบบและข้อกำหนด (System & Data Requirements).....	14
3.1 กำหนดผู้ใช้เป้าหมาย.....	14

3.2 กรณีใช้งานหลัก (Use Cases).....	14
3.2.1 UC-01: ถาม-ตอบภาษาธรรมชาติ (TH/EN)	15
3.2.2 UC-02: เปิดดูเอกสารต้นฉบับจาก URL	15
3.2.3 UC-03: ค้นหาเอกสารงานทะเบียนด้วยคีย์เวิร์ด.....	16
3.2.4 UC-04: ลบข้อมูลจากดัชนี.....	16
3.2.4 UC-05: อัปเดตดัชนี	17
3.3 ข้อกำหนดเชิงหน้าที่ (Functional Requirements)	17
3.4 ข้อกำหนดเชิงคุณภาพและการปฏิบัติงาน (Non-Functional Requirements)	18
3.5 ข้อกำหนดข้อมูล (Data Requirements)	19
3.5.1 ขอบเขตและนโยบายข้อมูล.....	19
3.5.2 ประเภทและรูปแบบข้อมูลที่รองรับ	20
3.5.3 แหล่งข้อมูลและโดเมน.....	22
3.5.4 Metadata ระดับ “เอกสาร”	23
บทที่ 4 การออกแบบและการพัฒนา (Design & Implementation).....	25
4.1 ภาพรวมสถาปัตยกรรมการทำงาน.....	25
4.2 การรวบรวมข้อมูลจากหน้าเว็บ.....	25
4.2.1 ภาพรวมการไหลของงาน	25
4.2.2 การตั้งค่า (CFG) และผังเอ้าต์พุต	27
4.2.3 การทำ URL ให้เป็นมาตรฐานเดียว (Canonicalization)	28
4.2.4 การรวบรวม Seed จาก Sitemaps.....	29
4.2.5 ดึง HTML แบบเรนเดอร์จริง (Playwright) และ Content-Type Gate	29
4.2.6 สกัดลิงก์และไฟล์สื่อจาก HTML	30
4.2.7 ดาวน์โหลดไฟล์ RAW และเขียน Manifest	31
4.2.8 จำกัดอัตราการร้องขอแบบต่อโฮสต์ (Per-Host Rate Limiter).....	32
4.2.9 ผลลัพธ์การดึงข้อมูล	33
4.3 การสกัดข้อมูลจากข้อมูลดิบ (RAW → documents.jsonl)	33
4.3.1 คอนฟิกและตัวช่วยเฉพาะงานสกัด	34
4.3.2 แปลง HTML เป็นข้อความ	34
4.3.3 แปลงรูปภาพเชิงข้อมูล	35
4.3.4 แปลง PDF เป็นข้อความ	36
4.3.5 ผลลัพธ์จากการสกัดข้อมูล	37
4.4 การทำ chunking และแปลงข้อมูลให้พร้อมฝังเวกเตอร์	38

4.4.1 ภาพรวมเทคนิคและพารามิเตอร์.....	38
4.4.2 กลไกการทำ chunking	40
4.4.3 การเปลี่ยนฟอร์แมตสำหรับระบบฝังเวกเตอร์	41
4.5 ระบบฝังเวกเตอร์.....	42
4.5.1 ภาพรวมการทำงานของระบบฝังเวกเตอร์ และเหตุผลการเลือกโมเดล	42
4.5.2 กลไกฝังเวกเตอร์และจัดเก็บใน	43
4.5.3 การลบและอัปเดตเวกเตอร์.....	44
4.6 ระบบค้นคืนคำตอบแบบเวกเตอร์	45
4.6.1 Retriever.....	45
4.6.2 Re-ranker.....	45
4.6.2 Answer Synthesizer (สังเคราะห์คำตอบพร้อมอ้างอิง).....	46
บทที่ 5 ผลการทดลองและการประเมินผล (Evaluation)	49
5.1 สรุปผลลัพธ์การดึงข้อมูล (Ingestion Summary).....	49
5.1.1 ภาพรวมเชิงปริมาณ.....	49
5.1.2 การกระจายตามโดเมน	49
5.1.3 การกระจายตามชนิดเอกสาร	50
5.1.4 การกระจายตามภาษา.....	50
5.1.5 เอกสารที่มี “ตาราง” และผลกระทบต่อการสกัด.....	50
5.1.6 หมายเหตุวิธีนับและข้อจำกัดของชุดข้อมูลนี้.....	51
5.2 ผลลัพธ์ของการตัดชิ้นข้อความ (Chunking Results).....	51
5.2.1 ภาพรวมเชิงปริมาณ.....	51
5.2.2 การกระจายตามโปรไฟล์.....	51
5.2.3 การกระจายตามชนิดเอกสาร (ของชิ้นที่ได้)	52
5.2.4 ภาษาโดยประมาณของชิ้น	52
5.2.5 โดเมนต้นทางของชิ้น	52
5.2.6 หมายเหตุเรื่องเมทาตาทา (สำคัญต่อขั้นฝังเวกเตอร์).....	52
5.3 การประเมินผลการค้นคืน (Retrieval)	53
5.4 ผลการทดลองตาม Use Case.....	55
5.4.1 การถาม-ตอบภาษาธรรมชาติ (TH/EN).....	55
5.4.2 เปิดดูเอกสารต้นฉบับจาก URL.....	56
5.4.3 ค้นหาเอกสารงานทะเบียนด้วยคีย์เวิร์ด.....	56
5.4.4 ลบข้อมูลจากดัชนี.....	57

5.4.5 อัปเดตดัชนี.....	57
5.5 อภิปรายผล	58
บทที่ 6 สรุปและข้อเสนอแนะ	59
6.1 สรุปผลตามวัตถุประสงค์.....	59
6.2 ปัญหาและแนวทางแก้ไข	60
6.3 ข้อเสนอแนะ.....	61
บรรณานุกรม	63
ภาคผนวก	66

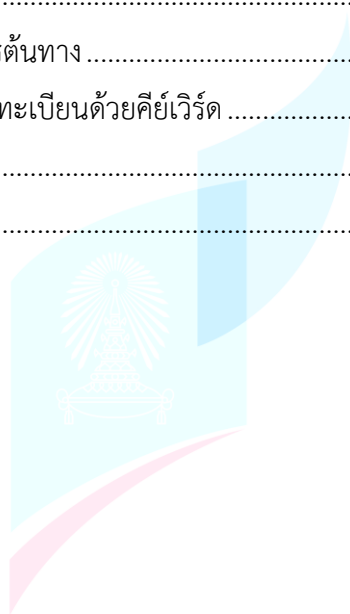


CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

สารบัญรูป

รูปที่ 3.1 Use case diagram	14
รูปที่ 4.1 ล็อกแสดงผลการดึงข้อมูล	33
รูปที่ 4.2 ผลลัพธ์การฝังเวกเตอร์.....	44
รูปที่ 5.1 ผลลัพธ์ก่อน re-rank	53
รูปที่ 5.2 ผลลัพธ์หลัง re-rank.....	54
รูปที่ 5.3 ผลลัพธ์คำตอบสรุปขั้นตอนลงทะเบียนเรียน	55
รูปที่ 5.4 ผลลัพธ์คำตอบภาษาอังกฤษ	55
รูปที่ 5.5 ตัวอย่างการทดสอบขอเอกสารต้นทาง	56
รูปที่ 5.6 ตัวอย่างการค้นหาเอกสารงานทะเบียนด้วยคิวเวิร์ด	56
รูปที่ 5.7 รูปตัวอย่างการลบข้อมูล	57
รูปที่ 5.8 การอัปเดตดัชนี.....	57

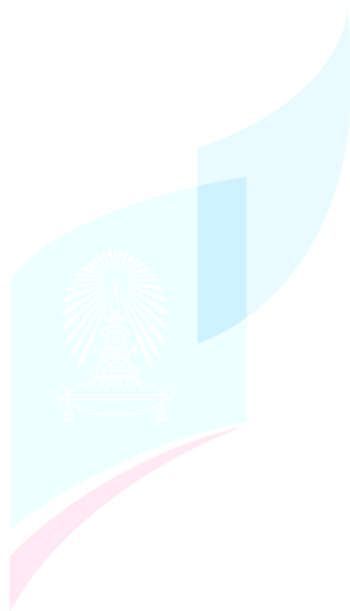


CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

สารบัญตาราง

ตารางที่ 3.1 ตารางปริมาณข้อมูลภายในโดเมนจุฬา.....	22
---	----



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

บทที่ 1 บทนำ

1.1 ภูมิหลังและที่มาของปัญหา

ข้อมูลที่บุคลากรของจุฬาลงกรณ์มหาวิทยาลัยต้องใช้งานในแต่ละวัน กระจายตัวอยู่บนหลายเว็บไซต์ และหลายระบบของหน่วยงานภายในมหาวิทยาลัย เช่น เว็บไซต์สำนักงานทะเบียนกลาง คลังผลงานวิชาการของจุฬาฯ (CUIR) เว็บไซต์ของหอสมุดกลาง รวมถึงเว็บไซต์ของคณะหรือหลักสูตรต่าง ๆ แต่ละแหล่งมีรูปแบบการนำเสนอข้อมูลที่แตกต่างกัน เช่น หน้าเว็บปกติ ไฟล์ PDF เอกสารตาราง รวมถึงอินโฟกราฟิกหรือภาพถ่ายที่มีข้อความสำคัญฝังอยู่ การที่ข้อมูลกระจายตัวเช่นนี้ ทำให้บุคลากรต้องใช้เวลามากในการค้นหาและตรวจสอบข้อมูลจากหลายที่ที่ละแห่ง

อีกทั้งข้อมูลที่เผยแพร่ออกมาจะมีการปรับปรุงเป็นระยะ โดยเฉพาะข้อมูลที่มีความเปลี่ยนแปลงตามรอบปีการศึกษา เช่น ปฏิทินการศึกษา กำหนดการลงทะเบียน หรือประกาศต่าง ๆ ส่งผลให้ในหลายกรณีข้อมูลเก่าและข้อมูลใหม่ปรากฏอยู่ร่วมกัน ในแหล่งเดียวกัน หากผู้ใช้ไม่ระมัดระวังหรือไม่มีระบบช่วยกรอง อาจเกิดความสับสนและอ้างอิงข้อมูลที่ไม่เป็นปัจจุบันได้ง่าย

แรงบันดาลใจของโครงการนี้จึงเกิดจากความต้องการที่จะจัดการกับปัญหาดังกล่าวอย่างเป็นระบบ ด้วยการพัฒนาเครื่องมือที่สามารถรวมข้อมูลจากหลายแหล่งและหลายรูปแบบ ให้สามารถค้นหาและเข้าถึงได้อย่างสะดวก รวดเร็ว และมีความน่าเชื่อถือ โดยเน้นการอ้างอิงกลับไปยังแหล่งข้อมูลต้นทางอย่างโปร่งใส เพื่อลดความซ้ำซ้อนในการค้นหาและลดความคลาดเคลื่อนของข้อมูลที่น่าไปใช้งานจริง

1.2 วัตถุประสงค์ของโครงการ

โครงการนี้มีจุดมุ่งหมายเพื่อพัฒนา ระบบ Retrieval-Augmented Generation (RAG) สำหรับบุคลากรของจุฬาลงกรณ์มหาวิทยาลัย เพื่อให้สามารถเข้าถึงข้อมูลที่กระจายอยู่ตามแหล่งต่าง ๆ ของมหาวิทยาลัยได้อย่างสะดวก รวดเร็ว และมีความน่าเชื่อถือ โดยระบบจะต้องสามารถ ดึงบริบทจากข้อมูลสาธารณะ ของมหาวิทยาลัยหลายโดเมนมารวมกัน และสร้างคำตอบที่มีการ อ้างอิงกลับไปยังแหล่งข้อมูลต้นทาง ได้อย่างชัดเจนและโปร่งใส

ในเชิงภาพรวม ระบบที่พัฒนาขึ้นจะต้องช่วยลดภาระของบุคลากรในการค้นหาและตรวจสอบข้อมูลจากหลายแหล่งที่ละแห่ง โดยเป้าหมายคือการให้ผู้ใช้สามารถถามคำถามเพียงครั้งเดียว แล้วระบบจะค้นคืนบริบทที่เกี่ยวข้องจากเอกสารหรือหน้าเว็บจริงหลายประเภท เช่น HTML PDF ตาราง หรือข้อมูลจากภาพ แล้วจึงสรุปคำตอบที่มีแหล่งอ้างอิงครบถ้วนกลับมา

นอกจากนี้ ระบบยังตั้งเป้าที่จะ รองรับทั้งภาษาไทยและภาษาอังกฤษ เนื่องจากเอกสารของมหาวิทยาลัยมีทั้งสองภาษาอยู่ในหลายส่วนของเว็บไซต์และระบบภายใน และต้องสามารถประเมินคุณภาพการทำงานได้อย่างเป็นระบบ

1.3 ขอบเขตของระบบ

ระบบครอบคลุม “ข้อมูลสาธารณะ” ภายใต้โดเมนของจุฬาฯ ที่เกี่ยวกับงานวิชาการและงานธุรการ เช่น หน่วยงานทะเบียน คลังผลงานวิชาการ เว็บไซต์คณะ/หลักสูตร โดยรองรับหลายรูปแบบไฟล์ที่พบจริง ได้แก่ หน้าเว็บ HTML PDF ตาราง ภาพ และอินโฟกราฟิกที่ต้องผ่าน OCR ระบบทำงานสองภาษา (TH/EN) และจะตอบแบบมีแหล่งอ้างอิงกลับไปยังต้นทางเสมอ ผู้ใช้สามารถกรองผลตามโดเมน ช่วงเวลา ภาษา หรือ ชนิดเอกสารได้ ขอบเขต ไม่รวม เนื้อหาที่ต้องล็อกอิน เนื้อหาภายใต้สัญญาเชิงพาณิชย์ หรือข้อมูลส่วนบุคคล/ ไม่เปิดเผยสาธารณะ

1.4 วิธีดำเนินงาน

1. กำหนดขอบเขต & นโยบายข้อมูล
2. เตรียมและสกัดเนื้อหา
3. ทำความสะอาด จัดรูปแบบ และแบ่งชิ้นส่วน
4. ผังเวกเตอร์ และ สร้างดัชนี
5. คั่นคืน และ ปรับอันดับ
6. สังเคราะห์คำตอบพร้อมอ้างอิง
7. ประเมินผลตามงานจริง

1.5 ประโยชน์ที่คาดว่าจะได้รับ

ระบบนี้ช่วยให้การค้นข้อมูลจากหลายโดเมนของจุฬาฯ ง่ายขึ้นมาก ผู้ใช้พิมพ์คำถามเพียงครั้งเดียวก็ได้คำตอบพร้อมลิงก์แหล่งที่มาที่เปิดดูได้ทันที ลดเวลาการหาเอกสารเอง และลดโอกาสหยิบข้อมูลล้าสมัย เพราะสามารถกดกลับไปตรวจสอบหน้าต้นทางได้เสมอ โดยคำตอบที่ได้มีความน่าเชื่อถือมากขึ้น เนื่องจากระบบเลือกบริบทที่เกี่ยวข้องที่สุดก่อนสรุปคำตอบและแสดงการอ้างอิงอย่างชัดเจน การสื่อสารงานภายในจึงตรงกัน เข้าใจง่าย และลดความคลาดเคลื่อนในการทำงานร่วมกัน

บทที่ 2

งานที่เกี่ยวข้องและเทคโนโลยีที่ใช้

2.1 ภูมิทัศน์ระบบสารสนเทศของจุฬาลงกรณ์มหาวิทยาลัย

ระบบสารสนเทศของจุฬาฯ กระจายตัวอยู่หลายโดเมนตามพันธกิจ (ทะเบียน-การศึกษา งานวิจัย ห้องสมุด/ทรัพยากร ไอทีโครงสร้างพื้นฐาน บุคคล จัดซื้อจัดจ้าง ฯลฯ) และนำเสนอข้อมูลหลากหลายรูปแบบ โดยเฉพาะหน้าเว็บ (HTML) เอกสารทางการ (PDF) และสื่อภาพ (IMG) ซึ่งสอดคล้องกับสถิติใน “รูปประกอบ” ของโครงการที่แสดงจำนวนหน้า/ลิงก์/ไฟล์ต่อโดเมนหลัก เช่น reg car/library research hrm it procurement www.chula.ac.th

2.1.1 แผนที่โดเมนและบทบาทหลัก:

- reg.chula.ac.th (Office of the Registrar) – ศูนย์ข้อมูลทะเบียนและวิชาการ (ปฏิทินการศึกษา ขั้นตอนเอกสาร ตาราง/กำหนดการ ฯลฯ) ซึ่งเผยแพร่ประกาศ/คู่มือ/บริการด้านทะเบียนแก่นิสิตและบุคลากร มีหน้าภาษาอังกฤษอย่างเป็นทางการให้ติดตามข้อมูลล่าสุด
- cuir.car.chula.ac.th (CUIR – Chulalongkorn University Intellectual Repository) – คลังดิจิทัลสำหรับผลงานวิชาการฉบับเต็ม (วิทยานิพนธ์ บทความ เอกสารการสอน ฯลฯ) เปิดให้เข้าถึง/ดาวน์โหลดเพื่อวัตถุประสงค์ทางการศึกษา (ไม่เชิงพาณิชย์) เหมาะอย่างยิ่งแก่การจัดทำ metadata-rich corpora และอ้างอิงต้นทางใน RAG
- car.chula.ac.th และ library.car.chula.ac.th (สำนักงานวิทยทรัพยากร/หอสมุดกลางและเครือข่าย) – พอร์ทัลบริการสารสนเทศของมหาวิทยาลัย และ OPAC/ฐานข้อมูลเครือข่ายห้องสมุด (หนังสือ E-books วิทยานิพนธ์ ฐานข้อมูลดัชนี ฯลฯ) โดยบางทรัพยากรอาจมีข้อกำหนดสิทธิ์การเข้าถึงตามสัญญา
- research.chula.ac.th (Office of Research Affairs) – ศูนย์รวมข่าว/ประกาศ/แพลตฟอร์มสนับสนุนการวิจัย และข้อมูลติดต่อหน่วยงานวิจัยของมหาวิทยาลัย (ภาษาไทย/อังกฤษ)
- procurement.chula.ac.th (ประกาศจัดซื้อจัดจ้าง) – ช่องทางเผยแพร่ข่าวจัดซื้อจัดจ้าง/ TOR/ แผนการจัดซื้อฯ อย่างเป็นทางการของมหาวิทยาลัย (เชื่อมโยงสำนักบริหารการเงิน การบัญชี และการพัสดุ)
- it.chula.ac.th (Office of Information Technology) – บริการไอทีโครงสร้างพื้นฐาน/สนับสนุนผู้ใช้ (Helpdesk/บริการเครือข่าย/ระบบบัญชีผู้ใช้งาน ฯลฯ)

- hrm.chula.ac.th (สำนักบริหารทรัพยากรมนุษย์) – ข้อมูลข่าวสาร บุคคล/สวัสดิการ/การอบรม/รับสมัคร และช่องทางติดต่อหน่วยงาน

2.1.2 ชนิดข้อมูลที่พบโดยทั่วไป:

- เอกสารทางการ (PDF): ปฏิทินการศึกษา ระเบียบ/แนวปฏิบัติ คู่มือบริการ TOR/ประกาศจัดซื้อจัดจ้าง รายงาน/เอกสารวิชาการ แบบฟอร์มราชการ และเอกสารประชาสัมพันธ์จากหน่วยงานต่าง ๆ เช่น Registrar CAR/Library Research ฯลฯ
- หน้าเว็บ (HTML): ข่าวและประกาศ คำอธิบายบริการ ขั้นตอนดำเนินการ คำถามที่พบบ่อย (FAQ) หน้าหลักสูตร/รายวิชา ตารางเวลา/กำหนดการ รายชื่อหน่วยงาน/ช่องทางติดต่อ บทความ/บล็อก หน่วยงาน เช่น Registrar Research IT HRM Procurement และโดเมนคณะ/สำนัก
- สื่อภาพ (IMG/อินโฟกราฟิก): โปสเตอร์/ประกาศประชาสัมพันธ์ อินโฟกราฟิกสรุปขั้นตอนหรือไทม์ไลน์ แผนผัง/ผังงาน แบนเนอร์กิจกรรม/อบรม ภาพสรุปเกณฑ์/เงื่อนไขบริการ ภาพประกอบข่าว/รายงานของหน่วยงานต่างๆ

2.2 แนวทางการทำ RAG ทัวไป

Retrieval-Augmented Generation (RAG) เป็นแนวทางสถาปัตยกรรมที่รวมกระบวนการค้นคืน (retrieval) ของเอกสารหรือแหล่งข้อมูลภายนอกเข้ากับกระบวนการสร้างคำตอบของโมเดลภาษา (generation) โดยมีจุดมุ่งหมายเพื่อนำเอาข้อมูลที่เป็นปัจจุบันหรือเฉพาะโดเมนมาช่วยเสริมความแม่นยำให้กับโมเดล การออกแบบระบบ RAG มักประกอบด้วยโมดูลหลักหลายชั้น ได้แก่ การรวบรวมและประมวลผลเอกสาร (ingestion & preprocessing) การสร้างตัวแทนเวกเตอร์ (embedding) การค้นคืน (retrieval) การเรียงลำดับ (reranking) และการสร้างคำตอบพร้อมอ้างอิง (generation + citation) แนวทางการออกแบบในแต่ละขั้นตอนมีตัวเลือกและ trade-off ที่ต้องพิจารณา เช่น วิธีแบ่งชิ้นข้อมูล (chunking) ประเภทของระบบค้นคืน เช่น vector search, full-text search หรือ hybrid วิธีเลือก embedding model วิธีสร้าง prompt และการจัดการกรณี edge (fallback) รวมถึงกลยุทธ์การอัปเดตฐานข้อมูลให้เป็นปัจจุบัน

องค์ประกอบเหล่านี้ทำให้ระบบ RAG มีความยืดหยุ่นและสามารถปรับให้เหมาะสมกับลักษณะข้อมูลและข้อจำกัดของการใช้งานได้

2.2.1 ความหมายและจุดประสงค์ของ RAG

Retrieval-Augmented Generation (RAG) คือเทคนิคที่ทำให้โมเดลภาษาขนาดใหญ่ (LLM) สามารถดึงข้อมูลจากแหล่งความรู้ภายนอก เช่น เอกสาร ฐานข้อมูล เว็บไซต์ มาประกอบกับคำถามของผู้ใช้ ก่อนสร้างคำตอบ โดยไม่จำกัดให้โมเดลอาศัยแต่ “สิ่งที่เรียนรู้มาแล้ว” เพียงอย่างเดียว โมเดล LLM ปกติทำงานโดยอาศัยน้ำหนัก (parameters) ที่ได้รับการฝึกมาแล้ว ซึ่งมีข้อจำกัดที่ว่าไม่สามารถอัปเดตความรู้ใหม่หลังจาก training ได้ นอกจากนี้โมเดลอาจ “hallucinate” หรือสร้างข้อมูลที่ไม่ถูกต้องได้ เนื่องจากไม่มีการอ้างอิงแหล่งข้อมูลภายนอก

แม้ LLM จะมีประสิทธิภาพสูงในงานภาษาทั่วไป แต่ยังมีข้อจำกัดสำคัญ ได้แก่

- Knowledge cutoff: โมเดลไม่ทราบข้อมูลหรือเหตุการณ์ที่เกิดขึ้นหลังจากจุดตัด (cutoff) ของข้อมูลฝึก
- Hallucination: โมเดลอาจสร้างข้อความที่ “ฟังดูสมเหตุสมผล” แต่ไม่มีมูลความจริง
- ขาดบริบทโดเมนเฉพาะ: สำหรับงานเฉพาะทางหรือหน่วยงาน เช่น ข้อมูลภายในมหาวิทยาลัย ข้อมูลอัปเดตจากเว็บในโดเมนเฉพาะ อาจไม่มีในชุดข้อมูลฝึกของโมเดล

ด้วยเหตุนี้ RAG จึงถูกนำมาใช้เพื่อเติมเต็ม “ความรู้ภายนอก” ให้โมเดล โดยให้โมเดลสามารถอ้างอิงข้อมูลปัจจุบันหรือเฉพาะโดเมนได้

คุณประโยชน์หลักของ RAG คือเสริมข้อมูลภายนอกเพื่อให้คำตอบมีความถูกต้อง / อัปเดต / มีอ้างอิง การผนวกข้อมูลภายนอกเข้าไปช่วยให้ระบบตอบคำถามได้อย่าง แม่นยำและเป็นปัจจุบันมากขึ้น โดยโมเดลไม่จำเป็นต้องฝึกใหม่ทุกครั้งเมื่อมีข้อมูลใหม่ อีกทั้งยังสามารถให้ผู้ใช้ตรวจสอบแหล่งอ้างอิงได้ ช่วยเพิ่มความน่าเชื่อถือของผลลัพธ์ และลดความผิดพลาดจาก hallucination โดยเฉพาะในงานที่เน้นเอกสารเชิงวิชาการ หรือข้อมูลทางราชการ

2.2.2 โครงสร้าง และ โมดูลทั่วไปของระบบ RAG

ระบบ RAG มักถูกออกแบบในลักษณะ modular ซึ่งประกอบด้วยโมดูลหลักหลายชั้นที่ทำงานต่อเนื่องเป็น pipeline เพื่อรองรับตั้งแต่การเตรียมข้อมูลไปจนถึงการให้คำตอบ โดยทั่วไปสามารถแบ่งเป็น 5 ขั้นตอนหลัก ได้แก่ การ Ingestion & Preprocessing, Chunking / Tokenization, Embedding & Indexing, Retrieval & Reranking, และ Generation & Citation อาจมีโมดูลเสริมอื่น เช่น fallback logic, query rewriting ขั้นตอนเหล่านี้มีการไหลของข้อมูล (data flow) ดังนี้ เริ่มจากรวบรวมเอกสาร/ข้อมูลจากแหล่งต่าง ๆ ประมวลผลและปรับให้อยู่ในรูปแบบที่ใช้งานได้ (cleaning, parse, OCR) แบ่งข้อความเป็นชิ้นย่อย (chunks) แล้วสร้าง embedding ของแต่ละชิ้นข้อความ เพื่อจัดเก็บ embedding + metadata ลงในฐานข้อมูลดัชนี (vector index) เมื่อผู้ใช้ส่งคำถาม ระบบจะแปลงคำถามเป็น embedding จะค้นข้อมูล (retrieval) จาก index แล้วเรียงลำดับ (reranker) และคัดเลือกข้อมูลเฉพาะที่เกี่ยวข้อง เพื่อสร้าง prompt ที่

รวมเงื่อนไขที่กำหนด ด้วยข้อมูล retrieved แล้วส่งเข้า LLM เพื่อให้สร้างคำตอบ พร้อมคืนผลพร้อมอ้างอิงถึงแหล่งข้อมูลต้นทาง (citation)

การแยกโมดูลเช่นนี้ช่วยให้ระบบสามารถปรับแต่งหรือทดสอบแต่ละส่วนอย่างอิสระ (modular testing) และอัปเดตส่วนใดส่วนหนึ่งได้โดยไม่กระทบทั้งระบบ เช่น การเปลี่ยน embedding model เพิ่ม reranker หรือสลับกลยุทธ์ chunking เป็นต้น

2.2.3 ตัวเลือกในการออกแบบ (Design Choices)

ในแต่ละโมดูลของระบบ RAG มี “จุดตัดสินใจ” (design choice) ที่สำคัญ ซึ่งมีผลต่อประสิทธิภาพและความแม่นยำของระบบ ดังนี้:

- วิธีแบ่งชิ้นข้อความ (Chunking)

สามารถใช้แบบ fixed-length chunks, sentence-level chunks, semantic chunking (แบ่งตามเนื้อหาหลัก), หรือ chunk แบบ layout-aware เช่น แยกตาราง/อินโฟกราฟิก ข้อดีของ chunk ย่อยคือช่วยลดบริบทซ้อนซ้อนและควบคุมขนาด token ได้ง่าย แต่ข้อเสียคืออาจทำให้ loss of context ถ้าแบ่งไม่ดี

- ประเภทของ Retrieval

มีทางเลือกหลักได้แก่ vector search (ค้นโดย embedding), sparse search / lexical search เช่น BM25, TF-IDF และแบบ hybrid search – ผสมทั้งสองแบบเพื่อให้ได้ผลลัพธ์ที่เหมาะสมกับทั้งคำสำคัญ (keywords) และความหมาย (semantics)

- การใช้ Reranker / Cross-Encoder

หลัง retrieval ขั้นแรก มักใช้โมเดล reranker เช่น cross-encoder ประเมินความเกี่ยวข้องของ chunks ที่ได้มาใหม่ เพื่อจัดลำดับให้ดีขึ้นและลด noise

- การออกแบบ Prompt / Context Augmentation

วิธีการผนวกเนื้อหาที่ดึงมา (retrieved context) กับคำถาม เช่น ใส่ context ก่อนหรือระหว่าง prompt, การเลือกจำนวน chunk ที่ใส่, การใช้ prompt template ที่บังคับให้โมเดลอ้างอิงแหล่งข้อมูล

- กลยุทธ์ fallback / การจัดการกรณีพิเศษ

ในกรณีที่ retrieval ไม่เจอข้อมูลที่เกี่ยวข้อง อาจทำ fallback เช่น ตอบว่า “ไม่พบ” หรือส่ง query ใหม่ (query rewriting) เพื่อกำหนดงานให้ชัดเจน

2.2.4 ประเภทของ RAG และ รูปแบบสถาปัตยกรรมที่หลากหลาย

ในงานวิจัยและการใช้งานจริง มีหลายรูปแบบสถาปัตยกรรม RAG ที่นิยมประยุกต์ใช้ให้เหมาะกับลักษณะข้อมูลหรือข้อจำกัดของระบบ ดังนี้:

- Naive RAG
เป็นรูปแบบพื้นฐานที่สุด ซึ่งทำงานแบบ “Retrieve → Generate” โดยดึง chunk ที่เกี่ยวข้องแล้วส่งให้ LLM สร้างคำตอบทันที
- Advanced / Hybrid RAG
ปรับปรุงด้วยเทคนิคเสริม เช่น การ query rewriting ก่อน retrieval การทำ cascade retrieval (ดึงกว้างแล้วคัดกรอง) การใช้ reranker การทำ context scoring หรือ multi-hop retrieval
- Agentic RAG / Dynamic RAG
ระบบสามารถตัดสินใจระหว่างการดึงข้อมูลเพิ่มเติมระหว่างการ generate (on-demand retrieval) หรือแบ่ง query เป็น sub-query แล้วเรียก retrieval หลายรอบในระหว่างการสร้างคำตอบ
- Multi-source / Heterogeneous RAG
รองรับการดึงข้อมูลจากหลายแหล่งหลายชนิด เอกสาร ฐานข้อมูล กราฟ โดยอาจมี agent แยกย่อยทำงานเฉพาะโดเมนข้อมูล เช่น agent สำหรับฐานข้อมูล agent สำหรับเอกสาร ฯลฯ
- Modular / Plug-in RAG
ออกแบบให้แต่ละโมดูล (ingestion, retriever, reranked และ generator) เป็นส่วนแยกที่สามารถสลับหรืออัปเดตได้ รองรับการทำงานและปรับแต่งแต่ละส่วน
- Hierarchical RAG / RAG แบบเชิงโครงสร้าง
สำหรับเอกสารใหญ่หรือโดเมนที่มีองค์ประกอบชั้น (section, chapter) ใช้ RAG แบบลำดับชั้น (hierarchical) โดยอาจให้ LLM ทำเลือกชั้นก่อน แล้วค่อยเจาะลงในรายละเอียด

2.2.5 ประเด็นที่ต้องพิจารณา ความท้าทาย และ ข้อจำกัด

แม้ระบบ RAG จะช่วยเสริมจุดอ่อนของโมเดลภาษา แต่ก็ยังมีความท้าทายหลายประการที่ผู้พัฒนาต้องเผชิญและออกแบบเพื่อไว้ล่วงหน้า ประเด็นสำคัญได้แก่:

- Latency & Throughput: การดึงข้อมูล (retrieval) และการเรียงลำดับ (reranking) ก่อนส่งให้โมเดลอาจเพิ่มเวลาแฝง (latency) ที่ผู้ใช้รับรู้ได้ จึงจำเป็นต้องมี trade-off ระหว่างคุณภาพผลลัพธ์กับความเร็วในการตอบ
- Hallucination / Misalignment: ถึงแม้จะมี retrieval มาเสริม โมเดลยังอาจ “แต่ง” ข้อความนอกเหนือจากข้อมูลที่ถูกดึงมา (hallucinate) ได้ เมื่อโมเดลสร้างคำตอบที่ไม่สอดคล้องกับเอกสารต้นทาง

- ข้อจำกัดของเอกสาร / ความไม่สมบูรณ์ของข้อมูล: หากเอกสารต้นทางมีการสแกนคุณภาพต่ำ OCR ผิดพลาด layout สลับซับซ้อน หรือข้อมูลไม่ครบ ระบบอาจดึง context ผิดหรือตีความผิด
- Duplicate / Noise / Irrelevant chunks: ชิ้นข้อความซ้ำหรือที่ไม่มีประโยชน์อาจก่อให้เกิดรบกวน (noise) ในผลลัพธ์ จึงต้องมีกลไกกรอง (filtering) หรือ reranker ที่ดี
- สิทธิ์การเข้าถึง / เงื่อนไขการใช้ข้อมูล: บางแหล่งข้อมูลอาจจำกัดการเข้าถึง (ต้องล็อกอิน, อยู่ภายในเครือข่าย, ข้อตกลงลิขสิทธิ์) ทำให้ไม่สามารถดึงเนื้อหาได้อย่างอิสระ
- การประเมินผล / ตัวชี้วัดที่เหมาะสม: การวัดประสิทธิภาพไม่เพียงแต่ดูว่า “คำตอบถูกหรือไม่” แต่ต้องประเมินความสอดคล้อง (faithfulness) ความเกี่ยวข้องของ context อัตราการ hallucination ฯลฯ ซึ่งการตั้งตัวชี้วัดที่เหมาะสมสำหรับโดเมนเฉพาะเป็นเรื่องท้าทาย
- การอัปเดตข้อมูล / การบำรุงรักษาดัชนี: ขณะที่ข้อมูลใหม่ถูกเพิ่มเข้ามา ระบบต้องสามารถอัปเดตฐานข้อมูลเวกเตอร์/ดัชนีได้ (incremental reindex) โดยไม่ทำให้ระบบหยุดชะงักหรือเสียประสิทธิภาพ

2.2.6 แนวทางปฏิบัติที่ดี

เพื่อให้ระบบ RAG มีความเสถียร ใช้งานได้จริง และลดข้อผิดพลาดที่อาจเกิดขึ้น มีแนวทางปฏิบัติที่แนะนำดังนี้:

- แยก pipeline อย่างเป็นโมดูล: แต่ละขั้นตอน (ingestion, chunking, retrieval, reranking และ generation) ควรแยกเป็นโมดูลอิสระ เพื่อให้ทดสอบ ดีบั๊ก และอัปเดตแยกส่วนได้
- ใช้ reranker / filter เพื่อลด noise: หลัง retrieval เบื้องต้น ให้ใช้โมเดล reranker เช่น cross-encoder หรือฟิลเตอร์ตาม metadata (แหล่ง, ความใหม่, ภาษา) เพื่อตัด chunk ที่ไม่เกี่ยวข้อง
- Fallback Strategy: กำหนดพฤติกรรมสำรองเมื่อ retrieval ล้มเหลว เช่น ตอบว่า “ไม่พบข้อมูลที่เกี่ยวข้อง” หรือทำ query rewriting แล้วลองค้นซ้ำ
- บันทึก provenance / trace ของแหล่งที่มา: เก็บ metadata อย่างเช่น source URL chunk ID ตำแหน่งข้อความ เพื่อให้สามารถตรวจสอบย้อนกลับ (auditability) ได้
- อัปเดตฐานข้อมูลแบบ incremental: เมื่อมีเอกสารใหม่ เขียนระบบให้เพิ่ม embedding ใหม่ลงใน index โดยไม่ต้อง rebuild ทั้งหมด
- ตั้ง Threshold / Guardrails: เช่น จำกัดจำนวน chunk ที่ส่งให้ LLM, จำกัดคะแนนความไม่แน่นอน (uncertainty) สูงสุด และตรวจสอบ consistency ก่อนคืนคำตอบ

- Prompt Engineering / Verification Prompt: ใช้ prompt ที่บังคับให้โมเดลอ้างอิงแหล่งข้อมูล หรือ prompt ย่อยช่วยตรวจสอบความถูกต้อง (self-verification)
- ประเมินผลด้วยชุดข้อมูล gold / validation สำหรับโดเมนเฉพาะ: สร้างชุดคำถามจริงในบริบทต่างๆ เพื่อประเมินระบบจริงใจ, แยก testset ต่าง ๆ (คำถามที่มี / ไม่มีคำตอบ)
- Monitoring / Logging / Alerting: ติดตามประสิทธิภาพ (latency, error rate, hallucination rate) แบบ real-time และแจ้งเตือนเมื่อมีปัญหา
- ทดลองปรับพารามิเตอร์ / ablation study: ทดลองค่าต่าง ๆ เช่น Top-K, chunk size, reranker weight เพื่อหา configuration ที่เหมาะสมที่สุด

2.3 เครื่องมือและไลบรารีหลัก

ในการพัฒนาโครงการ RAG นี้ มีการเลือกใช้ไลบรารีสำคัญหลายตัวซึ่งทำหน้าที่ในแต่ละขั้นตอนของ pipeline ตั้งแต่การดึงข้อมูล จนถึงการสกัดข้อความและ OCR ไลบรารีที่นำมาใช้นั้นล้วนแต่มีบทบาทสำคัญต่อการประมวลผลข้อมูลให้เป็นรูปแบบที่เหมาะสมสำหรับ LLM / โมดูล retrieval และ ultimate generation ต่อไป

2.3.1 Playwright

Playwright เป็นไลบรารีสำหรับควบคุมเบราว์เซอร์ เช่น chromium firefox และ webkit ผ่าน API เดียวกันทั้งแบบ synchronous และ asynchronous โดยในบริบทของระบบ RAG นี้ เรานำ Playwright มาใช้สำหรับ “การดึงหน้าเว็บที่มีการเรนเดอร์ด้วย JavaScript” หรือมีเนื้อหาที่โหลดแบบไดนามิก เช่น ข่าวประกาศ คณะหน่วยงาน ซึ่งวิธีการดึงข้อมูลธรรมดาด้วย HTTP request จะไม่สามารถเข้าถึงส่วนที่ถูก render ด้าน client ได้อย่างครบถ้วน Playwright ช่วยให้เราจำลองการเปิดหน้าเว็บแบบเบราว์เซอร์จริง (headless หรือ headful) สามารถรอให้ element ต่าง ๆ โหลดเสร็จก่อนจึงดึง source HTML ได้ ซึ่งช่วยลดปัญหาข้อมูลสูญหายจาก dynamic loading และรองรับการโต้ตอบกับหน้าเว็บ เช่น click, scroll, fill forms ได้ด้วย

จุดเด่นของ Playwright ได้แก่ระบบ auto-waiting (รอ element ให้พร้อมก่อนทำ action) ที่ช่วยลดปัญหา timing errors และ flaky scripts นอกจากนี้ยังสามารถรันแบบขนานในบริบทต่าง ๆ ได้ (หลาย context) ซึ่งสะดวกเมื่อนำไปประยุกต์กับ crawling ขนาดใหญ่

2.3.2 Trafilatura

Trafilatura เป็นไลบรารีสำหรับการสกัดเนื้อหาหลัก (main content) จาก HTML โดยตัด boilerplate เช่น header footer sidebar โฆษณาออก และรักษาโครงสร้างย่อย เช่น ย่อหน้า หัวข้อ รายการ ลิงก์ และ metadata ชื่อเรื่อง/วันที่ มั่นรองรับการดาวน์โหลดเว็บ การแยกข้อความ และแปลงผลลัพธ์เป็นรูปแบบต่าง ๆ เช่น plain text CSV XML ฯลฯ

ในระบบ RAG ของเรา Trafilatura ทำหน้าที่เป็น “cleaning & extraction module” สำหรับ HTML pages ที่เราได้รับผ่าน Playwright หรือ HTTP fetch หลังจากที่ได้ source HTML มาแล้ว เราใช้ Trafilatura เพื่อคัดเฉพาะส่วนเนื้อหาสาระหลัก (ตัด noise) ก่อนส่งต่อไปประมวลผล (tokenization, embedding) การใช้งาน Trafilatura ช่วยให้คุณภาพของเนื้อหาที่ feeding เข้าโมเดลดีขึ้น เพราะลดการปนของเนื้อหาไม่สำคัญ

2.3.3 PyMuPDF (fitz)

PyMuPDF หรือที่มักถูกเรียกว่า fitz เป็นไลบรารีประสิทธิภาพสูงสำหรับการทำงานกับไฟล์ PDF และเอกสารแบบอื่น ๆ (XPS, EPUB ฯลฯ) ไลบรารีนี้รองรับการสกัดข้อความ รูปภาพ metadata และรูปแบบต่าง ๆ ของหน้า PDF ได้อย่างรวดเร็ว โดยโค้ดภายในเขียนใน C ทำให้การประมวลผลมีประสิทธิภาพสูง

ภายในโครงการ RAG นี้ PyMuPDF ถูกใช้เป็น engine หลักในการ “อ่าน/แตกข้อความจาก PDF” สำหรับเอกสารต่าง ๆ เช่น ปฏิทิน กำหนดการ คู่มือ โครงการศึกษาที่อยู่ในรูป PDF เมื่อเปิดไฟล์ PDF ผ่าน PyMuPDF เราสามารถดึงแต่ละหน้า, ดึงข้อความพร้อม layout, ดึงภาพในหน้า, เป็นต้น PyMuPDF ยังรองรับ Table extraction เบื้องต้น และการแปลงหน้าเป็นภาพ (raster) ถ้าจำเป็นต่อ OCR จุดเด่นคือความเร็วและความยืดหยุ่นในการจัดการ PDF ที่มีโครงสร้างซับซ้อน

2.3.4 pymupdf4llm

pymupdf4llm เป็น extension/ยูทิลิตี้เสริมที่สร้างขึ้นบน PyMuPDF เพื่อแปลงหน้า PDF ให้กลายเป็นไฟล์ Markdown พร้อมโครงสร้าง (headers, lists, ตาราง) ซึ่งเหมาะกับการส่งเข้าโมเดล LLM/RAG ไลบรารีนี้จะตรวจจบบรูปแบบของข้อความในหน้า เช่น ข้อความปกติ, หัวข้อ (จาก size/font), ลิสต์, ตาราง และแปลงให้อยู่ในลำดับการอ่านที่ถูกต้องใน Markdown text พร้อมรองรับการแยกภาพ เพื่อเก็บไฟล์รูปภาพแยก

ประโยชน์ของ pymupdf4llm ในโครงการคือช่วย “ลดขั้นตอนการแปลงข้อความ PDF ให้เป็นโครงสร้างที่ LLM เข้าใจง่าย” โดยที่เราไม่ต้องเขียน logic เองทั้งหมด ไลบรารีดูแลเรื่อง layout detection + ordering ให้ ซึ่งช่วยลดความซับซ้อนของโค้ดด้าน parsing

2.3.5 EasyOCR

EasyOCR เป็นไลบรารี OCR (Optical Character Recognition) ที่รองรับหลายภาษา (รวมทั้งภาษาไทย/อังกฤษ) โดยใช้โมเดล deep learning ภายใน ซึ่งสามารถอ่านข้อความจากรูปภาพ, สแกน, อินโฟกราฟิกที่มีข้อความฝังได้ ในโครงการ RAG นี้ เราใช้งาน EasyOCR สำหรับกรณีที่ข้อความฝังอยู่ในรูปภาพภายใน PDF หรือในอินโฟกราฟิกที่แสดงบนเว็บไซต์

จุดเด่นของ EasyOCR คือรองรับหลายภาษา และใช้งานได้โดยไม่ต้องฝึกโมเดลเอง (out-of-box) รวมถึงประสิทธิภาพที่ดีในหลายๆ กรณี อย่างไรก็ตาม OCR มักมีความผิดพลาด (ตัวอักษรผิด, อ่านไม่ครบ) เมื่อคุณภาพของภาพไม่ดี, ภาพเบลอ, ตัวอักษรขนาดเล็ก หรือพื้นหลังซับซ้อน จึงควรมีกระบวนการ post-processing ร่วมด้วย

2.3.6 Tiktoken

tiktoken เป็นไลบรารี tokenizer ประสิทธิภาพสูงซึ่งออกแบบโดย OpenAI เพื่อใช้งานกับโมเดลภาษา (LLM) โดยจะแปลงข้อความเป็นโทเคน และแปลงกลับได้ โดยที่ความแม่นยำในการ mapping ระหว่างข้อความ และ โทเคนนั้นยังคงไว้ซึ่งความสมบูรณ์ของข้อความเริ่มต้น

ในการใช้งานกับระบบ RAG นี้ tiktoken ถูกนำมาประยุกต์ใช้เพื่อ “นับโทเคน” และ “ตัดซังก์” (chunking) ให้กับเนื้อหาที่ผ่านการสกัดข้อความ ก่อนที่จะส่งผ่านเข้า embedding/LLM ขั้นตอนนี้สำคัญมาก เพราะโมเดล LLM มีข้อจำกัดเรื่องความยาวโทเคน (max tokens) จึงต้องแบ่งเนื้อหาให้พอดีและเกิด overlap ที่เหมาะสม tiktoken ให้ API ที่รวดเร็วและมีประสิทธิภาพในการคำนวณว่าข้อความใดเกินขนาดได้ ทำให้ pipeline ของเราไม่เกิด error “token overflow” และควบคุมต้นทุนโทเคน (ราคาตามจำนวนโทเคน) ได้ด้วย

2.3.7 sentence-transformers / BGE-M3 embedding

sentence-transformers เป็นไลบรารีที่นิยมใช้สำหรับสร้าง embedding ของประโยคหรือข้อความ (sentence / passage) โดยอาศัยสถาปัตยกรรม Transformer + pooling layer ซึ่งสามารถใช้งานได้ง่าย เพิ่มประสิทธิภาพในการคำนวณ embedding หลายประโยคพร้อมกัน

ในโครงการนี้ เราเลือกใช้ embedding model ซึ่งเป็นตระกูล BGE-M3 (พัฒนาโดย BAAI / FlagEmbedding) มีคุณสมบัติโดดเด่นเรื่อง Multi-Functionality, Multi-Linguality, Multi-Granularity รองรับ retrieval แบบ dense, multi-vector, sparse พร้อมกัน; รองรับภาษาหลายภาษา; รองรับข้อความตั้งแต่สั้นถึงยาวสูงสุด ~ 8192 โทเคน) BGE-M3 ถูกออกแบบให้เป็นโมเดล embedding เดียวที่สามารถรองรับความต้องการ retrieval แบบต่าง ๆ ได้ในระบบเดียว ลดความซับซ้อนในการโยกโมเดลหลายแบบ

บทบาทของ sentence-transformers + BGE-M3 ใน pipeline คือเป็นโมดูล embedding โดยรับข้อความ (ซังก์) แล้วแปลงเป็นเวกเตอร์ความหมาย (dense vector) พร้อม metadata ที่เกี่ยวข้อง (เช่น id,

source) เวกเตอร์เหล่านี้จะถูกจัดเก็บใน vector database เพื่อใช้ค้นคืน (retrieval) ในขั้นถัดไป ด้วยคุณสมบัติ multi-granularity ของ BGE-M3 ทำให้สามารถรองรับข้อความขนาดต่าง ๆ ได้โดยไม่ต้องสลับโมเดลหลายชุด นอกจากนี้ BGE-M3 ยังสามารถใช้งานร่วมกับ reranker ได้ โดยโมเดล embedding เดียวให้ผล retrieval ที่ดีทั้ง Top-K และ precision ในขั้น reranking

2.3.8 FlagEmbedding

FlagEmbedding เป็นไลบรารีที่พัฒนาโดยชุมชน FlagOpen ซึ่งรวมทั้งโมเดล embedding และ reranker ในชุดเครื่องมือเดียว ไลบรารีนี้มีการปล่อยโมเดล BGE-reranker (cross-encoder) ที่ได้รับการออกแบบมาเพื่อต่อยอดจาก embedding model โดยเฉพาะ เหตุผลในการใช้ FlagEmbedding ในระบบ RAG คือเพื่อปรับปรุงคุณภาพเอกสาร หลังการค้นคืนเบื้องต้น

ในกระบวนการ retrieval แบบ Two-stage คือ เราใช้ embedding model (เช่น BGE) เพื่อดึงเอกสาร (หรือซังก์) ที่มีความใกล้เคียงกับ query แล้วนำผลลัพธ์นั้นมาจัดเรียงใหม่ ด้วย reranker แบบ cross-encoder ของ FlagEmbedding ซึ่งรับ input เป็นคู่ (query, passage) แล้วประเมินความเกี่ยวข้องโดยตรงแทนที่จะต้องแปลงทั้งคู่เป็น embedding แล้วคำนวณ similarity

การจัดอันดับใหม่(re-ranking) ด้วย FlagEmbedding ให้ผลแม่นยำกว่าการค้นแบบใช้ embedding อย่างเดียว เพราะมันสามารถจับรายละเอียดเล็กๆ ระหว่างคำถามกับเนื้อหาได้ดีกว่า

แต่การใช้โมเดลแบบ cross-encoder ซึ่งเป็นเทคนิคที่ FlagEmbedding ใช้ จะต้องใช้ทรัพยากรมาก และใช้เวลานาน เพราะต้องประมวลผลคำถามกับเนื้อหาเป็นคู่ ๆ ทีละคู่ ดังนั้นจึงนิยมใช้กับจำนวนผลลัพธ์ที่จำกัด (Top-K) เพื่อให้ได้สมดุลระหว่างความแม่นยำและความเร็วในการทำงาน

2.3.9 ChromaDB

ChromaDB เป็นฐานข้อมูลเวกเตอร์เปิด (open-source vector database) ที่ออกแบบมาเพื่อใช้งานร่วมกับโมเดลภาษา (LLM) โดยเฉพาะ มีฟีเจอร์ที่สำคัญเช่น การจัดเก็บ embeddings พร้อม metadata, การค้นหา similarity อย่างรวดเร็ว, การบันทึกประวัติ และการกรองตาม metadata ได้

ในการใช้งานกับ pipeline RAG ของเรา ChromaDB ทำหน้าที่เป็น “ที่เก็บ/ตัวค้น” (storage + retrieval) ของเวกเตอร์ที่สร้างจากข้อความ (หลัง chunking + embedding) พร้อมกับ metadata ประกอบ เช่น id, source_url, ช่วงหน้า, ภาษา ฯลฯ เมื่อต้องการค้นคืน เราสามารถส่ง embedding ของ query ไปให้ ChromaDB เพื่อดึง Top-K ซังก์ที่ใกล้เคียง จากนั้นใช้ reranker (FlagEmbedding) จัดลำดับใหม่ก่อนส่งเข้าโมดูล generation

จุดเด่นของ ChromaDB คือความเรียบง่ายในการติดตั้ง/ใช้งาน สามารถทำงานในโหมด local หรือ production ได้ รองรับการ persist data และ integration กับไลบรารี LLM framework หลายตัว เช่น LangChain (LangChain — Chroma integration) นอกจากนี้ ChromaDB ได้รับความนิยมในงาน RAG

tutorials ว่าเป็นทางเลือก vector database ที่ lightweight และเข้ากับ workflow ได้ง่าย (Medium: Simple RAG Using Python, ChromaDB, OpenAI)

2.3.10 OpenAI

OpenAI เป็นผู้ให้บริการโมเดลภาษาขนาดใหญ่ (LLM) ผ่าน API ซึ่งถูกใช้เป็นชั้น generation ในระบบ RAG ของเรา ในภาพรวม โมดูล RAG (Retriever-Augmented Generation) จะรับ query แล้วดึง context ที่เกี่ยวข้องจาก vector store เพื่อสร้าง prompt ร่วมกับ context ก่อนส่งเข้าโมเดล OpenAI เพื่อ “สร้างคำตอบพร้อมอ้างอิง”

การใช้ OpenAI API มีข้อดีคือเข้าถึงโมเดลที่มีประสิทธิภาพ เช่น GPT series, รองรับข้อความภาษาไทย/อังกฤษ และมักให้ผลลัพธ์ที่ fluent และครอบคลุม เมื่อรวมกับ context ที่ดึงมาจากฐานข้อมูลภายใน ระบบจึงสามารถตอบคำถามที่ “อัปเดต” และ “เฉพาะโดเมน” ได้

อย่างไรก็ดี การใช้ OpenAI API มีข้อจำกัดเรื่องต้นทุน (คิดตามจำนวนโทเคน) และข้อจำกัดเรื่องขนาด prompt (max tokens) ดังนั้น pipeline ต้องออกแบบให้ส่งเฉพาะ context ที่สำคัญ โดยคำนวณชั่ง/กรอง context ให้พอดี และต้องระวัง hallucination (โมเดลตอบคำผิดโดยไม่มีแหล่งอ้างอิง) จึงจำเป็นต้องใส่กลไก prompt engineering เช่น “ตอบพร้อมอ้างอิง source_url” และตรวจสอบความสอดคล้องกับ context ที่ถูกดึงมา

CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

บทที่ 3

การวิเคราะห์ระบบและข้อกำหนด (System & Data Requirements)

3.1 กำหนดผู้ใช้เป้าหมาย

- **นิสิตนักศึกษา**

ผู้ใช้กลุ่มนี้มีเป้าหมายเพื่อสืบค้นข้อมูลทางวิชาการที่จำเป็นต่อการดำเนินการด้านการศึกษา เช่น ขั้นตอน กำหนดการ และแบบฟอร์ม โดยต้องการคำตอบสั้น ชัดเจน รองรับภาษาไทยและอังกฤษ และสามารถเปิดดูต้นฉบับจากแหล่งอ้างอิงผ่าน URL ได้โดยตรง เพื่อยืนยันความถูกต้องก่อนตัดสินใจดำเนินการต่อไป

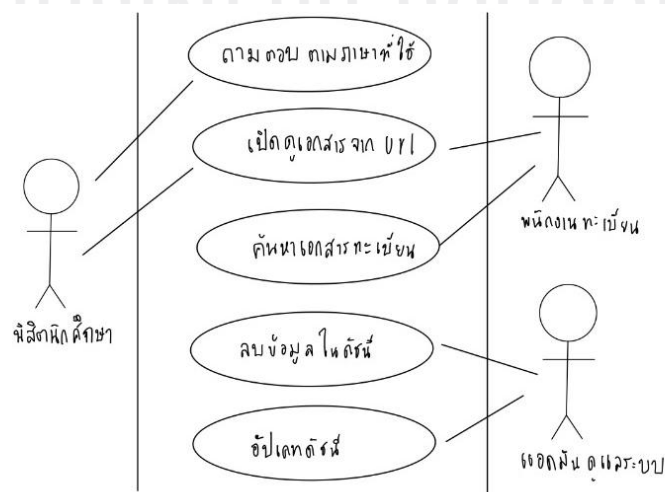
- **เจ้าหน้าที่บริการการศึกษา**

ผู้ใช้กลุ่มนี้มุ่งค้นหาและยืนยันเอกสารหรือประกาศฉบับล่าสุดที่เกี่ยวข้องกับงานทะเบียน เพื่อรองรับการให้บริการนิสิตและการปฏิบัติงานตามระเบียบ โดยต้องเข้าถึงต้นฉบับจาก URL อ้างอิงได้ทันที และสามารถระบุรายการที่ไม่ถูกต้องหรือล้าสมัยเพื่อแจ้งให้ผู้ดูแลระบบดำเนินการปรับปรุงฐานข้อมูลให้เป็นปัจจุบัน

- **แอดมินระบบ**

ผู้ใช้กลุ่มนี้มีเป้าหมายในการดูแลความครบถ้วนและความถูกต้องของดัชนีข้อมูลและเมทาดาทา โดยเฉพาะการจัดการและตรวจสอบค่า URL อ้างอิง (source_url) ให้เชื่อมโยงกับเอกสารต้นฉบับได้ถูกต้อง รวมถึงการลบข้อมูลตาม URL เมื่อมีคำขอแก้ไขหรือถอดถอน ตลอดจนติดตามผลการประเมินออนไลน์เป็นระยะเพื่อรักษาคุณภาพของผลลัพธ์ให้สอดคล้องกับการใช้งานจริงของผู้ใช้ทั้งสองกลุ่มแรก

3.2 กรณีใช้งานหลัก (Use Cases)



รูปที่ 3.1 Use case diagram

3.2.1 UC-01: ถาม-ตอบภาษาธรรมชาติ (TH/EN)

Primary Actor: นิสิตนักศึกษา

Scope: ระบบถาม-ตอบเชิงเอกสาร (RAG)

Level: User-goal

Preconditions: มีดัชนีเวกเตอร์พร้อมใช้งาน

Trigger: ผู้ใช้พิมพ์คำถามสั้น ๆ (ไทยหรืออังกฤษ)

Main Success Scenario:

1. ผู้ใช้ส่งคำถาม
2. ระบบตรวจจับภาษาและค้นคืน Top-K จากคลังเวกเตอร์
3. ระบบจัดลำดับซ้ำ ให้เหลือบริบทที่สัมพันธ์ที่สุด
4. ระบบสรุปคำตอบจากบริบทที่ดึงมา (no-guess)
5. ระบบแสดงคำตอบพร้อมรายการขึ้นข้อความที่เกี่ยวข้องและ source_url

Extensions:

- 2a. ไม่พบบริบทที่เกี่ยวข้องเพียงพอ → ระบบแจ้ง “ไม่พบ/ข้อมูลไม่พอ”

Postconditions (Success): ผู้ใช้ได้รับคำตอบสั้นและมีรายการ source_url สำหรับตรวจสอบต้นฉบับ

Postconditions (Failure): ระบบบันทึกเหตุการณ์และแจ้งแนวทางปรับคำถาม

3.2.2 UC-02: เปิดดูเอกสารต้นฉบับจาก URL

Primary Actor: นิสิตนักศึกษา, เจ้าหน้าที่บริการการศึกษา

Scope: การอ้างอิงต้นฉบับ

Level: Sub-function

Preconditions: ระบบมี source_url ผูกกับขึ้นข้อมูลที่ค้นได้

Trigger: ผู้ใช้เลือกขึ้นผลลัพธ์

Main Success Scenario:

1. ผู้ใช้เลือกขึ้นข้อมูลจากรายการผลลัพธ์
2. ระบบแสดง/ส่งออก source_url ของขึ้นนั้น
3. ผู้ใช้เปิดเอกสารต้นฉบับจาก source_url ในเบราว์เซอร์

Extensions:

- 2a. ขึ้นข้อมูลไม่มี source_url → ระบบแจ้งไม่สามารถเปิดต้นฉบับได้

Postconditions (Success): ผู้ใช้เข้าถึงต้นฉบับเพื่อยืนยันข้อมูล

Postconditions (Failure): ผู้ใช้กลับไปเลือกขึ้นอื่นหรือค้นใหม่

3.2.3 UC-03: ค้นหาเอกสารงานทะเบียนด้วยคีย์เวิร์ด

Primary Actor: เจ้าหน้าที่บริการการศึกษา

Scope: การค้นเอกสารทางการเพื่อใช้งาน

Level: User-goal

Preconditions: ดัชนีवेकเตอร์มีข้อมูลจากแหล่งทางการ

Trigger: ผู้ใช้ป้อนคีย์เวิร์ด/หัวข้อเอกสาร

Main Success Scenario:

1. ผู้ใช้พิมพ์คีย์เวิร์ดสั้น ๆ
2. ระบบค้นคืน Top-K ขึ้นข้อความที่ตรงประเด็น
3. ระบบแสดงรายการขึ้นข้อความพร้อม source_url
4. ผู้ใช้คัดลอก/เปิด source_url เพื่อตรวจสอบเอกสาร

Extensions:

- 2a. ผลลัพธ์ซ้ำซ้อน/คล้ายกันหลายรายการ → ผู้ใช้เลือกฉบับที่ต้องการด้วยตนเอง
- 4a. เอกสารถูกนำออกจากต้นทาง → ผู้ใช้แจ้งแอดมินระบบให้พิจารณาลบตาม URL

Postconditions (Success): ผู้ใช้เข้าถึงเอกสารที่เกี่ยวข้องเพื่อนำไปใช้งานจริง

Postconditions (Failure): ระบบเสนอให้ปรับคีย์เวิร์ดและลองค้นใหม่

3.2.4 UC-04: ลบข้อมูลจากดัชนี

Primary Actor: แอดมินระบบ

Scope: การบำรุงรักษาคลังवेकเตอร์

Level: Admin-goal

Preconditions: ระบุ source_url ที่ต้องการลบได้

Trigger: ได้รับคำขอให้ถอดเอกสาร/แก้ไขความถูกต้อง

Main Success Scenario:

1. แอดมินระบุค่า source_url เป้าหมาย
2. ระบบค้นรายการทั้งหมดที่เกี่ยวข้องในคลัง
3. ระบบลบรายการที่ผูกกับ source_url นั้น
4. ระบบยืนยันผล (จำนวนที่ลบ/จำนวนคงเหลือใน collection)

Extensions:

- 2a. ไม่พบรายการที่ผูกกับ URL → ระบบแจ้งผลเป็นศูนย์รายการ
- 3a. ลบไม่สำเร็จบางส่วน → ระบบรายงานรายการที่ค้างให้ตรวจสอบ

Postconditions (Success): คำค้นในอนาคตจะไม่ค้นขึ้นข้อมูลจาก URL ดังกล่าว

Postconditions (Failure): แอดมินตรวจสอบค่า URL/สิทธิ์และทำรายการใหม่

3.2.5 UC-05: อัปเดตดัชนี

Primary Actor: แอดมินระบบ

Scope: การปรับปรุงเนื้อหาในคลังเวกเตอร์

Level: Admin-goal

Preconditions: มีชุดเอกสารอินพุตพร้อมประมวลผล

Trigger: ต้องการเพิ่ม/อัปเดตข้อมูลในคลัง

Main Success Scenario:

1. แอดมินรันสคริปต์ตัดชิ้นข้อความ (chunking)
2. แอดมินรันสคริปต์ฝังเวกเตอร์ด้วยโมเดลที่กำหนด
3. ระบบบันทึกเวกเตอร์ลง Chroma และรายงานจำนวนที่เพิ่ม
4. แอดมินตรวจสอบ count/บันทึกผลรัน

Extensions:

2a. ฝังเวกเตอร์ล้มเหลวบางส่วน → ระบบบันทึกการผิดพลาดเพื่อตามแก้

3a. มีรายการเดิม → ระบบอัปเดต ตามพฤติกรรมที่ตั้งค่าไว้

Postconditions (Success): ดัชนีพร้อมให้ค้นด้วยข้อมูลล่าสุด

Postconditions (Failure): แอดมินแก้ไขอินพุต/คอนฟิกแล้วรันใหม่

3.3 ข้อกำหนดเชิงหน้าที่ (Functional Requirements)

- Functional Requirements-01: การรับคำถามและภาษาคำตอบ
ระบบต้องรองรับการรับคำถามเป็นภาษาไทยหรือภาษาอังกฤษ และต้องตอบกลับเป็นภาษาที่สอดคล้องกับภาษาของคำถาม
- Functional Requirements -02: การค้นหาข้อความที่เกี่ยวข้องด้วยเวกเตอร์
เมื่อได้รับคำถาม ระบบต้องค้นคืนข้อความที่เกี่ยวข้องสูงสุดตามดัชนีแบบเวกเตอร์ และต้องส่งคืนรายการผลลัพธ์จำนวนไม่น้อยกว่าค่าที่กำหนด พร้อมค่าความสอดคล้องของแต่ละรายการ
- Functional Requirements -03: การจัดลำดับผลลัพธ์
หากเปิดใช้งานการจัดลำดับผลลัพธ์เพิ่มเติม ระบบต้องดึงผลลัพธ์เบื้องต้นจำนวนตามที่กำหนด แล้วจัดลำดับใหม่เพื่อคัดเลือกจำนวนสุดท้ายตามค่าที่กำหนด และต้องแสดงค่าความสอดคล้องหลังจัดลำดับ

- Functional Requirements -04: การสังเคราะห์คำตอบจากบริบทที่ตรวจสอบได้
ระบบต้องสรุปคำตอบจากข้อความที่ค้นได้โดยอ้างอิงหลักฐานจากรายการผลลัพธ์ และในกรณีที่ไม่มีหลักฐานเพียงพอ ระบบต้องระบุอย่างชัดเจนว่าไม่สามารถให้คำตอบตามที่ร้องขอได้
- Functional Requirements -05: การแสดงที่มาของข้อมูล
ระบบต้องแสดงที่อยู่ต้นฉบับ (URL แหล่งที่มา) สำหรับทุกรายการผลลัพธ์ที่ถูกนำมาใช้ เพื่อให้ผู้ใช้สามารถตรวจสอบต้นฉบับได้
- Functional Requirements -06: การเปิดต้นฉบับจากที่อยู่ต้นทาง
เมื่อผู้ใช้เลือกเปิดดูต้นฉบับ ระบบต้องส่งต่อหรือเปิดหน้า/ไฟล์ตามที่อยู่ต้นทางได้ หากไม่มีที่อยู่ต้นทางในรายการนั้น ระบบต้องแสดงข้อความแจ้งให้ทราบ
- Functional Requirements -07: การเตรียมและเพิ่มข้อมูลเข้าสู่คลังค้นหา
ระบบต้องมีขั้นตอนสำหรับตัดข้อความเป็นส่วนย่อย แปลงเป็นเวกเตอร์ และเพิ่มเข้าสู่คลังข้อมูล เพื่อรองรับการค้นหาและการสังเคราะห์คำตอบในภายหลัง
- Functional Requirements -08: การลบข้อมูลตามที่อยู่ต้นฉบับ
ระบบต้องสามารถลบข้อมูลทั้งหมดที่เชื่อมโยงกับที่อยู่ต้นฉบับที่ระบุออกจากคลังข้อมูลได้ และหลังการลบ ผลการค้นหาต้องไม่ปรากฏข้อมูลที่ถูกลบดังกล่าว
- Functional Requirements -09: การทดสอบความพร้อมใช้งาน
ระบบต้องรองรับการทดสอบแบบย่อเพื่อยืนยันความพร้อมใช้งานในภาพรวม และต้องมีการตรวจสอบผลการทดสอบกับครั้งก่อนหน้าเพื่อเฝ้าดูแนวโน้มคุณภาพ

3.4 ข้อกำหนดเชิงคุณภาพและการปฏิบัติงาน (Non-Functional Requirements)

- Non-Functional Requirements -01: ความถูกต้องตามหลักฐาน
ระบบต้องให้คำตอบโดยอ้างอิงข้อมูลที่ค้นพบได้จริง และระบุอย่างชัดเจนเมื่อไม่มีข้อมูลเพียงพอ
- Non-Functional Requirements -02: เวลาตอบสนอง
ระบบต้องตอบกลับภายในเวลาที่เหมาะสมต่อการใช้งานประจำ
- Non-Functional Requirements -03: ความพร้อมใช้งาน
ระบบต้องมีความพร้อมใช้งานตามช่วงเวลาที่หน่วยงานกำหนด
- Non-Functional Requirements -04: การบันทึกเหตุการณ์และการตรวจสอบย้อนหลัง
ระบบต้องมีการบันทึกข้อมูลที่จำเป็นเพื่อรองรับการตรวจสอบและทวนสอบย้อนหลัง
- Non-Functional Requirements -05: ความถูกต้องของที่อยู่ต้นฉบับ
ระบบต้องเก็บและแสดงที่อยู่ต้นฉบับของข้อมูลได้อย่างครบถ้วนและถูกต้อง

- Non-Functional Requirements -06: การลบข้อมูลตามที่อยู่ต้นฉบับ
ระบบต้องสามารถลบข้อมูลที่เกี่ยวข้องกับที่อยู่ต้นฉบับที่ระบุได้อย่างถูกต้อง และไม่ปรากฏอีกหลังจากลบแล้ว
- Non-Functional Requirements -07: ความปลอดภัยของข้อมูลสำคัญ
ระบบต้องไม่เก็บรหัสหรือข้อมูลสำคัญแบบฝังตรงในโปรแกรม และต้องมีการจัดเก็บอย่างปลอดภัย
- Non-Functional Requirements -08: ความสามารถในการทำซ้ำผลลัพธ์
ระบบต้องให้ผลการค้นหาและการจัดลำดับซ้ำได้ เมื่อใช้ข้อมูลและการตั้งค่าเดียวกัน
- Non-Functional Requirements -09: การติดตามคุณภาพอย่างต่อเนื่อง
ระบบต้องมีการทดสอบแบบย่อและการเปรียบเทียบคะแนนในแต่ละรอบ เพื่อเฝ้าระวังความเปลี่ยนแปลงของคุณภาพ

3.5 ข้อกำหนดข้อมูล (Data Requirements)

3.5.1 ขอบเขตและนโยบายข้อมูล

กำหนดกรอบข้อมูลที่ระบบสามารถนำมาใช้ได้ ตลอดจนหลักเกณฑ์ในการรวบรวม ใช้งาน จัดเก็บ และถอดถอน เพื่อให้การดำเนินงานเป็นไปตามกติกาและตรวจสอบได้

- ขอบเขตข้อมูล
 - ใช้เฉพาะข้อมูลสาธารณะที่เข้าถึงได้โดยชอบด้วยกติกาที่กำหนดไว้
 - รองรับเนื้อหาในรูปแบบหน้าเว็บ เอกสาร และภาพที่แปลงข้อความได้
 - รองรับภาษาไทยและภาษาอังกฤษ
 - ไม่รวมข้อมูลที่ต้องลงชื่อเข้าใช้ ข้อมูลส่วนบุคคล และเนื้อหาที่มีข้อจำกัดสิทธิ์
- แหล่งข้อมูลและการอนุญาต
 - จัดทำรายชื่อโดเมนที่อนุญาต (allowlist) และรายการตัดออก (blocklist)
 - ปฏิบัติตามไฟล์กำหนดการเข้าถึงของเว็บไซต์ (เช่น robots.txt) และเงื่อนไขการใช้งานของแหล่งข้อมูล
 - ระบุหน่วยงานเจ้าของเนื้อหาเพื่อใช้อ้างอิงและประสานงานเมื่อมีข้อร้องขอ
- นโยบายการใช้งานและการอ้างสิทธิ์
 - ติดป้ายกำกับสิทธิ์ใช้งานของเอกสาร (เช่น สาธารณะ/มีเงื่อนไข/ไม่ระบุ)
 - ใช้ข้อมูลเพื่อการสืบค้นและอ้างอิงเท่านั้น ไม่ปรับแก้ไขความต้นฉบับ
 - แสดงที่อยู่ต้นฉบับ (URL) ทุกครั้งที่มีการนำเนื้อหาไปใช้ประกอบคำตอบ

- นโยบายการเก็บ รักษา และปรับปรุงข้อมูล
 - กำหนดความถี่ในการเก็บซ้ำตามลักษณะของแหล่งข้อมูล (เช่น รายสัปดาห์/รายเดือน)
 - จัดเก็บเมทาดาตาที่จำเป็นต่อการตรวจสอบย้อนหลัง (เช่น ที่อยู่ต้นฉบับ วันที่เผยแพร่ วันที่เก็บรุ่นข้อมูล)
 - จัดทำสำรองข้อมูลและแนวทางกู้คืนเมื่อเกิดเหตุขัดข้อง
- นโยบายการลบและถอดถอน (ตามที่อยู่ต้นฉบับ)
 - เมื่อได้รับคำขอถอดถอนหรือพบว่าเนื้อหาไม่ควรแสดงผล ต้องลบข้อมูลที่เกี่ยวข้องทั้งหมดที่ผูกกับที่อยู่ต้นฉบับนั้นออกจากคลังค้นหา
 - หลังการลบ ต้องไม่ปรากฏข้อมูลดังกล่าวในผลการค้นหาอีก และบันทึกเหตุการณ์การดำเนินการไว้
- นโยบายความปลอดภัยและความเป็นส่วนตัว
 - ไม่จัดเก็บข้อมูลที่มีความอ่อนไหวหรือข้อมูลส่วนบุคคล
 - จัดเก็บค่าตั้งและรหัสที่เกี่ยวข้องกับระบบในพื้นที่ที่ปลอดภัย ไม่ฝังไว้ในโปรแกรมโดยตรง
 - ปิดบังข้อมูลที่สามารถระบุตัวบุคคลได้ในบันทึกเหตุการณ์
- การตรวจติดตามและทวนสอบ
 - บันทึกเหตุการณ์สำคัญของกระบวนการเก็บและใช้งานข้อมูล เพื่อรองรับการตรวจสอบย้อนหลัง
 - ทบทวนรายการโดเมนที่อนุญาต นโยบายสิทธิ์ใช้งาน และขั้นตอนถอดถอนเป็นระยะ เพื่อให้สอดคล้องกับการเปลี่ยนแปลงของแหล่งข้อมูลและข้อกำหนดของหน่วยงาน

3.5.2 ประเภทและรูปแบบข้อมูลที่รองรับ

กำหนดชนิดเนื้อหาและรูปแบบไฟล์ที่ระบบสามารถนำเข้า แปลง และใช้เพื่อการค้นหาและตอบคำถาม

- ประเภทข้อมูลตามแหล่งกำเนิด
 - ข้อมูลจากหน้าเว็บไซต์สาธารณะ (ข้อความที่มองเห็นได้โดยไม่ต้องลงชื่อเข้าใช้)
 - เอกสารเผยแพร่สาธารณะของหน่วยงาน เช่น ประกาศ ระเบียบ คู่มือ
 - สื่อภาพที่มีข้อความและสามารถแปลงเป็นตัวอักษรได้

- รูปแบบไฟล์หลักที่รองรับ
 - หน้าเว็บ: HTML
 - เอกสาร: PDF (เป็นหลัก) และไฟล์เอกสารทั่วไปที่แปลงเป็นข้อความได้
 - ภาพ: รูปแบบมาตรฐาน เช่น PNG/JPG ที่ผ่านการแปลงข้อความเป็นตัวอักษรก่อนนำเข้า
- การแปลงเป็นข้อความเพื่อใช้งาน
 - หน้าเว็บ: ดึงเฉพาะเนื้อหาส่วนหลัก ตัดเมนู/หัวท้ายซ้ำออก
 - เอกสาร: แยกข้อความจากไฟล์ และรักษาโครงสร้างหัวข้อ/รายการ/ตารางเท่าที่จำเป็น
 - ภาพ: แปลงข้อความจากภาพเป็นตัวอักษร แล้วตรวจทานความสมบูรณ์ก่อนนำเข้า
 - ผลลัพธ์หลังแปลงต้องอยู่ในรูปแบบข้อความพร้อมเมตาดาตาขั้นต่ำ รวมถึงที่อยู่ต้นฉบับ
- ภาษาและการเข้ารหัส
 - รองรับภาษาไทยและภาษาอังกฤษ
 - จัดเก็บข้อความด้วยรูปแบบการเข้ารหัสที่รองรับอักขระไทยอย่างถูกต้อง เช่น UTF-8
 - ปรับรูปแบบตัวเลขและวันที่ให้อยู่ในมาตรฐานเดียวกันเมื่อจำเป็น
- เมตาดาตาขั้นต่ำที่ต้องแนบกับข้อมูล
 - ที่อยู่ต้นฉบับ (URL) และวันที่เก็บ
 - ชื่อเรื่อง/หัวข้อ และชนิดไฟล์
 - ภาษาและหน่วยงานเจ้าของเนื้อหา (ถ้ามีระบุ)
 - รหัสอ้างอิงของชิ้นข้อมูลเมื่อแบ่งเป็นส่วนย่อย
- คุณลักษณะการจัดรูปข้อมูลก่อนจัดเก็บ
 - ล้างส่วนที่ซ้ำซ้อน เช่น ส่วนหัว/ท้าย ค breadcrumbs เมนูนำทาง
 - รักษาโครงสร้างที่จำเป็นต่อความเข้าใจ (หัวข้อย่อย รายการ ตาราง) โดยแปลงให้อ่านได้ในรูปแบบข้อความ
 - ตรวจซ้ำและตัดเนื้อหาที่ซ้ำหรือใกล้เคียงกันออก ลดความซ้ำซ้อนในคลัง
- ข้อจำกัดและขอบเขตที่ไม่ครอบคลุม
 - ไม่รองรับข้อมูลที่ต้องลงชื่อเข้าใช้หรือมีข้อจำกัดสิทธิ์
 - ไม่รับประมวลผลไฟล์ที่ไม่สามารถแปลงเป็นข้อความได้อย่างน่าเชื่อถือ
 - รายละเอียดเชิงเทคนิคเกี่ยวกับตัวแปลงข้อความ วิธีดึงเนื้อหาจากแต่ละชนิดไฟล์ และค่าตั้งเฉพาะ จะอธิบายในบทที่ 4 “การออกแบบและพัฒนา” เพื่อไม่ซ้ำซ้อนกับกรอบข้อกำหนดในหัวข้อนี้

3.5.3 แหล่งข้อมูลและโดเมน

ตารางที่ 3.1 ตารางปริมาณข้อมูลภายในโดเมนจุฬา

โดเมน (Domain)	HTML	Links	PDF	IMG
hrm.chula.ac.th	345	1,289	651	812
it.chula.ac.th	531	949	331	1,502
reg.chula.ac.th	301	759	314	363
car.chula.ac.th	2,880	146,730	398	3,447
library.car.chula.ac.th	1	55	2	7
research.chula.ac.th	340	945	68	4,059
procurement.chula.ac.th	301	397	42	1
cunex.chula.ac.th	0	0	0	0
www.chula.ac.th	8,151	17,308	138	97,908
รวมทั้งหมด (TOTAL)	12,850	168,432	1,944	108,099

กำหนดชุดโดเมนต้นทางให้สอดคล้องกับงานปัจจุบันในโค้ดและรูปประกอบ โดยยึดหลัก “เนื้อหาทางการ เปิดตรวจสอบได้ มีที่อยู่ต้นฉบับ (URL) ชัดเจน และเกี่ยวข้องกับการศึกษา/การทะเบียน/การบริการ นิสิตโดยตรง”

- รายการโดเมนที่อนุญาต
 - reg.chula.ac.th — เว็บไซต์สำนักงานการทะเบียน แหล่งประกาศ ปฏิทินการศึกษา ขั้นตอน และบริการงานทะเบียนโดยตรงของมหาวิทยาลัย จึงเป็นฐานอ้างอิงหลักสำหรับคำถามเชิงระเบียบและกำหนดการของนิสิต
 - www.reg.chula.ac.th — โดเมนรุ่นเดิมของสำนักงานการทะเบียน ซึ่งยังมีหน้า/ลิงก์อ้างอิงที่ถูกใช้แพร่หลาย จึงคงไว้เพื่อให้ครอบคลุมแหล่งเนื้อหาที่เชื่อมโยงมาจากเอกสารหรือคู่มือเดิม
 - it.chula.ac.th — เว็บไซต์หน่วยงานเทคโนโลยีสารสนเทศของมหาวิทยาลัย ให้ข้อมูลบริการด้านไอทีที่นิสิตใช้งานจริง (บัญชี ระบบอีเมล บริการช่วยเหลือ ฯลฯ) ซึ่งมักเชื่อมโยงกับขั้นตอนการเรียนและลงทะเบียนออนไลน์
 - sa.chula.ac.th — เว็บไซต์สำนักบริหารกิจการนิสิต ให้ข้อมูลและแบบฟอร์มบริการนิสิต (ติดต่อ รับรอง เอกสาร/สวัสดิการ) ที่ใช้ประกอบการปฏิบัติงานจริงของผู้เรียน

- โดเมนที่ตัดออก
 - www.chula.ac.th — เว็บไซต์ภาพรวม/ข่าวประชาสัมพันธ์ของมหาวิทยาลัย (ข่าว กิจกรรม บทความ) มีใช้แหล่งหลักของระเบียบ แบบฟอร์ม หรือขั้นตอนงานทะเบียน จึงก่อ “สัญญาณรบกวน” ต่อการสืบค้นเชิงภารกิจ
 - car.chula.ac.th, library.car.chula.ac.th — เว็บไซต์หอสมุดและระบบสืบค้นทรัพยากรสารสนเทศ (OPAC/ฐานข้อมูล) เนื้อหาเป็นรายการทรัพยากรและแบบฟอร์มบริการเฉพาะ ทำให้เกิดหน้าและลิงก์จำนวนมากที่ไม่ใช่ “ประกาศ/ขั้นตอนงานทะเบียน” โดยตรง จึงตัดออกเพื่อความตรงประเด็น
 - hrm.chula.ac.th — เว็บไซต์สำนักบริหารทรัพยากรมนุษย์ เนื้อหามุ่งนโยบาย/สิทธิประโยชน์บุคลากร ไม่ตรงกับกรณีใช้งานหลักของนิสิตหรืองานทะเบียน จึงตัดออก
 - research.chula.ac.th — เว็บไซต์สำนักบริหารวิจัย เน้นข่าว/มาตรฐาน/บริการสนับสนุนนักวิจัย ไม่ใช่แหล่งประกาศหรือขั้นตอนงานทะเบียน จึงตัดออกจากชุดปัจจุบัน
 - procurement.chula.ac.th — พื้นที่ประกาศจัดซื้อจัดจ้างของมหาวิทยาลัย ไม่เกี่ยวข้องกับงานนิสิตหรือทะเบียนโดยตรง จึงตัดออกเพื่อลดเนื้อหาที่ไม่เกี่ยวข้อง
 - cunex.chula.ac.th — ในรอบสำรวจตามรูป ไม่พบหน้าสำหรับนำเข้า (จำนวนเป็นศูนย์) และโดยลักษณะงานเป็นบริการเฉพาะทางที่มักต้องลงชื่อเข้าใช้ จึงไม่รวมในระบบปัจจุบัน (จะพิจารณาอีกครั้งหากมีเนื้อหาสาระณะที่เกี่ยวข้อง) คาดว่าน่าจะต้องมีการเข้าสู่ระบบเพื่อเข้าถึงเนื้อหา
- เหตุผลเชิงปฏิบัติการ
 - ทั้งสามโดเมนมีสัดส่วนเนื้อหาเป็นทางการสูง
 - ตัดโดเมนที่มีเนื้อหาซึ่งเพิ่มปริมาณหน้าและลิงก์จำนวนมากแต่ไม่ช่วยตอบโจทย์หลัก ทำให้การค้นหาคำถามและตรงคำถามมากขึ้น

3.5.4 Metadata ระดับ “เอกสาร”

กำหนดเขตข้อมูลที่ต้องแนบกับ “เอกสารหนึ่งฉบับ” ก่อนการตัดเป็นส่วนข้อความและนำเข้าเพื่อสืบค้น โดยยึดแนวทางการพกพา ที่อยู่ต้นฉบับ เพื่อเปิดต้นฉบับและลบตามที่อยู่ได้ และอิงหลักสากลด้านเมตาดาตาเพื่อความสม่ำเสมอและตรวจสอบย้อนกลับได้

- วัตถุประสงค์ในการเก็บเมตาดาตา
 - ระบุแหล่งที่มาอย่างชัดเจนสำหรับการเปิดต้นฉบับและการถอดถอนตามที่อยู่
 - ควบคุมรุ่น ตรวจสอบการเปลี่ยนแปลง และเชื่อมโยงกับส่วนข้อความที่ตัดออกมา

- วางโครงสร้างให้รองรับการกรอง/จัดการตามเมทาดาตาได้ในอนาคต (เช่น หน่วยงาน ภาษา ชนิดไฟล์)
- รายการเมทาดาตาด้านล่าง
 - document_id — รหัสอ้างอิงภายในของเอกสาร ใช้เชื่อมโยงกับส่วนข้อความทุกส่วน
 - source_url — ที่อยู่ต้นฉบับของเอกสาร ใช้เปิดต้นฉบับ และเป็นกุญแจสำคัญของการลบ “ตามที่อยู่ต้นฉบับ” ในคลังข้อมูลवेक्टर
 - title — ชื่อเรื่อง/หัวข้อของเอกสาร
 - doctype — ชนิดไฟล์ เช่น HTML PDF DOCX ภาพที่แปลงข้อความ
 - lang — ภาษาหลักของเนื้อหา เช่น th, en
 - publish_date — วันที่เผยแพร่ตามต้นฉบับ
 - crawl_date — วันที่เก็บเข้าระบบ
 - content_hash — ค่าตรวจสอบเนื้อหา เช่น แฮชของข้อความหลังแปลง เพื่อการเปลี่ยนแปลงระหว่างรอบเก็บ

* หมายเหตุการจัดวางชื่อฟิลด์: เลือกใช้ชื่อแบบสั้น อ่านตรงความหมาย และคงเสถียร มีแนวคิดกับมาตรฐาน เพื่อให้นำไปใช้ข้ามระบบได้ง่ายในอนาคต

- กฎการบันทึก
 - เวลาทั้งหมดให้บันทึกในรูปแบบเดียวกัน
 - ค่า lang ใช้รหัสสั้นสม่ำเสมอ (th, en)
 - source_url ปรับให้อยู่รูปแบบมาตรฐานเดียวกันก่อนบันทึก
 - หากสกัดบางช่องไม่ได้ ให้คงเขตข้อมูลไว้และเว้นว่าง (ไม่ตัดทิ้ง) เพื่อความสม่ำเสมอของสคีมา
- ความสัมพันธ์กับ “ส่วนข้อความหลังทำการ chunking”
 - ส่วนข้อความทุกชิ้น ต้องพก document_id และ source_url ของเอกสารแม่ เพื่อ:
 - (1) เปิดต้นฉบับจากผลลัพธ์ได้ทุกชิ้น
 - (2) ลบทั้งเอกสารได้ในคราวเดียวด้วยเงื่อนไขที่อยู่ต้นฉบับ (ลบตาม URL) ในคลังवेक्टर

บทที่ 4

การออกแบบและการพัฒนา (Design & Implementation)

4.1 ภาพรวมสถาปัตยกรรมการทำงาน

สถาปัตยกรรมของระบบ RAG สำหรับภาษา ถูกออกแบบเป็น สองเส้นทางหลักที่ทำงานร่วมกัน คือ 1. เส้นทางประมวลผลออนไลน์เพื่อดึงข้อมูล และ 2. เส้นทางตอบคำถามแบบเรียลไทม์ (Online Query Path) โดยทั้งสองเส้นทางแชร์ ฐานข้อมูลเวกเตอร์ (Vector DB) และ เมทาเดตา ร่วมกัน เพื่อให้การสืบค้นมีความถูกต้อง โปร่งใส ตรวจสอบย้อนกลับได้ และรองรับบริบทสองภาษา (TH/EN) อย่างสอดคล้องกับโดเมนเนื้อหาของภาษา

ภาพรวมการทำงานเริ่มจากการดึงข้อมูลสาธารณะจากโดเมนที่ได้รับอนุญาต ผ่านโมดูลรวบรวมข้อมูล จากนั้นสกัดเนื้อหาจากหน้าเว็บ เอกสารPDF และภาพ พร้อมทำโอซีอาร์เมื่อจำเป็น แล้วดำเนินการทำความสะอาดและกำหนดคุณภาพของข้อความด้วยการทำให้รูปแบบสม่ำเสมอ การตัดเนื้อหาซ้ำ การกำหนดที่อยู่อ้างอิงมาตรฐาน การตรวจจับภาษา ต่อด้วยการแบ่งชิ้นข้อความตามโปรไฟล์ที่เหมาะสมสำหรับเนื้อหา เมื่อได้ชิ้นข้อความแล้วจึงทำการฝังเวกเตอร์และสร้างดัชนีในฐานข้อมูลเวกเตอร์ ขั้นตอนการใช้งานสำหรับการตอบคำถามเริ่มจากรับคำถามของผู้ใช้ ตรวจจับภาษาและคำถาม ดึงเอกสาร จากนั้นจัดอันดับซ้ำด้วยตัวจัดลำดับเพื่อให้ได้บริบทที่สัมพันธ์ที่สุด แล้วสังเคราะห์คำตอบจากหลักฐาน พร้อมระบุการอ้างอิงและที่อยู่ต้นฉบับสุดท้ายบันทึกตัวชี้วัดการทำงานเพื่อนำไปตรวจสอบคุณภาพและใช้ปรับปรุงระบบในรอบถัดไป

หากผู้อ่านต้องการดูซอร์สโค้ดการทำงานและผลการทำงานสามารถดูได้โดยภาคผนวกในส่วยท้ายของรายงาน

4.2 การรวบรวมข้อมูลจากหน้าเว็บ

4.2.1 ภาพรวมการไหลของงาน

ขั้น “ดึงข้อมูลดิบ (RAW)” คือด่านแรกของสายพาน RAG ที่รับผิดชอบดึงหน้าเว็บและไฟล์สื่อจากโดเมนที่กำหนด แล้วบันทึกหลักฐานการเก็บ (manifest) เพื่อรองรับการสกัดข้อความและสร้างดัชนีในบทยกถัดไป แนวคิดหลักคือการแยกโมดูล ingestion ออกจากขั้น retrieval/ reranking/ generation ให้ตรวจสอบได้และสลับส่วนประกอบได้ง่ายตามหลัก Modular/Plug-in RAG ตามแนวทางที่วางไว้ในบทที่ 1 ซึ่งช่วยให้ระบบทดลอง เปลี่ยนรุ่นเครื่องมือ หรือยกระดับบางส่วนได้โดยไม่กระทบทั้งสายพานงานทั้งหมด

ขอบเขตการทำงานถูกล็อกด้วยรายการโดเมนที่อนุญาต (ALLOWED_HOSTS) และชุดจุดเริ่มต้น (START_URLS) ที่กำหนดไว้ในคอนฟิก สคริปต์จะเตรียมโครงสร้างเอาต์พุตสำหรับ HTML PDF และรูปภาพ รวมถึงพารามิเตอร์ของเมทาเดตา ที่ใช้บันทึกรายการหน้าและไฟล์ที่พบ โดยกำหนดเพดานจำนวนหน้า (MAX_PAGES) และความลึกของการไต่ลิงก์ (MAX_DEPTH) เพื่อคุมขนาดงานและเวลาประมวลผลให้เหมาะสมกับบริบทงานจริงของโดเมนที่เลือกไว้

เมื่อเริ่มทำงาน ระบบจะคิว URL ตั้งต้นจาก START_URLS แล้วขยายรายการจากไฟล์ sitemap ที่เกี่ยวข้อง หากพบว่า seed URL ชี้ไปยังไฟล์ XML (เช่น sitemap หรือดัชนีของ sitemap) จะทำการไล่ตักขึ้นอย่างปลอดภัยจนได้ URL หน้าเนื้อหาที่แท้จริง ก่อนจะเข้าคิวรอประมวลผล โดยใช้ allowlist เป็นด่านคัดกรองตั้งแต่ต้นทางเพื่อลดสิ่งรบกวนและคงความเกี่ยวข้องกับขอบเขตงานที่กำหนดไว้ในรายงานนี้

การดึงหน้า (fetch) ใช้เบราว์เซอร์จริงผ่าน Playwright เพื่อให้ได้ HTML หลังการเรนเดอร์ หนึ่งหน้าจะถูกเปิด ตรวจสอบสถานะ และนำ HTML ที่ได้ไปบันทึกลงโฟลเดอร์ output/raw/html พร้อมทั้งบันทึกรายการหน้า (record = “page”) ลงใน manifest.jsonl เพื่อเก็บ URL ต้นทาง URL แบบ canonical พารไฟล์ และเวลาเก็บ การใช้เบราว์เซอร์จริงทำให้หน้าไดนามิกโหลดองค์ประกอบเสร็จสมบูรณ์ ลดปัญหาหน้าขาดส่วนสำคัญซึ่งจะเอื้อต่อการสกัดในบทถัดไป

หลังบันทึกหน้าแล้ว สคริปต์จะสกัดลิงก์ภายในเพื่อหาหน้าถัดไปและทรัพยากรสื่อ (PDF/รูป) จากนั้นใช้แนวทาง “RAW-first” ดาวน์โหลดไฟล์ PDF และรูปภาพทันที พร้อมระบุชื่อไฟล์ด้วยแฮชของ URL (SHA-1) เพื่อให้การอ้างอิงกลับทำได้เสถียร โดยทุกไฟล์ที่เก็บจะถูกลงบันทึกเป็นรายการ “asset” ใน manifest ระบุชนิดไฟล์ ต้นทาง พารเก็บ ที่มาของหน้า และเวลา ทั้งหมดนี้ช่วยให้ตรวจสอบย้อนกลับได้สะดวกในส่วนสรุปผลของบทรายงาน และเป็นฐานข้อมูลกลางให้ขึ้นสกัดข้อความในบท 4.3 ใช้งานต่อเนื่องอย่างเป็นระบบ

เพื่อให้ข้อมูลคงที่และลดสำเนาซ้ำ URL ทุกตัวจะถูกทำให้เป็นรูปแบบมาตรฐาน (canonical) ทั้งในขั้นตอนบันทึกหน้าและก่อนการเข้าคิวลิงก์ใหม่ โดยตัดพารามิเตอร์ติดตาม บังคับการใช้ https ในกรณีเหมาะสม และทำ path ให้เป็นปกติ (เช่นตัด /index.html) รวมถึงพิจารณา <link rel=“canonical”> ของหน้าในกรณีที่เว็บระบุไว้เอง กลไกนี้ช่วยให้การนับ “หน้าเดียวกัน” ไม่ถูกปะปนด้วยรูปแบบ URL ต่างกันเล็กน้อย ซึ่งจะอธิบายรายละเอียดเงื่อนไขและตัวอย่างในหัวข้อ 4.2.3 ต่อไป

มาตรการด้านความสุภาพต่อเซิร์ฟเวอร์และเสถียรภาพงานประกอบด้วยตัวจำกัดอัตราการร้องขอ (Rate Limiter) แบบต่อโฮสต์ และนโยบายไม่ได้ลิงก์เกินเพดานความลึก/จำนวนหน้า ทุกครั้งก่อนร้องขอไฟล์สื่อจะรอให้ถึงคิวของโฮสต์นั้น ๆ เพื่อลดโอกาสถูกปฏิเสธหรือเกิด timeout เมื่อประมวลผลแต่ละหน้าสำเร็จ หากยังไม่แตะเพดานความลึก ระบบจะเข้าคิวลิงก์ใหม่ที่ผ่านการทำ canonical และ allowlist แล้ว พร้อมบันทึกความคืบหน้าเป็นช่วง ๆ และปิดท้ายด้วยบันทึก “step1_done” เพื่อยืนยันจำนวนงานที่ทำจริงในการสรุปผลของบทนี้

ผลลัพธ์ที่ได้จากขั้นนี้จึงประกอบด้วย 1. ชุดไฟล์ RAW ได้แก่ HTML/ PDF/ รูปภาพ ที่เก็บในโฟลเดอร์เฉพาะ และ 2. แฟ้ม manifest.jsonl ซึ่งเป็นสารบัญหลักฐานการเก็บทั้งหมด โดยผูกความสัมพันธ์ระหว่างหน้า ต้นทาง และไฟล์สื่อไว้อย่างครบถ้วน ข้อมูลชุดนี้จะถูกส่งต่อให้กระบวนการ “สกัดข้อความและจัดโครงสร้าง” เพื่อรวมเป็น documents.jsonl ในบท 4.3 ตามกรอบการออกแบบระบบ RAG ในบทที่ 1 ที่เน้นความตรวจสอบได้และการแยกโมดูลอย่างเป็นระเบียบ

4.2.2 การตั้งค่า (CFG) และผังเอาต์พุต

ส่วนการตั้งค่าถูกออกแบบให้เป็น “จุดศูนย์กลางของขอบเขตงานและเส้นทางข้อมูล” เพื่อให้สายพาน RAG แยกส่วนได้จริงตามหลัก Modular/Plug-in RAG ในบทที่ 1 กล่าวคือ เรากำหนดโดเมนที่อนุญาต จุดเริ่มการเก็บข้อมูล พารามิเตอร์เก็บผลลัพธ์ และพารามิเตอร์ควบคุมการรวบรวมไว้ในคลาสเดียว ทำให้ย้าย ปรับ หรือทดแทนองค์ประกอบปลายทางได้โดยไม่ต้องแก้โค้ดหลายจุด และยังรักษาความสามารถในการตรวจสอบย้อนกลับจากรายงานไปยังไฟล์จริงได้โดยตรง

ขอบเขตของโดเมนและเมล็ดลิ่งเริ่มต้นระบุผ่าน ALLOWED_HOSTS และ START_URLS เพื่อให้ตัวเก็บข้อมูลไม่ไหลออกนอกบริบทจริงของงาน แม้ลิงก์จะอ้อมไปหน้าอื่น ระบบยังตรวจสอบ host_allowed ก่อนเข้าคิวเสมอ ผลคือคิวงานจะสะอาดและสัมพันธ์กับพื้นที่สาธารณะของจุฬาฯ ที่กำหนดไว้ในรายงาน ไม่สร้างข้อมูลขยะหรือความเสี่ยงเกินจำเป็นในภายหลัง

รูปแบบผังเอาต์พุตกำหนดเส้นทางแยกตามชนิดข้อมูล ได้แก่ OUT_HTML สำหรับหน้าเว็บที่เรนเดอร์แล้ว OUT_PDF สำหรับไฟล์เอกสาร และ OUT_IMG สำหรับไฟล์รูปภาพ พร้อมโฟลเดอร์รวมกลาง OUT_STAGE ซึ่งเก็บแฟ้มเมตาาดา โดยมีฟังก์ชัน ensure_dirs สร้างโครงโฟลเดอร์ให้พร้อมใช้งานตั้งแต่ต้น การแยกพารตามชนิดช่วยให้ขั้นสกัดในบท 4.3 อ่านและประมวลผลแบบ “ตามสาย” ได้สะดวกและลดการสแกนซ้ำที่ไม่จำเป็นในโค้ดถัดไป

แฟ้ม MANIFEST เป็นสมุดทะเบียนกลางของทุกสิ่งที่เก็บ ทั้งหน้าและทรัพยากร (asset) โดยบันทึกชนิด ต้นทาง พารไฟล์ และเวลาบันทึก ข้อมูลใน manifest นี้เป็นสะพานเชื่อมไปยังไฟล์ DOCS ซึ่งเป็นเป้าหมายเอาต์พุตของกระบวนการสกัดในบท 4.3 ต่อไป การคงพาร MANIFEST และ DOCS ไว้อย่างชัดเจนเปิดทางให้สรุปผลในรายงานถึงสถิติงานเก็บข้อมูลได้ทันทีโดยไม่ต้องสแกนโฟลเดอร์ซ้ำซ้อน

พารามิเตอร์ควบคุมการรวบรวม เช่น MAX_PAGES และ MAX_DEPTH ทำหน้าที่เป็นเพดานเพื่อควบคุมเวลารันและขอบเขตเชิงปริมาณของงาน ในขณะที่ PER_HOST_RPS เป็นพื้นฐานของตัวจำกัดอัตราแบบต่อโฮสต์ซึ่งใช้จริงในขั้นดึงข้อมูลและดาวน์โหลดไฟล์ ลดโอกาสถูกปฏิเสธการเชื่อมต่อและเสถียรความคืบหน้า ส่วนตัวเลือกการเรนเดอร์และรอโหลดหน้าอย่าง HEADLESS, NAV_TIMEOUT_MS และ WAIT_SELECTOR ทำให้ได้ HTML หลังเรนเดอร์ที่พร้อมต่อการสกัด โดยไม่ต้องลงรายละเอียดของเครื่องมือดึงหน้าในหัวข้อนี้ (จะอธิบายใน 4.2.5)

โดยสรุป บทนี้ทำหน้าที่กำหนด “ขอบเขต-เส้นทาง-กติกา” ของการรวบรวมข้อมูลให้กระชับพอสำหรับงานจริง และพอเพียงสำหรับการต่อยอดไปยังหัวข้อถัดไปโดยไม่เกิดการอธิบายซ้ำ ใน 4.2.3 จะลงรายละเอียดเรื่องการทำให้ URL ให้เป็นมาตรฐานเดียวก่อนเข้าคิว ใน 4.2.4 จะกล่าวถึงการแตกชั้น sitemap และการต่อคิวอย่างปลอดภัย และใน 4.2.5 จะอธิบายการดึงหน้าแบบเรนเดอร์จริงและประเด็นการรอโหลดองค์ประกอบหน้าที่จำเป็นต่อคุณภาพการสกัด

4.2.3 การทำ URL ให้เป็นมาตรฐานเดียว (Canonicalization)

หัวข้อนี้มีเป้าหมายเพื่อให้ลิงก์ที่ “อ้างอิงเนื้อหาเดียวกัน” ถูกแทนด้วยสตริง URL เดียวอย่างสม่ำเสมอ ลดข้อมูลซ้ำซ้อนระหว่างการเก็บหน้า การดาวน์โหลดไฟล์สื่อ และการทำดัชนีในขั้นถัดไป ทั้งยังทำให้การตั้งชื่อไฟล์ตามแฮชของ URL มีเสถียรภาพ และรองรับการ “ลบ/อัปเดตตาม URL” ในอนาคตได้จริง หลักการออกแบบคือจัดชั้นการปกติรูป (normalization) ให้ชัดเจน ได้แก่ การรวมลิงก์อย่างปลอดภัย (safe join), การทำความสะอาดคิรืสตริงจากพารามิเตอร์ติดตาม และการทำให้โครงสร้าง URL เป็นรูปแบบเดียวกัน (สคีม โสสต์ พาท และส่วนย่อยอื่น ๆ) พร้อมทั้งพิจารณา canonical ที่ผู้ให้บริการเว็บประกาศในหน้าเมื่อเหมาะสม

การรวมลิงก์อย่างปลอดภัยเริ่มจากการปฏิเสธสคีมที่ไม่ใช่ http/https เช่น javascript:, data:, file:, [mailto:](#) และ [tel:](#) จากนั้นใช้ตัวแกะส่วนประกอบ URL เพื่อรวม base_url กับ href ให้เป็น absolute URL โดยถอดทอน fragment (#...) ออกเสมอ เนื่องจากส่วนนี้ไม่มีผลต่อการร้องขอทรัพยากรจริง การรวมจะถูกครอบไว้ด้วยนโยบายโดเมนที่อนุญาต หากโฮสต์ปลายทางไม่อยู่ในรายการที่กำหนดจะยุติการเข้าคิวเพื่อคงความปลอดภัยของขอบเขตงานและลดสัญญาณรบกวนตั้งแต่ต้นทาง

ในขั้นการทำความสะอาดคิรืสตริง ระบบจะลบพารามิเตอร์ที่มีวัตถุประสงค์ติดตามหรือทำให้เกิด URL แยกแขนงโดยไม่เปลี่ยนเนื้อหา เช่น utm_source, utm_medium, utm_campaign, utm_content, gclid, fbclid, dclid, icid รวมถึงพารามิเตอร์เซสชันที่ทำให้มีข้อมูลซ้ำได้ ทั้งนี้เพื่อคงเฉพาะคีย์ที่มีความหมายต่อการเข้าถึงเนื้อหา หากมีความจำเป็นต้องคงพารามิเตอร์บางชนิดไว้ (เช่น page, q, lang) ให้กำหนดเป็น allowlist ที่ชัดเจน ผลลัพธ์ของขั้นนี้คือชุดคิรืที่จัดเรียงตามชื่อคีย์และไม่มีรายการซ้ำ ช่วยให้การเปรียบเทียบ URL เป็นการเปรียบเทียบสตริงอย่างตรงไปตรงมา

การทำให้โครงสร้าง URL เป็นมาตรฐานเดียวเน้นความสม่ำเสมอและความสอดคล้องกับพฤติกรรมของเว็บทั่วไป โดยบังคับใช้ https เมื่อเป็นไปได้ ลดการใช้พอร์ตปริยาย (ตัด :80 เมื่อเป็น http และ :443 เมื่อเป็น https) ทำโฮสต์ให้เป็นตัวพิมพ์เล็ก และเข้ารหัสโฮสต์ที่เป็นโดเมนนานาชาติด้วยมาตรฐาน IDNA เพื่อให้เป็น ASCII ทั้งหมด สำหรับพารจะทำการทำให้เป็นปกติด้วยการลบส่วน . และ .. ที่ไม่จำเป็น ลดเครื่องหมายทับซ้ำ และกำหนดกติกากับ index.html/index.php โดยแปลงไปเป็นรูทของพารเดียวกันเพื่อหลีกเลี่ยงการมีสอง URL ซี่ไปยังเนื้อหาเดียวกัน ทั้งยังคุมตรรกะของเครื่องหมายทับท้ายให้คงที่ (เช่น อนุญาตทับท้ายเฉพาะรูทของโฮสต์) ซึ่งแนวทางนี้ลดความคลุมเครือระหว่าง “ไดเรกทอรี” กับ “ไฟล์” ลงอย่างมีนัยสำคัญ

การอ้างอิง canonical จากเอกสาร HTML ใช้เพื่อยอมรับเจตนาของเจ้าของเว็บเมื่อพบ <link rel="canonical" href="..."> โดยจะใช้เฉพาะกรณีที่ URL นั้นยังอยู่ในขอบเขตโดเมนที่อนุญาตและไม่ขัดกับกฎความปลอดภัย ในกรณีที่ canonical อ้างออกนอกโดเมนหรือมีสคีม/พอร์ตผิดปกติ ระบบจะเพิกเฉยและยึดตาม canonical ที่คำนวณได้เองจากกฎก่อนหน้าเพื่อคงความปลอดภัยและความคงเสถียรของดาต้าเลเยอร์

เพื่อลดการอธิบายซ้ำซ้อนกับหัวข้อถัดไป ขั้นตอนนี้จะกล่าวเพียงตรรกะหลักและสัญญาณประสิทธิของผลลัพธ์ ส่วนการประยุกต์ใช้งานจริงจะเชื่อมโยงในสองจุดสำคัญ คือการเข้าคิวลิงก์ในหัวข้อ 4.2.4 ซึ่งจะเรียกใช้

canonicalization ก่อนบันทึกเข้าสู่คิวทุกครั้ง และการบันทึก manifest/การตั้งชื่อไฟล์ในหัวข้อ ต่อไป ซึ่งจะพึ่งพา URL หลัง canonicalization เพื่อให้เกิดตัวระบุ (identifier) ที่เสถียรในระบบทั้งหมด

4.2.4 การรวบรวม Seed จาก Sitemaps

วัตถุประสงค์ของส่วนนี้คือสร้าง “คิวตั้งต้นที่สะอาด ครอบคลุม และตรวจสอบย้อนกลับได้” ก่อนเข้าสู่ขั้นตอนหน้าแบบเรนเดอร์จริงในหัวข้อ 4.2.5 แนวคิดหลักคืออาศัย sitemap ที่ระบุโดยผู้ดูแลเว็บไซต์เป็นแหล่งอ้างอิงทางการ เพื่อลดการพึ่งพาการไล่ลิงก์ภายในหน้า HTML ตั้งแต่รอบแรก โดยขั้นตอนจะเริ่มจากตรวจสอบ robots.txt เพื่อดึงรายการ Sitemap: ที่ประกาศไว้ แล้วเติมจุดตรวจมาตรฐานอย่าง /sitemap.xml, /sitemap_index.xml และ /wp-sitemap.xml จากนั้นจึงอ่านไฟล์ sitemap เหล่านี้เพื่อขยายเป็นรายการหน้าเนื้อหา ก่อนจะรวมเข้ากับ START_URLS แล้วทำ canonicalization และกรองด้วย allowlist อีกรอบเพื่อได้เซต URL เริ่มต้นที่พร้อมใช้งานจริงในงานเก็บข้อมูลของเรา

กลไกค้นหา sitemap เริ่มจากการร้องขอ robots.txt ด้วยฟังก์ชัน fetch_text และสแกนบรรทัดที่ขึ้นต้นด้วย Sitemap: เพื่อรวบรวมรายการลิงก์ จากนั้นระบบจะเติมพารามิเตอร์มาตรฐานสามตำแหน่งไว้เสมอ เพื่อกรณีที่ผู้ดูแลไม่ได้ประกาศใน robots.txt จุดนี้ช่วยเพิ่ม recall ของ seed โดยไม่เสี่ยงขยายออกนอกโดเมน เพราะทุก URL จะถูกตรวจผ่าน allowlist อีกชั้นในภายหลัง การเลือกใช้ fetch_text ที่ส่งคืนสตริงเมื่อสถานะคำขอเป็น 200 เท่านั้น ยังช่วยตัดความผิดพลาดเบื้องต้นออกจากสายพานงาน

เมื่อได้ลิงก์ sitemap แล้ว ระบบจะอ่านไฟล์ XML และดึงค่า <loc>...</loc> ทุกตัวออกมาเป็นรายการ URL โดยครอบไว้ด้วยการตรวจ host_allowed เพื่อจำกัดให้อยู่ในขอบเขตโดเมนที่กำหนด แนวทางนี้รองรับทั้ง sitemap ประเภท URL set และ sitemap index ที่มี <loc> ชี้ไปยังไฟล์ลูกหลายชุดได้ในแบบเดียวกัน ผลลัพธ์ของการแปลงนี้ถูกจำกัดจำนวนสูงสุดต่อไฟล์เพื่อป้องกันการระเบิดของคิวงานในกรณีที่มีการประกาศลิงก์จำนวนมากผิดปกติบนฝั่งเว็บเป้าหมาย

กระบวนการสุดท้ายของการรวบรวม seed จะรวมรายการที่ได้จากทุก sitemap กับ START_URLS แล้วทำ canonicalization ทีละตัวเพื่อให้ได้ตัวแทน URL ที่สม่ำเสมอ จากนั้นจึงกรองซ้ำด้วย allowlist และชุด seen เพื่อกำจัดรายการที่ซ้ำกันทุกกรณี ก่อนคืนลิสต์ seed ที่พร้อมส่งเข้าคิวประมวลผลในขั้นถัดไป ประโยชน์เชิงระบบของขั้นนี้คือทำให้การตั้งชื่อไฟล์ตามแฮชของ URL เสถียร การนับสถิติใน manifest ไม่บิดเบือนจากความแตกต่างด้านรูปแบบ URL และลดโอกาสไล่ลิงก์วนซ้ำในลูบตั้งหน้า

4.2.5 ดึง HTML แบบเรนเดอร์จริง (Playwright) และ Content-Type Gate

หัวข้อนี้อธิบายกลไก “ดึงหน้าเว็บหลังการเรนเดอร์จริง” ด้วย Playwright ควบคู่กับด่านคัดกรองชนิดเนื้อหา (Content-Type gate) เพื่อให้ได้ HTML ที่พร้อมต่อการสกัดในบท 4.3 โดยรักษาความสุภาพต่อเซิร์ฟเวอร์และความเสถียรของสายพานงาน การทำงานเริ่มจากการเปิดเบราว์เซอร์ Chromium ในโหมด headless สร้าง context และหน้าใหม่หนึ่งหน้า แล้วจึงเข้าสู่ลูบประมวลผลคิว URL ที่เตรียมจาก 4.2.4 โดย

ทุกคำขอจะผ่านตัวจำกัดอัตราต่อโฮสต์ก่อน เพื่อไม่ยั้งเกินจำเป็นและลดโอกาส timeout ระหว่างนำทางไปยังหน้าเป้าหมาย

เมื่อเรียกไปยัง URL ระบบจะรอจนถึงสถานะ DOMContentLoaded และตรวจรหัสตอบกลับ หากการตอบกลับล้มเหลวหรือมีรหัสสถานะตั้งแต่ 400 ขึ้นไปหน้านั้นจะถูกข้ามโดยบันทึกเหตุการณ์เป็น log เพื่อคงความก้าวหน้าของงาน หลังจากนั้นจะอ่านส่วนหัว content-type และพิจารณาเฉพาะชนิดที่สกัดข้อความได้ เช่น text/html, application/xhtml+xml หรือ text/plain หากชนิดเนื้อหาไม่อยู่ในชุดที่รองรับ ระบบจะบันทึกว่า “skip_non_html” แล้วไม่ดึงเนื้อหาต่อ ทั้งหมดนี้ช่วยประหยัดเวลาและพื้นที่เก็บข้อมูล โดยกันไม่ให้ไฟล์ใบหรือสคริปต์ที่ไม่ใช่ HTML หลุดเข้าสู่สายพานสกัดในบอทถัดไป

กรณีหน้าที่ผ่านเกณฑ์เนื้อหา ระบบจะรอตัวชี้วัดพร้อมแสดงผลตามที่กำหนดไว้ในคอนฟิก (WAIT_SELECTOR) เพื่อให้แน่ใจว่าองค์ประกอบสำคัญในหน้าซึ่งโหลดแบบไดนามิกได้ถูกเรนเดอร์เสร็จจริง จากนั้นจึงดึง HTML เต็มหน้าออกมาเป็นสตริง การใช้กลยุทธ์ auto-wait ตามธรรมชาติของ Playwright ลดปัญหา timing-error และทำให้คุณภาพของเอกสารต้นทางที่เข้าสู่โมดูลสกัดมีเสถียรภาพมากขึ้น

เมื่อได้ HTML แล้ว ขั้นตอนนี้จะไม่ “ทำความสะอาดเนื้อหา” เพื่อหลีกเลี่ยงการซ้ำกับบท 4.3 แต่จะทำการแปลง URL ของหน้าให้เป็นรูปแบบมาตรฐาน (canonical) บันทึกไฟล์ HTML ลงไฟล์เดอร์เอาต์พุต และลงทะเบียนแถว record: “page” ลงใน manifest.jsonl โดยใส่ URL ต้นทาง ค่า canonical พาธไฟล์ และเวลาเก็บ ทั้งนี้เพราะเป้าหมายของ 4.2.5 คือการรับประกันว่า “ได้หน้า HTML หลังเรนเดอร์ที่ตรวจสอบย้อนกลับได้” ก่อนจะส่งต่อให้ส่วนสกัด โค้ดที่เกี่ยวข้องกับการเขียนแฟ้มและบันทึก manifest จึงอยู่เพียงเท่าที่จำเป็น และรายละเอียดการสกัดเชิงเนื้อหาจะย้ายไปอธิบายในบอทถัดไปตามแผนการเขียนทั้งหมด

เพื่อความครบถ้วนของสายพาน เมื่อจบการดึงหนึ่งหน้า ระบบจะเรียกตัวสกัดลิงก์และไฟล์สื่อจาก HTML เดียวกันเพื่อเตรียมต่อคิวและดาวน์โหลดไฟล์ RAW-first ซึ่งเป็นตรรกะต่อเนื่องจาก 4.2.4 และจะถูกลงรายละเอียดในหัวข้อที่เกี่ยวข้องอยู่แล้ว ที่ส่วนนี้จึงกล่าวเพียงว่า “หลังดึงหน้าเสร็จ จะมีการแตกลิงก์และนับเป็นหน้าเสร็จหนึ่งหน่วย” พร้อมบันทึกความคืบหน้าเป็นช่วง ๆ

4.2.6 สกัดลิงก์และไฟล์สื่อจาก HTML

ส่วนนี้รับ HTML หลังเรนเดอร์จาก 4.2.5 แล้ว “กลั่น” ออกมาเป็น 3 ชุดข้อมูล ได้แก่ 1. ลิงก์หน้าเว็บที่จะใช้ขยายคิว 2. ลิงก์ไฟล์ PDF และ 3. ลิงก์รูปภาพที่จะถูกดาวน์โหลดแบบ RAW-first ใน 4.2.7 เป้าหมายหลักคือให้ได้ลิสต์ URL ที่ “ถูกต้อง อยู่ในขอบเขต และไม่ซ้ำ” ออกแบบจึงเน้น 4 แนวคิด: รวมลิงก์อย่างปลอดภัยให้เป็น absolute URL, ทำให้ URL อยู่ในรูปแบบมาตรฐานเดียว (canonical) ก่อนใช้ตัดสินใจ, กรองขอบเขตด้วย allowlist/สคีมที่อนุญาต, และกำจัดรายการซ้ำ/ฟุ้งซ่านตั้งแต่ต้นน้ำ

ขั้นแรก สคริปต์จะพาร์ส HTML ด้วยตัวแยกโครงสร้าง (เช่น BeautifulSoup/lxml) เพื่อค้นหาแหล่งอ้างอิงทั่วไป ได้แก่ <a href>, , srcset, และแอตทริบิวต์ lazy-loading อย่าง data-src, data-href รวมถึง style=”background-image:url(…)” ในบางธิม จากนั้นจึงแปลงเป็น absolute URL ด้วยตัว

รวมลิงก์ที่ป้องกันสคีมไม่ปลอดภัย เช่น javascript:, data:, mailto:, [tel:](#) และลบ fragment (#...) ออกเสมอ เพราะไม่มีผลต่อทรัพยากรฝั่งเซิร์ฟเวอร์ ก่อนจะค่อย ๆ นำ URL ที่ผ่านการคัดกรองเข้าสู่ท่อ canonicalization (ดูหลักการใน 4.2.3) เพื่อให้เทียบเท่ากันได้แบบสตริงตัวต่อตัว ลด false-duplicate ที่เกิดจากพารามิเตอร์ติดตามและความต่างเรื่องเครื่องหมายทับ/ไฟล์ index

ถัดมา สคริปต์จะจัดเข้า “ตะกร้า” ตามชนิดการใช้งาน โดยถ้าลงท้ายด้วยนามสกุลสื่อที่ไม่ใช่หน้า HTML จะถูกคัดเข้าหมวดสื่อทันที เช่น .pdf เข้าหมวดเอกสาร และ .jpg เข้าหมวดรูป ส่วนลิงก์ที่ไม่สื่อชัดด้วยนามสกุล (เช่นปลายทาง redirect หรือ URL ไม่ระบุนามสกุล) จะยังคงอยู่ในหมวด “ลิงก์หน้า” เพื่อรอ Content-Type gate ใน 4.2.5/4.2.7 เป็นผู้ตัดสินอีกชั้นหนึ่ง แนวทางนี้ลดการพลาดไฟล์สื่อที่ซ่อนอยู่หลังลิงก์แบบ dynamic นอกจากนี้เพื่อคงความสุภาพต่อคิว ขั้นตอนสก็ตจะจำกัดจำนวนผลสูงสุดต่อหน้า (เช่น 200–500 รายการต่อหมวด) และเรียงผลแบบกำหนดได้ (เช่น breadth-first/ลำดับปรากฏ)

การกรองโดเมน/ซับโดเมนที่อนุญาตจะถูกบังคับใช้หลัง canonicalization เสมอเพื่อความถูกต้องของการเปรียบเทียบ และจะมีชุดกฎย่อยเพื่อลดรายการไม่จำเป็น เช่น ตัดไฟล์มัลติมีเดียไม่ซ้ำลิงก์ซ้ำภายในหน้าเดียวกัน (เช่น anchor เดิมต่าง query), และข้ามลิงก์ที่ชี้กลับมาหน้าปัจจุบันโดยไม่เพิ่มสารสนเทศทั้งหมดนี้ช่วยให้ manifest ใน 4.2.7 สะอาดและสถิติผลลัพธ์ท้ายบท 4.2 สื่อความหมายจริง ไม่ถูกบั่นด้วยรายการซ้ำซ้อนหรือไฟล์ที่ไม่เกี่ยวข้องกับการสกัดข้อความ

ส่วนการดาวน์โหลดจริง การบันทึกแถว asset ใน manifest.jsonl และนโยบายคิว/ความลึก จะขยับไปอธิบายในหัวข้อถัดไป

4.2.7 ดาวน์โหลดไฟล์ RAW และเขียน Manifest

ส่วนนี้ทำหน้าที่ “ทำให้หลักฐานการเก็บข้อมูลมีตัวตนบนดิสก์และตรวจสอบย้อนกลับได้” โดยทันทีที่ได้ HTML หลังเรนเดอร์จากหัวข้อ 4.2.5 และชุด URL สื่อ (PDF/รูป) จากหัวข้อ 4.2.6 ระบบจะบันทึกไฟล์ดิบ (RAW) ลงโพลเดอร์ที่กำหนด แล้วลงทะเบียนเหตุการณ์เก็บเป็นแถวข้อมูลใน output/stage/manifest.jsonl แนวคิดสำคัญมีสามประการ ได้แก่ ความเสถียรของตัวระบุผ่านการตั้งชื่อไฟล์จากแฮชของ URL (SHA-1) ความเป็นอะตอมิกของการเขียน manifest แบบ JSON Lines เพื่อกันข้อมูลค้างคาเมื่อเกิดข้อผิดพลาด และความสามารถในการตรวจสอบย้อนกลับด้วยฟิลด์ที่เชื่อมโยงกันชัดเจนระหว่าง source_url (ที่อยู่ทรัพยากร), page_url (หน้าที่พบทรัพยากร) และ local_path (ที่เก็บจริงบนดิสก์)

กระบวนการเริ่มจากหน้า HTML ที่ผ่านเกณฑ์ใน 4.2.5 หน้าเดียวจะถูกบันทึกเป็นไฟล์ใน output/raw/html โดยตั้งชื่อจาก SHA-1 ของ URL หลัง canonicalization เพื่อให้เป็นตัวแทนหน้าเดียวอย่างคงรูป จากนั้นจึงเขียนหนึ่งแถวลง manifest.jsonl ด้วยชนิดระเบียบ record=“page” โดยบันทึกเวลาปัจจุบันในรูปแบบ ISO, URL ต้นทาง, ค่า canonical, พารไฟล์, ขนาดไฟล์ และรหัสตอบกลับ/ชนิดเนื้อหาถ้ามีข้อมูลดังกล่าว จุดนี้ตั้งใจ “ไม่ทำความสะอาดเนื้อหา” เพื่อหลีกเลี่ยงการซ้ำกับบท 4.3 ซึ่งจะเป็นผู้รับช่วงการแปลง/สกัดข้อความทั้งหมด

ถัดมาคือการดาวน์โหลดไฟล์สื่อบนแบบ RAW-first รายการ PDF และภาพที่ได้จาก 4.2.6 จะถูกดึงผ่าน HTTP ภายใต้ตัวจำกัดอัตราต่อโฮสต์เดียวกับที่ใช้ใน 4.2.5 เพื่อคงความสุภาพต่อปลายทาง แต่ละไฟล์จะถูกตรวจหัวข้อ Content-Type และขนาดโดยคร่าวก่อนบันทึกเพื่อลดไฟล์เสียหรือไฟล์ที่ไม่ตรงชนิด หากไม่พบส่วนขยายที่เชื่อถือได้จาก URL จะพิจารณาเดา (infer) จาก Content-Type ที่ตอบ ชื่อไฟล์สุดท้ายประกอบด้วย sha1(url) + ext เพื่อให้การอ้างอิงกลับและการ “ลบตาม URL” ในอนาคตทำได้ตรงไปตรงมาด้วย canonical URL เดิม โดยบันทึกลงโฟลเดอร์ output/raw/pdfs หรือ output/raw/images ให้เป็นระเบียบสอดคล้องกับขั้นตอนถัดไป

ทุกครั้งที่บันทึกสำเร็จ ระบบจะเขียนหนึ่งแถวใน manifest.jsonl ด้วย record="asset" พร้อม asset_type เช่น pdf หรือ image และฟิลด์สัมพันธ์ page_url เพื่อชี้ว่าทรัพยากรนี้ถูกพบจากหน้าใด ถ้าดาวน์โหลดล้มเหลว แถวดังกล่าวจะถูกเขียนในสถานะ "error" แยกกันแทนการหยุดทำงาน เพื่อรักษาการก้าวหน้าของสายพานและทำให้การสรุปผลท้ายบท 4.2 สะท้อนสถานการณ์จริงของการเก็บข้อมูล โดยในทางปฏิบัติจะมีนโยบาย retry ระดับต่ำ เช่น 1-2 ครั้งต่อ URL พร้อม backoff สั้น ๆ เพื่อกันความผิดพลาดชั่วคราวจากเครือข่ายโดยไม่ทำให้โฮสต์ปลายทางรับภาระเกินจำเป็น

รูปแบบสคีมาของแถวใน manifest.jsonl ตั้งใจให้อ่านง่ายและพึงพาฟิลด์จำนวนน้อย ได้แก่ ts เวลาเก็บแบบ ISO, record ชนิดระเบียบ ("page" หรือ "asset"), source_url ค่า URL หลัง canonicalization, page_url ซึ่งจะว่างสำหรับระเบียบชนิดหน้าและจะชี้กลับหาหน้าต้นทางสำหรับระเบียบชนิดสื่อ, local_path พารามิเตอร์บนดิสก์, content_type, status, size และ sha1 ของ URL เพื่อใช้ยืนยันตัวตนอีกชั้นหนึ่ง สคีมานี้คือ “แหล่งความจริงเดียว” (single source of truth) ที่บท 4.3 จะใช้เป็นอินพุตในการสกัดข้อความไปยัง documents.jsonl ต่อไป และเป็นจุดยึดสำหรับการทำรายงานสถิติ (จำนวนหน้า/ไฟล์ต่อโดเมน อัตราสำเร็จ/ล้มเหลว) โดยไม่ต้องสแกนโฟลเดอร์ซ้ำ

4.2.8 จำกัดอัตราการร้องขอแบบต่อโฮสต์ (Per-Host Rate Limiter)

ส่วนนี้ทำหน้าที่ “ตั้งจังหวะ” กับการดึงหน้าและการดาวน์โหลดไฟล์ทั้งหมด เพื่อให้สุภาพต่อเซิร์ฟเวอร์ปลายทาง ลดโอกาสถูกปฏิเสธการเชื่อมต่อ (429/503/timeout) และทำให้ผลการเก็บมีเสถียรภาพ การออกแบบยึดหลัก per-host fairness คือกระจายจังหวะตามโฮสต์จริง ๆ ไม่ใช่จำกัดแบบรวมทั้งหมด ด้วยแนวคิดง่าย ๆ ว่า “หนึ่งโฮสต์ = หนึ่งคิวเล็ก ๆ ที่ปล่อยคำขอได้ไม่เกิน PER_HOST_RPS ครั้งต่อวินาที” ผลคือหากระบบต้องคุยกับหลายโดเมนพร้อมกัน งานจะเดินไปพร้อม ๆ กันอย่างยุติธรรม โดยไม่ปล่อยให้โดเมนเดียวถูกยิงถี่ ส่วนพารามิเตอร์กำหนดจากคอนฟิกใน 4.2.2 ได้แก่ PER_HOST_RPS (อัตรา), NAV_TIMEOUT_MS (เวลาเฝ้ารอหน้า), และนโยบาย retry ระดับต่ำ (กำหนดใน 4.2.7) เพื่อให้ทั้งสายพานยังคงเดินหน้าเมื่อพบข้อผิดพลาดชั่วคราว

กลไกหลักเป็นลักษณะ token-less time gating ต่อโฮสต์: ระบบเก็บ “เวลาที่สามารถยิงครั้งถัดไปได้” ของแต่ละโฮสต์ไว้ในหน่วยเวลาโลก (UTC) เมื่อมีคำขอใหม่เข้ามา จะตรวจนาฬิกา ถ้ายังไม่ถึงคิวก็รอส่วน

ต่าง (sleep แบบ async) เล็กน้อยก่อนปล่อยให้ไปต่อ การลอกพอร์ตปริยาย (80/443) และการ normalize โฮสต์ (ตัวพิมพ์เล็ก/IDNA) ทำให้นิยาม “ต่อโฮสต์” มีเอกภาพสอดคล้องกับ canonicalization ใน 4.2.3 ทั้งยังคุ้มครองขอบเขตให้แยกคิวกัน เช่น www.reg.chula.ac.th กับ sa.chula.ac.th ซึ่งเหมาะกับงานที่กระจายหลายไซต์ย่อยขององค์กรเดียวกัน

4.2.9 ผลลัพธ์การดึงข้อมูล

ผลการดำเนินการของขั้นตอนดึงข้อมูลดิบสะท้อนผ่านทั้งบันทึกเหตุการณ์และแฟ้ม manifest.jsonl ซึ่งทำหน้าที่เป็นหลักฐานกลางของงานที่เกิดขึ้นจริง ในรอบการรันนี้ ระบบสิ้นสุดงานด้วยข้อความบันทึกว่าได้ประมวลผลหน้าเว็บสำเร็จจำนวน 3,367 หน้า โดยยืนยันด้วยบรรทัดสรุปที่ถูกเขียนเมื่อปิดลูประมวลผล ทั้งนี้ตัวเลข “จำนวนหน้า” ที่รายงานไว้ตรงกับจำนวนระเบียบชนิด page ใน manifest.jsonl หลังการคัดกรองซ้ำซ้อนด้วยการทำ URL ให้เป็นมาตรฐานเดียว (canonical) จึงสามารถใช้เป็นตัวแทนปริมาณความครอบคลุมเริ่มต้นได้อย่างน่าเชื่อถือ และเป็นฐานให้ขั้นสกัดข้อความในบท 4.3 ใช้งานต่อไปอย่างเป็นระบบ

[2025-10-01T18:14:26.892649+00:00] INFO step1_done pages=3367

รูปที่ 4.1 ล็อกแสดงผลการดึงข้อมูล

ในเชิงคุณภาพของข้อมูลที่ได้ ระบบยังคงกตึกการคัดกรองชนิดเนื้อหาไว้ตั้งแต่ด้านตั้งหน้า ทำให้เฉพาะเอกสารที่สกัดได้จริง เช่น text/html หรือไฟล์สื่อที่ยืนยันชนิดด้วยส่วนหัว Content-Type เท่านั้นที่จะเข้าสู่คลัง RAW และถูกลงทะเบียนใน manifest.jsonl โดยหน้าเว็บหนึ่งหน้าจะถูกบันทึกเป็นไฟล์ HTML หลังการเรนเดอร์ พร้อมระเบียบชนิด page ขณะที่ไฟล์ PDF และรูปภาพที่พบจากการสกัดลิงก์ในแต่ละหน้าจะถูกดาวน์โหลดแบบ RAW-first และลงทะเบียนเป็นระเบียบชนิด asset ที่เชื่อมโยงกลับไปยัง page_url ต้นทางเพื่อรองรับการตรวจสอบย้อนกลับ

4.3 การสกัดข้อมูลจากข้อมูลดิบ (RAW → documents.jsonl)

ขั้นนี้มีหน้าที่แปลงไฟล์ดิบที่ได้จากบท 4.2 — ได้แก่ HTML หลังเรนเดอร์, PDF, และรูปภาพ—ให้เป็น “เอกสารข้อความพร้อมเมตาดาตา” ที่เก็บอยู่ใน documents.jsonl พร้อมสำหรับการฝังเวกเตอร์และการประเมินผลในบทถัดไป จุดเน้นคือทำให้ข้อความ “อ่านได้จริง” (clean/normalize), “กรองคุณภาพ” ตามเกณฑ์ขั้นต่ำ, “มีโครงสร้างพอประมาณ” เพื่อรองรับการแบ่งชิ้นส่วน (chunking), และ “ตรวจสอบย้อนกลับ” สู่ URL ต้นทางได้ตลอด โดยยึด manifest จาก 4.2 เป็นแหล่งความจริงสำหรับที่มาและเวลาการเก็บ

การไหลของงานในบทนี้จะแยกตามชนิดอินพุตสั้น ๆ: HTML จะถูกสกัดข้อความและหัวเรื่องโดยเคิร์ฟโครงหน้า, PDF จะพยายามดึงข้อความจากเลเยอร์ตัวอักษรก่อนและมี OCR เป็นทางเลือกเมื่อจำเป็น, ส่วนรูปภาพจะใช้ OCR เฉพาะกรณีที่พบว่ามีข้อความสื่อสารจริง จากนั้นทุกกรณีจะผ่านการทำความสะอาดและตรวจคุณภาพ (เกณฑ์จำนวนโทเค็นขั้นต่ำ, สัดส่วนภาษาไทยขั้นต่ำ, ชุดอักขระที่อนุญาต, และการลบซ้ำ

แบบ exact-hash หลัง normalize) ก่อนจะแบ่งขึ้นด้วยโปรไฟล์ที่เหมาะสมกับชนิดเนื้อหา แล้วจึงเขียนลง documents.jsonl แบบ JSON Lines ทีละแถว

เป้าหมายของ 4.3 จึงจำกัดอยู่ที่ “ได้ข้อความคุณภาพดี + เมทาเดตาครบ + พร้อมแบ่งขึ้น” เท่านั้น โดยรายละเอียดการทำงานจะแตกเป็น 4.3.1–4.3.9 ตามลำดับ: คอนฟิกและตัวช่วย, ตัวสกัด HTML, ตัวสกัด PDF (พร้อม OCR fallback), OCR รูปภาพ, ทำความสะอาด/มาตรฐานข้อความ, ประสิทธิภาพ (quality gates), โครงเมทาเดตาเอกสาร, โปรไฟล์การแบ่งขึ้น และการเขียนผลลัพธ์ลงไฟล์ หากต้องการดูผลลัพธ์ฉบับเต็มสามารถดูได้ในภาคผนวก

4.3.1 คอนฟิกและตัวช่วยเฉพาะงานสกัด

ส่วนคอนฟิกในบทนี้กำหนดเฉพาะ “พารามิเตอร์ของการสกัดข้อความและคุณภาพเอกสาร” โดยแยกจากคอนฟิกการดึงข้อมูลในบท 4.2 เพื่อให้ปรับทีละขั้นได้อย่างอิสระ แกนหลักประกอบด้วยพารามิเตอร์/เอนต์รี (IN_MANIFEST, OUT_DOCS), เกณฑ์คุณภาพขั้นต่ำ (MIN_TOKENS สำหรับจำนวนโทเค็นหลังทำความสะอาด, TH_RATIO_THRESHOLD สำหรับสัดส่วนอักขระไทยขั้นต่ำในบริบทเนื้อหาไทย โปรไฟล์การแบ่งขึ้น (CHUNK_SIZE_STRUCT, CHUNK_OVERLAP สำหรับเนื้อหาโครงสร้างดี และ CHUNK_SIZE_OCR สำหรับข้อความจาก OCR), และ มีการเตรียมสภาพแวดล้อมไว้ผ่าน POPPLER_PATH สำหรับการแปลง/เรนเดอร์ PDF และ OCR_LANGS เพื่อชี้ชุดภาษา OCR ที่สอดคล้องกับบริบทไทย-อังกฤษ ซึ่งจะถูกใช้งานจริงในสายการสกัดทั้งโครงสร้างของบท 4.3 ตามกรอบไลบรารีที่อธิบายในบทที่ 1 (PyMuPDF/pymupdf4llm และ EasyOCR) เพื่อรักษาความต่อเนื่องของคุณภาพข้อความและโครงสร้างก่อนเข้าสู่การฝังเวกเตอร์และประเมินผลในบทถัดไป ทั้งหมดนี้ออกแบบให้ “พอจำเป็น” เพื่อคุมคุณภาพเอกสารโดยไม่ผูกติดกับไลบรารีภายนอกมากเกินไปในระดับรายงาน

ฟังก์ชันช่วย (utilities) ในบทนี้มีสี่กลุ่มสั้น ๆ: 1. การอ่านแถวจาก manifest.jsonl เฉพาะที่เกี่ยวข้องกับเอกสาร (page/asset) พร้อมคัดชนิดเป้าหมาย, 2. การทำความสะอาด/ทำให้ข้อความเป็นมาตรฐานเดียว (normalize) เช่น NFKC, ตัดเว้นวรรคซ้ำ, เก็บ URL/ตัวเลข, 3. การวัดคุณภาพขั้นพื้นฐาน เช่นนับโทเค็นหลังทำความสะอาด, ประเมินสัดส่วนไทย และตรวจชุดอักขระที่อนุญาต, และ 4. ตัวเขียนผลลัพธ์แบบ JSON Lines (append_jsonl) ที่รับประกันอะตอมมิกของการบันทึก ทั้งหมดนี้จะถูกเรียกใช้โดยตัวสกัดตามชนิดอินพุตในหัวข้อ 4.3.2–4.3.4 และเกณฑ์คุณภาพใน 4.3.6 โดยตรง เพื่อลดการซ้ำซ้อนของโค้ด

4.3.2 แปลง HTML เป็นข้อความ

ส่วนนี้ทำหน้าที่อ่านหน้า HTML ที่บันทึกไว้ แล้ว “กลั่น” ให้เหลือข้อความพร้อมเมทาเดตาขั้นต่ำ ก่อนเขียนออกเป็นแถวใน documents.jsonl กลไกหลักอาศัย trafilatura.extract(...) เพื่อดึงข้อความแบบคงรูปเลย์เอาต์พอประมาณและดึงเมทาเดตาจาก <meta> ถ้ามี ส่วนกรณีที่ trafilatura ไม่คืนข้อมูล จะถอย

ไปใช้ BeautifulSoup.get_text() เป็นทางเลือกสุดท้าย ข้อมูลแต่ละหน้าเมื่อผ่านการสกัดแล้วจะถูกหักซ้ำด้วยค่าแฮชของเนื้อหา (content SHA-1) เพื่อหลีกเลี่ยงการออกเอกสารซ้ำ และคงตัวระบุ (id) ที่ผูกกับ URL แบบคงรูปเพื่อการอ้างอิงและอัปเดตในอนาคต

การดึงเมตาดาตาใช้ trafilatura.extract(..., output_format="json", with_metadata=True) โดยแปลงผลลัพธ์ JSON ให้เป็นพิลด์ง่าย ๆ ได้แก่ title, published_at, และ updated_at (แมปจากคีย์ title, date, modified/date_modified) หากเกิดข้อผิดพลาดจะบันทึกคำเตือนและคืนว่างเพื่อไม่ให้หยุดสายการประมวลผล เมตาดาตาดังนี้ช่วยให้ผู้อ่านตรวจแหล่งที่มาและกรอบเวลาของเอกสารได้ โดยไม่ลงลึกถึงโครงสร้างเมตาดาตารุ่นสูงอื่น ๆ

การดึงข้อความตั้งค่าพารามิเตอร์ที่ระบุ “ดึงให้ครบก่อน ค่อยกรอง” ได้แก่ include_tables=True เพื่อพยายามรักษาตาราง, include_formatting=True เพื่อเก็บหัวเรื่อง/รายการพื้นฐาน, avor_recall=True เพื่อโน้มน้าวไปยัง recall เมื่อมีทางเลือก, และ deduplicate=True เพื่อลดการทำซ้ำขณะสกัด หาก trafilatura คืนค่าว่างหรือไม่รองรับรูปหน้า HTML นั้น ฟังก์ชันจะลดยไปใช้ BeautifulSoup.get_text ซึ่งได้ข้อความแบบเรียบง่าย

ลูประมวลผลจะเปิดอ่าน manifest.jsonl ที่ละแถว เลือกเฉพาะระเบียบชนิดหน้า (record == “page”), ตรวจสอบว่า html_path และ URL แบบคงรูป (canonical) มีค่าถูกต้อง จึงเปิดไฟล์ HTML มาสกัดข้อความ เมื่อได้ข้อความแล้วจะคำนวณ content_sha1(clean) เพื่อตรวจหาซ้ำกับชุด SEEN_SHA1 (ซึ่งถูกโหลดมาก่อนหน้าผ่าน seen_sha1()) ถ้าพบว่าเนื้อหาซ้ำจะข้าม ไม่เขียนเอกสารซ้ำอีก เมื่อผ่านด่านนี้จะสร้างระเบียบผลลัพธ์หนึ่งแถว โดย id ยึดตาม sha1_text(src) เพื่อให้ “หนึ่ง URL = หนึ่งตัวระบุ” ที่เสถียร ส่วน content_hash คือ SHA-1 ของเนื้อหาหลังสกัด และ captured_at คือเวลาบันทึก ณ ตอนสร้างระเบียบ ถ้ามีเมตาดาตาสกัดสำเร็จก็จะรวมเข้าในแถวนั้น จากนั้นเขียนลง documents.jsonl และนับจำนวนงานที่เพิ่ม ก่อนปิดท้ายด้วยบันทึก step2_done เพื่อยืนยันผลรวม

4.3.3 แปลงรูปภาพเชิงข้อมูล

ส่วนนี้คัดเฉพาะ “รูปภาพเชิงข้อมูล” (infographic) ออกจากบรรดาไฟล์ภาพทั้งหมด แล้วทำ OCR ความละเอียดสูงเพื่อดึงข้อความ ก่อนบันทึกเป็นแถวใน documents.jsonl จุดสำคัญมีสามช่วงคือการคัดเลือกภาพที่ควร OCR, การทำ OCR พร้อมตัวชี้วัดคุณภาพอย่างย่อ, และการเขียนผลลัพธ์

อัลกอริทึมคัดเลือกเริ่มจากการอ่านระเบียบชนิด asset ที่เป็นภาพจาก manifest.jsonl แล้วเปิดไฟล์ภาพบนดิสก์ตามพาส path จากนั้นฟังก์ชัน is_infographic(...) จะตัดสินใจว่าควร OCR หรือไม่โดยพิจารณาหลายเกณฑ์ร่วมกัน ภาพที่เล็กมาก (ด้านสั้น < 320 พิกเซล) จะถูกตัดออกทันทีเพราะมีโอกาสดึงข้อความได้ต่ำ หากผ่านเบื้องต้น ภาพจะถูกย่อให้ด้านยาว ≈ 800 พิกเซลเพื่อจัดสเกลให้อ่านง่ายและคำนวณรวดเร็ว แล้วใช้ EasyOCR อ่านตัวอักษรในโหมดกล่องคำ (detail=1) ถ้า OCR ไม่พบข้อความเลย อัลกอริทึมยังไม่ปิด

โอกาสทันที แต่จะใช้เบาะแสจาก alt_text และ src_hint ที่ติดมากับแถว manifest เพื่อตรวจหาคีย์เวิร์ดที่พบได้บ่อยในอินโฟกราฟิก เช่น “กำหนดการ”, “ประกาศ”, “ขั้นตอน”, “ลงทะเบียน”, “reg”, “chula” เป็นต้น เมื่อ OCR มีผลลัพธ์ จะคำนวณสัดส่วนพื้นที่ข้อความโดยรวมจากผลลัพธ์หลายกล่องผ่านสูตรพื้นที่รูปหลายเหลี่ยม (shoelace) แล้วหารด้วยพื้นที่ภาพทั้งหมด พร้อมคำนวณค่าเชื่อมั่นเฉลี่ยของ OCR ภาพหนึ่งภาพจะถูกจัดว่าเป็นอินโฟกราฟิกทันทีหากจำนวนกล่องข้อความตั้งแต่หกกล่องขึ้นไป หรือสัดส่วนพื้นที่ข้อความ ≥ 0.08 และค่าเชื่อมั่นเฉลี่ย ≥ 0.5 หรือมีคีย์เวิร์ดที่กำหนดอยู่ในข้อความรวม ทั้งหมดนี้ทำให้การคัดเลือกยืดหยุ่นต่อกรณีที่ข้อความบางส่วนพร่าเลือนแต่ยังมีบริบทบ่งชี้ว่าเป็นสื่อสารสารสำคัญ

เมื่อผ่านการคัดเลือก ระบบจะทำ OCR แบบความละเอียดสูงอีกครั้งหนึ่งโดยขยายภาพตามขนาดที่ปรับตามด้านยาว ($\times 1.8$ สำหรับภาพเล็กกว่า 1600 พิกเซล เพื่อช่วยให้อักขระชัดขึ้น และ $\times 1.2$ สำหรับภาพใหญ่) พร้อมอ่านข้อความใหม่ในโหมดกล่องคำเพื่อเก็บทั้งเนื้อหาและสถิติคุณภาพ จากผลลัพธ์ OCR จะประกอบข้อความรวม (text_img) หากว่างเปล่าจะข้ามทันที แต่ถ้ามีข้อความ ระบบจะคำนวณตัวชี้วัดคุณภาพสองค่าอย่างย่อ ได้แก่ ocr_avg_conf ซึ่งเป็นค่าเชื่อมั่นเฉลี่ย และ ocr_text_area_ratio ซึ่งเป็นสัดส่วนพื้นที่ข้อความต่อภาพ ชุดตัวชี้วัดนี้เป็นหลักฐานให้อ่านประเมินความน่าเชื่อถือของ OCR โดยไม่ต้องเก็บรายละเอียดเชิงตำแหน่งทุกกล่อง ก่อนเขียนผลลัพธ์แต่ละภาพ ระบบจะหักซ้ำด้วยแฮชของ “ข้อความหลัง OCR” (content_sha1(text_img)) โดยอาศัยชุด SEEN_SHA1 หากพบว่ามีข้อความนี้ถูกเก็บไว้แล้วจะไม่สร้างเอกสารซ้ำเพื่อหลีกเลี่ยงข้อมูลซ้ำข้ามหน้าและข้ามไฟล์ภาพที่มีเนื้อหาตรงกัน การเลือกใช้แฮชของข้อความ เป็น id และ content_hash ทำให้เอกลักษณ์ของเอกสารผูกกับเนื้อหาจริง ไม่ขึ้นกับตำแหน่งที่พบ ช่วยควบคุมกรณีที่ภาพอินโฟกราฟิกเดียวกันถูกใช้ซ้ำบนหลายหน้า ขณะที่ฟิลด์อ้างอิงอย่าง source_url (URL ของไฟล์ภาพ) และ page (หน้าเว็บที่พบภาพ) ยังคงอยู่ครบเพื่อการตรวจสอบย้อนกลับ

เมื่อพร้อมบันทึก แถวผลลัพธ์หนึ่งแถวจะมี id เท่ากับแฮชข้อความ, source_type="image", is_infographic=True, เนื้อหาเต็มใน text_full, ข้อมูลประกอบอย่าง alt, เวลา captured_at, ความเชื่อมั่นเฉลี่ย ocr_avg_conf, สัดส่วนพื้นที่ข้อความ ocr_text_area_ratio, และการอ้างอิงกลับหน้าต้นทางใน page ต่อจากนั้นจึงเขียนลง documents.jsonl แบบต่อบรรทัด และปิดท้ายการทำงานด้วยบันทึก step3_done เพื่อยืนยันจำนวนเอกสารภาพที่เพิ่มเข้าไปจริง

4.3.4 แปลง PDF เป็นข้อความ

ส่วนนี้รับไฟล์ PDF ที่ถูกเก็บไว้แล้วบนดิสก์ แปลงให้เป็นข้อความรูปแบบ Markdown หรือ text layout คงรูป แล้วเขียนออกเป็นแถวใน documents.jsonl โดยยึดหลัก “พยายามใช้เลเยอร์ตัวอักษรให้สุดก่อน ถ้าไม่ได้ค่อยถอยไป OCR แบบเฉพาะหน้า” เพื่อให้ได้คุณภาพข้อความที่ดีที่สุดเท่าที่ไฟล์เอื้ออำนวย พร้อมใส่ธงบอกว่ามีตารางหรือไม่เพื่อช่วย downstream ในภายหลัง

อัลกอริทึมเริ่มจากการเปิดอ่าน manifest.jsonl ที่สแกนและคัดเฉพาะระเบียบชนิด asset ที่มี asset_type เป็น “pdf” จากนั้นตรวจสอบว่า path ของไฟล์และ url ของทรัพยากรครบถ้วนจริงก่อนดำเนินการ แล้วจะเรียก pdf_to_markdown_with_ocr_fallback(pdf_path) เพื่อดึงข้อความ โดยฟังก์ชันนี้ทำงานแบบสามชั้นเรียงลำดับความน่าเชื่อถือจากสูงไปต่ำ ชั้นแรกใช้ pymupdf4llm.to_markdown(...) เพื่อดึงเป็น Markdown ที่เคอร์ฟโครงเอกสาร เช่น หัวเรื่อง รายการ และกล่องข้อความ ถ้าได้ผลลัพธ์ที่ “ผ่านเกณฑ์เบื้องต้น” ไม่มีอัตราอักษรแปลก(ตัวอักษรที่อ่านไม่ได้) เกิน 5% และผ่านการระบุภาษาว่าเป็นไทยหรืออังกฤษด้วยความเชื่อมั่นเพียงพอที่จะคืนผลทันที ชั้นที่สองหากชั้นแรกล้มเหลว จะใช้ pdftotext ทั้งไฟล์ (-layout -enc UTF-8) เพื่อคงแนวคอลัมน์และเว้นวรรคได้ดีพอประมาณ เมื่อผลลัพธ์ผ่านเกณฑ์เดียวกันจะหยุดที่ชั้นนี้ ชั้นที่สามหากสองชั้นแรกยังไม่ผ่าน จะเข้าสู่ “กู๋รายหน้า” โดยเปิดเอกสารด้วย PyMuPDF แล้ววนทีละหน้า ลอง page.get_text(“text”, sort=True) ก่อน ถ้าไม่ผ่านให้เรียก pdftotext เฉพาะหน้านั้น และถ้ายังไม่ได้อีกจะดึงข้อความที่ใช้การ สุดท้ายจึงเรนเดอร์หน้าเป็นภาพด้วย DPI สำหรับ OCR (ค่าพื้นฐาน 220) แล้วอ่านด้วย EasyOCR เฉพาะหน้านั้น วิธีต่อหน้าช่วยจำกัดขอบเขตการใช้ OCR เฉพาะจุดที่จำเป็นและลด noise ที่มักเกิดจากการ OCR ทั้งไฟล์

เกณฑ์คุณภาพที่ใช้ตลอดทั้งสามชั้นถูกรวมอยู่ใน _qc_ok(...) และ is_usable_th_en(...) โดย _qc_ok จะกรองข้อความที่สั้นเกินไป หรือมีสัดส่วนอักษรแปลก(ตัวอักษรที่อ่านไม่ได้) สูงเกิน 5% ส่วน is_usable_th_en ใช้ langid ตรวจสอบว่าข้อความเป็นไทยหรืออังกฤษด้วยความเชื่อมั่นขั้นต่ำตามที่กำหนด ฟังก์ชัน md_has_table(...) จะตรวจสอบ “มีตาราง” แบบง่ายทั้งกรณี HTML <table> และ Markdown table (| + บรรทัดคั่น ---) เพื่อบันทึกในเมทาดาทา การตั้งค่าเส้นทางไปยัง Poppler ถูกผนวกเข้ากับ PATH ชั่วคราวก่อนเรียก pdftotext ทั้งแบบทั้งไฟล์และแบบรายหน้า เพื่อความพกพาข้ามแพลตฟอร์ม

เมื่อได้ข้อความสุดท้ายแล้ว ระบบจะคำนวณแฮชเนื้อหา (content_sha1) เพื่อหักล้างกับชุด SEEN_SHA1 ถ้าเคยมีข้อความเดียวกันถูกบันทึกไปแล้วจะข้ามทันที เพื่อหลีกเลี่ยงการสร้างเอกสารซ้ำแม้จะมาจากคนละไฟล์หรือหน้าต่างกัน หากเป็นข้อความใหม่ ระบบจะสร้างระเบียบหนึ่งแถวประกอบด้วย id เท่ากับแฮชเนื้อหา, source_url ของ PDF, source_type=“pdf_md”, รูปแบบ format=“markdown” (แม้บางกรณีจะเป็น text layout แต่ยังเป็นชุดเดียวสำหรับ downstream), ธง has_tables, เนื้อหาเต็ม text_full, แฮช content_hash, เวลา captured_at, และ page ที่อ้างอิงไปยังหน้าต้นทาง จากนั้นจึงต่อบรรทัดลง documents.jsonl และนับจำนวนเอกสารที่เพิ่ม ก่อนปิดท้ายด้วยบันทึกสถานะ step4_done เพื่อยืนยันผลรวมการประมวลผล PDF

4.3.5 ผลลัพธ์จากการสกัดข้อมูล

การสกัดข้อมูลได้ผลลัพธ์ออกเป็นเอกสารสามชนิดหลักตามแหล่งกำเนิด ได้แก่ pdf_md, image (จากอินโฟกราฟิกผ่าน OCR), และ html โดยในรอบนี้มีจำนวนเอกสารรวม 2,473 แถว แบ่งเป็น PDF 623 แถว (~25.2%), รูปภาพ 763 แถว (~30.9%), และ HTML 1,087 แถว (~44.0%) ซึ่งสะท้อนว่าส่วนเนื้อหาที่

เป็นหน้าเว็บหลังเรนเดอร์ยังคงเป็นสัดส่วนใหญ่สุด ขณะที่ภาพอินโฟกราฟิกมีสัดส่วนมากพอสมควร และเอกสาร PDF มีจำนวนพอประมาณสำหรับงานเอกสารทางการและคู่มือใช้งานที่พบในโดเมนที่เก็บมา

สำหรับ เอกสารชนิด pdf_md ตัวอย่างที่ปรากฏในภาพแสดงเอกสารทางการ เช่น “ประกาศค่าปรับงานวิทยาศาสตร์ เรื่อง อัตราเงินเรียก... (reg.chula.ac.th)” และเอกสารแนวกู่มือใช้งาน เช่น “คู่มือติดตั้งโปรแกรม Cisco AnyConnect สำหรับ Mac OS X (it.chula.ac.th)” รวมถึงเอกสารเครือข่ายไร้สาย “ChulaGuestWifiWin7.pdf” ระบบสกัดข้อความในรูป Markdown/เลย์เอาต์คงรูปได้ครบประเด็นสำคัญ แม้ตัวอย่างทั้งสามระบุ has_tables=False แต่ในภาพรวมยังมีไฟล์อื่นที่ตรวจพบตารางได้ ข้อความที่ได้ผ่านเกณฑ์คุณภาพ (ไม่มีตัวอักษรเสีย สัดส่วนอักขระนอกชุดอนุโลมต่ำ และระบุภาษา TH/EN ได้) ก่อนจะถูกบันทึกเอกสารหนึ่งแถวต่อหนึ่งไฟล์โดยมี content_hash คมความซ้ำ

ในส่วนของ เอกสารชนิด image ระบบคัดเฉพาะภาพที่มีลักษณะ “อินโฟกราฟิก” แล้วทำ OCR ความละเอียดสูง ตัวอย่างที่ปรากฏมีทั้งภาพ “ตาราง/กำหนดการ” ของหน่วยงาน, ภาพประชาสัมพันธ์กิจกรรมพร้อมข้อความไทย-อังกฤษปนกัน, และภาพคู่มือที่มีคำว่า username/password ปรากฏเด่นชัดหลายรายการถูกติดธง is_infographic=True ตามตรรกะคัดกรอง (จำนวนกล่องข้อความ, สัดส่วนพื้นที่ข้อความ, ค่าเชื่อมั่น OCR และคีย์เวิร์ด) แม้คุณภาพ OCR ของบางภาพจะไม่สมบูรณ์นักจากต้นฉบับที่บีบอัดสูง แต่สามารถดึงแก่นสาร (เช่น คำสำคัญ, ขั้นตอน, คำสั่ง) ออกมาได้ และทุกแถวถูกผูกกลับด้วย source_url ของไฟล์ภาพและ page ของหน้าที่พบภาพเพื่อการตรวจสอบย้อนกลับ

สำหรับ เอกสารชนิด html ตัวอย่างแสดงทั้งไฟล์ข้อความที่ให้ดาวน์โหลด (เช่น .txt ฝั่ง reg.chula ที่มีโครงสร้างกึ่งตาราง) และหน้าเว็บของหน่วยงานนักศึกษา (sa.chula.ac.th) ซึ่งมี title ถูกสกัดจากเมตาดาตา และมีเนื้อหาภาษาไทยเป็นหลัก ระบบดึงข้อความด้วย trafilatura แล้วตัดซ้ำด้วย content_hash ทำให้หน้าเว็บที่มีรูปแบบ URL แตกต่างกันเล็กน้อยไม่ถูกนับซ้ำใน documents.jsonl คุณภาพตัวอย่างที่เห็นสอดคล้องกับกฎคัดกรอง—ข้อความผ่านเกณฑ์ความยาวและประกอบด้วยอักขระชุดอนุโลมเป็นส่วนใหญ่จึงพร้อมสำหรับการใช้งานต่อ

4.4 การทำ chunking และแปลงข้อมูลให้พร้อมฝังเวกเตอร์

4.4.1 ภาพรวมเทคนิคและพารามิเตอร์

เป้าหมายของส่วนนี้คือแปลงเอกสารข้อความจาก documents.jsonl ให้เป็น “ชิ้นข้อมูล (chunks)” ที่ขนาดพอดีและต่อเนื่องพอสมควรสำหรับการฝังเวกเตอร์ โดยคงการตรวจสอบย้อนกลับถึงเอกสารแม่ไว้ครบ สคริปต์ทำงานแบบ สตรีมที่ละบรรทัด (JSONL) เริ่มจาก อ่านเอกสาร → เลือกโปรไฟล์ chunking → แบ่ง

ข้อความด้วยตัวค้นซ้ำขึ้น ๆ → บังคับหั่นแข็งหากยังเกินเพดาน → เขียนแถวผลลัพธ์ลง chunks.jsonl
ทันที พร้อมนับสถิติรวมท้ายงาน เทคนิคการหั่น คือ “recursive split” ไล่ตัวค้นจากหยาบไปละเอียด

สคริปต์ใช้ลิสต์ตัวค้น SEPARATORS = [“\n\n”, “\n”, “. “, “ “, “”] ไล่จากย่อหน้า บรรทัด
ประโยค ช่องว่าง (ว่าง) เพื่อพยายามรักษาโครงสร้างธรรมชาติของข้อความก่อน หากยังยาวเกินเพดานจะเข้าสู่
ขั้น hard chunk ซึ่งหั่นตามจำนวนโทเค็นแบบหน้าต่างเลื่อนพร้อม overlap เพื่อคงบริบทข้ามขึ้น การมีตัวค้น
ว่าง เป็น “รู้สุดท้าย” รับประกันว่าไม่มีขึ้นใดเกินเพดานที่กำหนด การเลือกโปรไฟล์อัตโนมัติ (ชนิดข้อความ
ต่างกัน ขนาด/ซ้อนทับต่างกัน)

ตัวเลือกจะอิงฟิตในเอกสารและลักษณะข้อความ:

- image (มาจาก OCR) ⇒ ตั้งโปรไฟล์ ocr_raw (ไม่มีโครงสร้าง)
- pdf_md, html, text ⇒ ตรวจว่า “มีโครง” หรือไม่ด้วยฟังก์ชัน looks_structured (เช็คหัว
Markdown #, ตาราง HTML/Markdown, และสัดส่วนบรรทัด)
 - ถ้ามีโครง ⇒ โปรไฟล์ structured
 - ถ้าไม่ ⇒ โปรไฟล์ ocr_raw

เหตุผล: เนื้อหาที่เป็น Markdown/HTML มักมีย่อหน้า-หัวข้อที่ดี จึงรองรับขึ้นที่ใหญ่กว่าได้ ส่วนข้อความจาก
OCR มักมี noise/ตัดคำไม่ดี จึงใช้ขึ้นเล็กลงเพื่อลดการนำส่วนเกินที่ไม่เกี่ยวข้องเข้าไปฝัง
พารามิเตอร์หลัก

- ขนาดขึ้น (tokens) และอัตราซ้อนทับ
 - structured : CHUNK_SIZE_STRUCT = 512 และ CHUNK_OVERLAP_STRUCT = 0.12
→ คิดเป็น ซ้อนทับ ~12% ของแต่ละขึ้น
 - ocr_raw : CHUNK_SIZE_OCR = 256 และ CHUNK_OVERLAP_OCR = 0.20 → ซ้อนทับ
~20% เพื่อช่วยพยุงบริบทในข้อความคุณภาพต่ำ
 - ในโค้ดจะคำนวณ จำนวนโทเค็นที่ซ้อนทับจริง เป็น $ov = \text{int}(\text{round}(\text{size} * \text{overlap}))$
- การนับโทเค็น
 - โหมดหลักใช้ tiktoken (encoding cl100k_base) เพื่อให้ขนาดขึ้นสอดคล้องกับโมเดล
สมัยใหม่

- ฟังก์ชันตรวจว่า “มีโครงสร้าง” (looks_structured)
 - ตรวจพบหัว # ของ Markdown, <table> HTML, หรือตาราง Markdown (| ... | + บรรทัดคั่น ---) ถือว่า *structured*
 - กรณีไม่พบ ใช้สัดส่วนจำนวนบรรทัดต่อความยาวข้อความเป็นตัวช่วยอนุมาน

กลไก ID/ลำดับ และการกำจัดซ้ำ

- เอกสารแม่: คำนวณ parent_id จาก content_hash เพื่อให้ “ตัวระบุเอกสารแม่” เสถียรแม้รีรันซ้ำ
- ชี้น: id = f”{parent_id}:{chunk_index}” และบันทึก chunk_total เพื่อให้รู้ว่าชี้นนี้เป็นลำดับที่เท่าไรจากทั้งหมด (ช่วยเวลาประกอบผลลัพธ์)

4.4.2 กลไกการทำ chunking

ส่วนนี้จะอธิบาย “วิธีหั่นข้อความให้เป็นชี้น” ตั้งแต่การเลือกรูปแบบชี้น (profile) → การคำนวณขนาด/ซ้อนทับ → อัลกอริทึมแยกตามตัวคั่นแบบไล่ชั้น (recursive) → การหั่นแข็งแบบเลื่อนหน้าต่าง (hard chunk) เพื่อรับประกันว่าไม่มีชี้นใดเกินเพดานโทเค็น ทั้งหมดทำงานบนข้อความดิบที่อ่านจาก documents.jsonl

เริ่มจากการ กำหนดโปรไฟล์ชี้น ต่อเอกสารหนึ่งชี้นข้อความ ถ้า source_type == “image” จะใช้โปรไฟล์ ocr_raw ทันที เพราะมาจาก OCR และมี noise มากกว่า ถ้าเป็น pdf_md|html|text ระบบจะประเมินว่า “ดูมีโครงสร้าง” ด้วยฟังก์ชัน looks_structured ซึ่งตรวจหวัข้อ Markdown (#), ตาราง HTML/Markdown และสัดส่วนบรรทัด ถ้าผ่านจะใช้โปรไฟล์ structured มิฉะนั้นกลับไป ocr_raw เหตุผลคือข้อความมีโครงสร้างสามารถรับ “ชี้นใหญ่กว่า” ได้โดยยังคงความหมาย ส่วน OCR ควรใช้ชี้นเล็กเพื่อลดการปนเปื้อนของข้อความหลงทาง

เมื่อทราบโปรไฟล์แล้ว สคริปต์คำนวณ จำนวนโทเค็นซ้อนทับ จากสัดส่วนที่ตั้งค่าไว้ในคอนฟิก structured → size=512, overlap_rate=0.12 และ ocr_raw → size=256, overlap_rate=0.20 โดยแปลงเป็นโทเค็น เพื่อใช้เป็น “ทาง” ที่จะต่อเข้ากับชี้นถัดไป ช่วยรักษาบริบทข้ามชี้น (เช่น ย่อหน้าต่อเนื่อง หรือประโยคที่โดนตัดกลาง)

ฟังก์ชัน _chunk_once_at_sep คือกลไกระดับจุลภาคสำหรับ “หั่นหนึ่งรอบด้วยตัวคั่นเดียว” หลักการคือสะสมชี้นส่วนลงบัฟเฟอร์ (buf) จนจะล้นเพดานโทเค็น เมื่อเกินจึง “ปิดชี้น” และเริ่มสะสมใหม่ โดย ผนวก overlap ของทางชี้นก่อน เข้าไปเป็นจุดเริ่มต้นของบัฟเฟอร์ถัดไปเพื่อคงบริบท จากนั้นถ้าบัฟเฟอร์ใหม่ยังเกินเพดาน ระบบจะยอมแตก “แข็ง” ด้วย _hard_chunk เฉพาะกรณีนั้นจบ เพื่อไม่ให้ติดลูปและรับประกันว่าเดินหน้าต่อได้ทุกสถานการณ์

กรณีที่หลังจากผ่านตัวค้นครบทุกตัวแล้วยังมีขึ้นใด “เกินเพดาน” อยู่ สคริปต์จะใช้ `_hard_chunk` ซึ่งเป็นหน้าต่างเลื่อนตามจำนวนโทเค็นจริง อัลกอริทึมนี้กำหนด $\text{step} = \text{size} - \text{overlap}$ แล้วเลื่อนไปเรื่อย ๆ จนสุดข้อความ ทำให้ทุกชิ้นมีขนาดไม่เกิน `size` แน่นนอน และมีการซ้อนทับตามที่สั่ง

สคริปต์ยังให้ความสำคัญกับ ความเสถียรของผลลัพธ์ เมื่อได้ลิสต์ขึ้นแล้ว จะกำหนดตัวต้นขึ้นด้วย `id` โดยใช้ `parent_id` มาจาก `content_hash` ของเอกสาร (ถ้ามี) หรือ `sha1(source_url)` ทำให้ค่าตัวระบุคงที่ข้ามการรันซ้ำตราบเท่าที่เนื้อหาเดิมเท่าเดิม ขณะเดียวกัน มีชุด `seen_ids` ป้องกันการเขียนขึ้นซ้ำกรณีพิเศษ (เช่น เอกสารซ้ำในอินเทอร์เน็ต) ช่วยให้ `chunks.jsonl` ไม่มีรายการซ้ำโดยไม่ต้องหยุดงาน

4.4.3 การเปลี่ยนฟอร์แมตสำหรับระบบฝังเวกเตอร์

วัตถุประสงค์ของส่วนนี้คือทำให้ “ชิ้นข้อความ (chunks)” ที่ได้จาก 4.4.2 อยู่ในรูปแบบที่ส่งเข้าโมเดลฝังเวกเตอร์ BGE-M3 ได้ทันที โดยไม่ต้องมีการแปลงเพิ่มเติมในขั้นตอนฝัง เพื่อความเรียบง่ายและความเข้ากันได้ของตัวโมเดล เนื้อหาที่จะฝังเก็บไว้ที่ฟิลด์ `text` เพียงฟิลด์เดียว ส่วนฟิลด์อื่นทำหน้าที่เป็นกุญแจอ้างอิงและตัวชี้วัดคุณภาพระหว่างทางเท่านั้น

อินพุตของเราคือไฟล์ `chunks.jsonl` ซึ่งบันทึกชิ้นข้อความที่ละบรรทัดพร้อมเมตาดาตา เช่น `id`, `parent_id`, `source_url`, และ `text` เนื้อหาหลักของการฟอร์แมตจึงมีเพียงสองภารกิจ ทำความสะอาดข้อความเล็กน้อยให้เหมาะกับการฝัง และควบคุมขนาดโทเค็นให้ไม่เกินเพดานเผื่อ (เช่น 8,000 โทเค็น) เพื่อป้องกันกรณีขอบที่ขึ้นยาวผิดปกติ ทั้งสองอย่างนี้ทำ โดยตั้งใจโครงสร้างที่ช่วยการค้นคืน เช่น หัวข้อ (# ...), รายการ, และตาราง Markdown/TAG ที่สำคัญไว้ครบ

สำหรับไอดีของข้อมูล เราคงกลไก `parent_id` และ `id` ตามแบบเดียวกับชิ้นข้อความ เพื่อให้ฝั่งดัชนีสามารถรวมขึ้นกลับเป็นเอกสารแม่เวลาแสดงปริวิวหรือทำการจัดกลุ่มผลลัพธ์ `parent_id` ยึดจาก `content_hash` ของเอกสาร หรือ `sha1(source_url)` ในขณะที่ `id` ของแต่ละชิ้นมีรูปแบบคงที่ `parent_id:chunk_index` สิ่งนี้ทำให้การอัปเดตเวกเตอร์ในอนาคต (เช่น re-embed เฉพาะเอกสารหนึ่งชุด) ทำได้แบบเจาะจงโดยไม่กระทบแถวอื่น

การควบคุมความยาวใช้ตัวนับโทเค็นเดียวกับขึ้น กลยุทธ์ในการทำคือ ตรวจ-ตัด-ยืนยัน ตรวจนับก่อนฝัง หากเกินเพดานให้ตัดหัวทางอย่างระมัดระวังด้วยฟังก์ชันตัดตามโทเค็น แล้วนับซ้ำเพื่อยืนยันว่าจำนวนโทเค็นที่จะส่งเข้าโมเดลไม่เกินจริง ฟิลด์ `n_tokens` ที่บันทึกไว้หลังตัดจึงสะท้อนของจริง ไม่ใช่ค่าที่ประเมินไว้ล่วงหน้า

คุณภาพของข้อความก่อนฝังยังพึ่งการทำความสะอาดเบา ๆ เท่านั้น เช่น แปลงบรรทัดให้เป็น `\n` เดียว, ตัดช่องว่างส่วนเกิน, และลบอักขระควบคุมที่ไม่จำเป็นโดยไม่ตัดแปลงการจัดรูปย่อหน้าและหัวข้อ เหตุผลคือการค้นคืนเชิงความหมายได้ประโยชน์จากสัญญาณโครงสร้าง (หัวข้อ ตาราง หัวคอลัมน์) มากกว่าข้อความที่ถูก “รีด” จนราบเรียบ และการแตะโครงสร้างมากเกินไปอาจทำให้สูญเสียบริบทที่มีผลต่อความหมายของข้อความ

ผลลัพธ์สุดท้ายคือไฟล์ `bge_m3_input.jsonl` ที่ยังคงชุดคีย์เดิมเกือบทั้งหมดเพื่อให้ดีบั๊กง่ายและพร้อมประกอบกับระบบจัดดัชนี/เมตริกในภายหลัง โดยแถวหนึ่งบรรทัดเป็นตัวแทนขึ้นหนึ่งขึ้นที่พร้อมฝัง เวิร์กโฟลว์ฝังฝังจึงเหลือเพียงอ่านไฟล์นี้เป็นสตรีม ส่งฟิลด์ `text` เข้าโมเดลตามลำดับ (batch ตามความเหมาะสม) และบันทึกเวกเตอร์กลับโดยใช้ `id` เป็นคีย์หลัก หากภายหลังต้องปรับค่าพาดานโทเค็นหรือกติกาทำความเข้าใจ สะอาด ก็เปลี่ยนที่ขึ้นฟอร์แมตนี้เพียงจุดเดียวโดยไม่ต้องแก้งานฝังฝังหรือตัวสร้างดัชนี

4.5 ระบบฝังเวกเตอร์

4.5.1 ภาพรวมการทำงานของระบบฝังเวกเตอร์ และเหตุผลการเลือกโมเดล

ระบบฝังเวกเตอร์ของงานนี้ทำงานต่อเนื่องจากขั้น 4.4 (ที่ได้ `chunks` พร้อมฝัง) โดยรับ `text` ของแต่ละชิ้นเข้าโมเดลฝังเวกเตอร์เพื่อแปลงเป็นเวกเตอร์เชิงความหมาย จากนั้นเก็บเวกเตอร์พร้อมเมทาดาทาลงฐานเวกเตอร์ (เช่น ChromaDB) เพื่อรองรับการค้นคืนแบบ semantic/kNN ในภายหลัง กระบวนการจึงแบ่งออกเป็น 3 ตอนหลัก: (1) เตรียมอินพุต ตรวจสอบความยาวและทำความสะอาดข้อความเบา ๆ (รักษาหัวข้อ/ตารางไว้), (2) ฝัง (Embed) ส่งข้อความเข้าสู่โมเดลผลิตเวกเตอร์แบบ L2-normalized เพื่อให้ใช้ระยะวัดอย่าง cosine/dot ได้เสถียร, และ (3) จัดเก็บ/จัดดัชนี โดยบันทึกเวกเตอร์ลงคอลเลกชันของเวกเตอร์สโตร์ พร้อม `id`, `parent_id`, `source_url` สำหรับการอ้างอิงย้อนกลับและอัปเดต/ลบภายหลัง

- เหตุผลในการเลือกโมเดลฝังหลัก — BGE-M3

เลือก BGE-M3 (BAAI/bge-m3) เป็นโมเดลเริ่มต้น เพราะออกแบบมาสำหรับงานค้นคืนจริง โดยชูจุดเด่น สาม “M” *Multi-Functionality* (รองรับ dense, multi-vector และแม้แต่ sparse/lexical), *Multi-Linguality* (รองรับมากกว่า 100 ภาษา จึงเหมาะกับชุดข้อมูลไทย-อังกฤษปนกัน), และ *Multi-Granularity* (รองรับข้อความสั้น-ยาวได้จนถึงราว 8,192 โทเค็น ต่ออินพุต) ซึ่งครอบคลุมขนาดขึ้น 256/512 โทเค็นที่เราออกแบบไว้ และยังมี headroom หากต้องขยายขนาดในอนาคต นอกจากนี้ โมเดลเปิดเผยบน Hugging Face ภายใต้สัญญาอนุญาต MIT ทำให้ใช้งานภายในเครื่อง/ออฟไลน์ได้ยืดหยุ่นโดยไม่ต้องพึ่ง API ภายนอกในสภาพแวดล้อมที่อ่อนไหวด้านข้อมูล

- เปรียบเทียบตัวเลือกที่พบใช้งานบ่อย

- OpenAI (ตระกูล `text-embedding-3-*`) — จุดแข็งคือคุณภาพและความสม่ำเสมอบนงาน retrieval เชิงอุตสาหกรรม พร้อมรองรับอินพุตยาว ~8,192 โทเค็น ต่อชิ้น (ทั้งรุ่น *small* และ *large* ตามตารางในคู่มือ embeddings ทางกร) เหมาะเมื่อยอมรับการเรียก API และข้อจำกัดข้อมูล/ต้นทุนได้ โมเดลนี้ยังมีคู่มือการนับโทเค็น (ผ่าน tiktoken) ให้ควบคุมพาดานอินพุตอย่างเป็นระบบ ซึ่งสอดคล้องกับวิธีที่เราเตรียมขึ้นไว้ล่วงหน้า แต่มีข้อเสียอย่าง token limit ที่กำหนดให้ผู้ใช้ที่อยู่ในระดับต่ำ ไม่สามารถใช้งานได้เกิน 1,000,000 token per min ทำให้ไม่ใช้โมเดลนี้

- Cohere (เช่น embed-multilingual-v3.0) — จุดเด่นคือความสามารถ “พหุภาษา” และ ecosystem ที่หนุนงาน RAG โดยตรง เอกสารทางการให้รายละเอียดพารามิเตอร์ max_tokens และข้อจำกัดต่อ batch ทั้งนี้ บนการให้บริการบางแพลตฟอร์มที่เป็นผู้รีแพ็คเกจ เช่น Oracle OCI มีการระบุ เพดาน ~512 โทเค็น/อินพุต จึงควรตรวจสอบข้อกำหนดของผู้ให้บริการที่ใช้งานจริง ก่อนตัดสินใจ

โดยสรุป โมเดลที่เลือก (BGE-M3) ให้สมดุลระหว่าง ความครอบคลุมภาษา-ความยาวอินพุต-ความยืดหยุ่นในการดีพลอย (ทำงานออฟไลน์ได้และยอมรับการปรับแต่งบนโค้ดเราเอง) ขณะที่ยังเปิดทางให้สลับไปใช้บริการเชิงพาณิชย์อย่าง OpenAI/Cohere ได้ง่าย หากเงื่อนไขด้านกำกับข้อมูล/ต้นทุนและ TPM ทำให้เหมาะสมกว่าในสภาพแวดล้อมโปรดักชันของผู้ใช้

4.5.2 กลไกฝังเวกเตอร์และจัดเก็บใน

ส่วนนี้อธิบายสคริปต์ที่ทำหน้าที่ “อ่านขึ้นข้อความ → ฝังเป็นเวกเตอร์ → จัดเก็บลงฐานเวกเตอร์” แบบเป็นชุด (batch) โดยยัดอินพุตจากไฟล์ embeddable.jsonl ซึ่ง หนึ่งบรรทัดแทนหนึ่งชิ้น (chunk) และต้องมีอย่างน้อยฟิลด์ id, text, และ metadata (ภายในควรมี {"url": ...} ไว้ใช้บริหารวงจรชีวิตข้อมูล ภายหลัง)

ส่วนแรกของสคริปต์เป็นการเตรียม คอนฟิกผ่านตัวแปรแวดล้อม ได้แก่ พารามิเตอร์ (IN_JSONL), โฟลเดอร์เก็บ Chroma (CHROMA_PATH), ชื่อคอลเลกชัน (COLLECTION_NAME), ชื่อโมเดลฝัง (EMBEDDING_MODEL ค่าเริ่มต้น BAAI/bge-m3), ระยะวัดของดัชนี (HNSW_SPACE ค่าเริ่มต้น “cosine”), และขนาดชุด (BATCH_SIZE) การตั้งค่าเหล่านี้ทำให้สคริปต์ยืดหยุ่นต่อสภาพแวดล้อมจริง เช่น เครื่องทดลอง/เครื่องโปรดักชัน โดยไม่ต้องแก้ไขโค้ดแกนกลาง

ถัดมาเป็น คลาสฝังเวกเตอร์ BGEM3EmbeddingFunction ซึ่งห่อหุ้ม SentenceTransformer ของโมเดลที่ ระบุไว้ เมื่อถูกเรียกด้วยลิสต์ข้อความ ฟังก์ชันจะคืนเวกเตอร์แบบ L2-normalized (normalize_embeddings=True) แล้วแปลงเป็น float32 เพื่อประหยัดพื้นที่จัดเก็บ ผลคือเวกเตอร์ทุกตัวอยู่บนทรงกลมหน่วย เหมาะกับการใช้ cosine distance ที่เราตั้งไว้ใน Chroma ช่วงนี้คือจุดที่ผูก “โมเดลฝัง” เข้ากับ “เวกเตอร์สโตร์”: เมื่อเราส่ง documents ให้คอลเลกชัน Chroma ระบบจะเรียกฟังก์ชันนี้ให้โดยอัตโนมัติและบันทึกเวกเตอร์พร้อมเอกสารกับเมทาดาตาในครั้งเดียว

จากนั้นสคริปต์สร้าง คอนเนกชันแบบ persistent ไปยัง Chroma (chromadb.PersistentClient) และเรียก get_or_create_collection(...) เพื่อเปิดคอลเลกชันตามชื่อ หากยังไม่มีก็สร้างใหม่โดยกำหนด embedding_function=... และเมทาดาตาดัชนี {"hnsw:space": "cosine"} ตั้งแต่นั้น การเก็บแบบ persistent ทำให้ข้อมูลยังอยู่แม้รีสตาร์ทเครื่องหรือกระบวนการ

แกนกลางของงานอยู่ที่ ลูปอ่านไฟล์ JSONL แบบสตรีม: โปรแกรมจะอ่านทีละบรรทัด แปลงเป็นอ็อบเจกต์ JSON แล้วดึง id, text, metadata สะสมไว้ในลิสต์ ids/docs/metadatas จนครบ BATCH_SIZE จึงเรียก collection.add(...) เพื่อ “ฝัง + เพิ่ม” ลงดัชนีครั้งเดียวโดยจะทำงานเป็นชุด การทำงานแบบเป็นชุดช่วยลดภาระทั้งฝั่งโมเดลและฝั่งฐานเวกเตอร์ หลังจบบรรทัดอ่าน จะมีการฝังก่อนค้างอยู่ก่อนพิมพ์สรุปจำนวนรวม (collection.count())

```
[ADD] +256 (total=108544)
[ADD] +256 (total=108800)
[ADD] +93 (total=108893)
[DONE] collection.count() = 108893
```

รูปที่ 4.2 ผลลัพธ์การฝังเวกเตอร์

4.5.3 การลบและอัปเดตเวกเตอร์

ส่วนนี้อธิบาย “วินัยการดูแลดัชนีเวกเตอร์” หลังจากฝังข้อมูลเสร็จแล้ว โดยยึดดีไซน์ไอดีและเมทาดาตาที่เรากำหนดไว้ใน 4.4–4.5.2 เป็นหลัก จุดมุ่งหมายคือให้ฐานเวกเตอร์ สอดคล้องกับแหล่งความจริง (JSONL ล่าสุด) อยู่เสมอ และสามารถย้อนรอยการแก้ไขได้

ชุดความสามารถที่รองรับ

- ลบ (delete)
 - ลบแบบระบุเงื่อนไขเมทาดาตา: where={"url": <value>} เหมาะสำหรับ “ลบทั้งเอกสารแม่” ทุกชิ้นในคราวเดียว
 - ลบแบบระบุไอดี: ids=[...] ใช้เมื่อทราบรายการขึ้นที่ต้องการลบอย่างเฉพาะเจาะจง
- อัปเดต (update)
 - แก้ documents/embeddings/metadatas “ของไอดีที่มีอยู่” โดยเขียนทับค่าเดิม มีประโยชน์เมื่อ ข้อความเดิมใช้ได้ แต่ต้องปรับเมทาดาตา เช่น เพิ่ม tag, เปลี่ยน url หรือรี-ฝังเฉพาะบางชิ้น

4.6 ระบบค้นคืนคำตอบแบบเวกเตอร์

4.6.1 Retriever

ขั้น Retriever ทำหน้าที่แปลงคำถามของผู้ใช้เป็นเวกเตอร์ (ฝังคำถามใช้โมเดลเดียวกับฝังเอกสารที่ฝังไว้ล่วงหน้า) แล้วเรียกฐานเวกเตอร์ ChromaDB เพื่อดึง “ชิ้นข้อความผู้สมัคร” ชุดแรก พร้อมข้อมูลประกอบ (เอกสาร/เมทาเดตา/ระยะ) สำหรับขั้นตอนถัดไป (*re-rank* หรือสังเคราะห์คำตอบ)

อินพุต-เอาต์พุต

- อินพุต: ตัวแปรสตริงคำถาม หรือ query (ไทย/อังกฤษ)
- เอาต์พุต: ลิสต์ผลค้นคืนจำนวน k รายการ โดยแต่ละรายการประกอบด้วย documents (ชิ้นข้อความ), metadatas เช่น url, และ distances ที่ Chroma คืนกลับมา

วิธีการ

โค้ดเปิดคอลเลกชัน Chroma ที่ตั้งค่าระยะ cosine ไว้ล่วงหน้า จากนั้นเรียก `collection.query(...)` ด้วย `query_texts=[query]` และระบุ `include=["documents","metadatas","distances"]` เพื่อให้ได้ทั้งข้อความ เนื้อหาเมทาเดตา และ “ระยะ” ระหว่างเวกเตอร์คำถามกับเวกเตอร์เอกสาร แล้วในไน้ตบุ๊กมีการแปลงระยะเป็น “คะแนนความใกล้เคียง” สำหรับเรียงลำดับเบื้องต้น จากนั้นผลลัพธ์ถูกแปลงเป็นชุด contexts โดยวาง URL (จาก metadatas) ไว้หน้าหน้าเนื้อหาแต่ละชิ้น เพื่อให้ตรวจสอบย้อนกลับแหล่งที่มาได้ชัดเจนในขั้นสังเคราะห์คำตอบ

พารามิเตอร์สำคัญ (อ้างอิงไน้ตบุ๊ก)

- จำนวนผลรอบแรก k ที่ขอจาก Chroma เช่น TOP_K หรือค่าที่กำหนดในเซลล์ก่อนหน้านี้
- ตัวเลือก include อย่างน้อย documents, metadatas, distances
- เงื่อนไขกรองล่วงหน้า (ถ้ามี) ผ่านอาร์กิวเมนต์ `where=...` เพื่อจำกัดโดเมน/แหล่ง/ช่วงเวลาในเมทาเดตา

4.6.2 Re-ranker

ขั้น Re-ranker มีหน้าที่ “คัดให้ตรงคำถามจริง ๆ” จากผลค้นคืนชุดแรกที่ได้จาก Retriever โดยใช้โมเดลแบบ cross-encoder ให้คะแนนความเกี่ยวข้องจาก คู่ (query, document) แล้วเลือกเฉพาะ Top-FINAL_K ส่งต่อไปยังขั้นสังเคราะห์คำตอบ

หลักการ

ต่างจากการค้นแบบเวกเตอร์ (bi-encoder) ที่คำนวณความใกล้เคียงจากเวกเตอร์สองฝั่งอย่างอิสระ โมเดล cross-encoder จะ “อ่านคำถามพร้อมเอกสาร” แล้วให้คะแนนเชิงบริบท จึงลดเคสที่คำหรือ

เวกเตอร์ใกล้เคียงกันแต่ “ไม่ใช่คำตอบ” ได้ดี โคดในโน้ตบุ๊กเลือกใช้ FlagReranker และกำหนด `normalize=True` เพื่อให้สเกลคะแนนอยู่ในช่วง 0–1 นำไปจัดอันดับได้ตรงไปตรงมา

อินพุต-เอาต์พุต

- อินพุต: เอกสารจาก Retriever จำนวน `INITIAL_K` (ข้อความ + เมทาดาตา) และสตริง query
- เอาต์พุต: เอกสารที่ผ่านคัดกรอง `FINAL_K` พร้อมคะแนน `scores` (ถูกจัดเรียงมาก→น้อย) และเมทาดาตาเดิม (เช่น url) สำหรับนำไปประกอบบริบท

พารามิเตอร์ที่อ้างอิงจากโน้ตบุ๊ก

- `RETRIEVAL_MODE="rerank"` — เปิดโหมดคัดกรองด้วย cross-encoder
- `INITIAL_K` — จำนวนผลจาก Retriever ชุดแรกที่ส่งเข้าคัดกรอง
- `FINAL_K` — จำนวนผลสุดท้ายหลัง re-rank
- `RERANKER_MODEL="BAAI/bge-reranker-v2-m3"` — โมเดลที่ใช้กับ FlagReranker

ในโคดตั้ง `use_fp16=True` เพื่อเร่งความเร็วเมื่อมี GPU ที่รองรับ และลด latency

ลำดับการทำงาน

1. ดึงแคนดิเดตจาก Chroma (ยังไม่ดูระยะ):
ใช้ `col.query(query_texts=[query], n_results=INITIAL_K, include=["documents","metadatas"])` เพื่อได้ docs, metas
2. คำนวณคะแนน re-rank ด้วย cross-encoder:
สร้างรายการคู่ `[ [query, d] for d in docs ]` แล้ว `compute_score(..., normalize=True)` ได้ scores ช่วง 0–1
3. จัดอันดับและเลือก Top-FINAL_K:
เรียงดัชนีตามคะแนนมาก→น้อย แล้วเลือกเฉพาะหน้าสุดและซิงก์ docs/metas/scores ให้ตรงกัน
4. เตรียม context ต่อขึ้น (ใช้ต่อใน 4.6.3):
แปะ URL จาก metas นำหน้าตัวเนื้อหา เพื่อให้ตรวจย้อนแหล่งที่มาได้

4.6.3 Answer Synthesizer (สังเคราะห์คำตอบพร้อมอ้างอิง)

Answer Synthesizer ทำหน้าที่รับ contexts ที่ประกอบจากผลค้นคืน (และ/หรือผ่าน re-rank แล้ว) มาต่อเป็นก้อนบริบท (`context_blob`) แล้วสร้างพรอมป์ให้โมเดลภาษา (LLM) ตอบ จากบริบทเท่านั้น

หากไม่มีหลักฐานเพียงพอให้ระบุว่า “ไม่พบข้อมูล” แนวทางนี้ทำให้คำตอบโปร่งใส ตรวจสอบย้อนกลับได้ เพราะในแต่ละชั้นของบริบทมีการวาง URL นำหน้าไว้ตั้งแต่ชั้นประกอบบริบท อินพุต-เอาต์พุต.

- อินพุต: 1. รายการ contexts ที่มีรูปแบบ URL: <link>\n<เนื้อหา> ต่อรายการ และ 2. สตริงคำถาม query
- เอาต์พุต: ข้อความ answer ที่สังเคราะห์จากบริบทเดียวกัน โดยรักษาสำนวนกระชับ ชัดเจน และใช้ภาษาให้สอดคล้องกับภาษาของคำถาม

หลักการและเหตุผลการทำงาน

1. ประกอบบริบทเดียว: โคดนำ contexts มาต่อกันด้วยตัวคั่น \n\n---\n\n เพื่อให้ LLM มองเห็นขอบเขตของหลักฐานแต่ละชั้นชัดเจน และคง URL ของแหล่งที่มาทุกชั้นเพื่อรองรับการอ้างอิง
2. พรอมป์กำกับพฤติกรรม: ข้อความพรอมป์ (ไทย) กำชับให้ “ตอบจาก Context เท่านั้น ถ้าไม่พบให้บอกว่าไม่พบ” และให้รักษาสำนวนกระชับ—ช่วยลดโอกาสเกิด *hallucination* และคุมความยาวคำตอบ
3. เรียก LLM ในรูป Chat Completion: โคดกำหนดโมเดลผ่านตัวแปร OPENAI_MODEL (ระบุค่าในโน้ตบุ๊กเป็น “gpt-4o-mini”) และส่งพรอมป์ทั้งหมดในบทสนทนาแบบ single-turn (messages=[{"role": "user", "content": prompt}]) พร้อมเพดานโทเค็นคำตอบ (max_tokens=...) เพื่อควบคุมต้นทุน/ความยาวผลลัพธ์

พารามิเตอร์สำคัญ

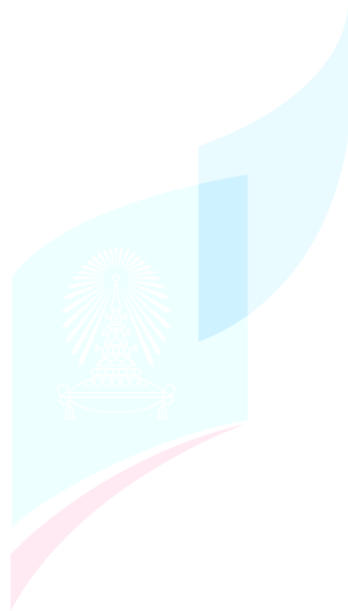
- OPENAI_MODEL — ชื่อโมเดลที่ใช้สังเคราะห์คำตอบ ในงานนี้ใช้ gpt 5 mini
- คีย์/การตั้งค่าคล้ายสุมอื่น ๆ เช่น temperature = 0.2

จากนั้นชั้นนี้จะคืน answer ไปยังผู้ใช้ หรือไปยังเลเยอร์ตอบกลับของระบบ

ข้อควรระวัง

- ไม่มีบริบท: หาก contexts ว่าง/สั้นมาก ให้ส่งพรอมป์ที่สะท้อน “หลักฐานไม่พอ” เพื่อให้ LLM ตอบว่าไม่พบข้อมูลแทนการคาดเดา
- ขนาดบริบท: ควรควบคุมจำนวนชั้น/ความยาว context_blob ตั้งแต่ชั้นคัดเลือก (4.6.2) เพื่อไม่ให้ล้นหน้าต่างโมเดล

- อ้างอิง: การวาง URL ในแต่ละชั้นตั้งแต่ต้นทำให้คำตอบสามารถอ้างอิงได้ง่าย—ถึงแม้โค้ดจะไม่ได้ตัดคำตอบให้ใส่เชิงอรรถอัตโนมัติ แต่ผู้ใช้/ส่วนแสดงผลสามารถดึง URL จากบริบทที่ใช้จริงไปแสดงคู่คำตอบได้ทันที
- สภาพแวดล้อมรัน: ต้องมีการตั้งค่าสำหรับ client_llm และเครือข่ายพร้อมใช้งาน ในกรณีรันไม่สำเร็จ จะเตือนว่าไม่มีเอาต์พุตถูกสร้าง (สอดคล้องกับข้อควรระวังของโน้ตบุ๊ก)



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

บทที่ 5

ผลการทดลองและการประเมินผล (Evaluation)

5.1 สรุปผลลัพธ์การดึงข้อมูล (Ingestion Summary)

ตามขอบเขตที่กำหนด ระบบดึงข้อมูลจากโดเมนทางการของจุฬาลงกรณ์มหาวิทยาลัย ได้แก่ reg.chula.ac.th, it.chula.ac.th และ sa.chula.ac.th (รวมทุกซับโดเมน) โดยรองรับเอกสาร 3 ประเภทคือ (1) หน้าเว็บ HTML, (2) PDF ที่ผ่านการแปลงเป็น Markdown และ (3) ภาพ/อินโฟกราฟิกที่ผ่าน OCR จากนั้นดำเนินการ canonicalize URL, คัดซ้ำ และบันทึกเมตาดาตาไฟล์ลง documents.jsonl เพื่อใช้เป็นฐานสำหรับการประเมินผลในบทนี้ ชุดข้อมูลฉบับที่รายงานนี้ถูกรวบรวมในช่วงเวลา 04 ต.ค. 2025 เวลา 00:15–02:54 น.

5.1.1 ภาพรวมเชิงปริมาณ

จากการประมวลผลไฟล์ documents.jsonl พบว่า

- จำนวนเอกสารรวม 2,473 รายการ
- ขนาดตัวอักษรรวม 35,502,176 อักขระ (ประมาณ 8,876,487 โทเคน*)
- ค่าเฉลี่ยต่อเอกสาร 14,355.9 อักขระ/เอกสาร (ประมาณ 3,589.4 โทเคน/เอกสาร)

* การประมาณโทเคนใช้อัตราแปลงเชิงประจักษ์ ~4 อักขระ/โทเคน เพื่อให้เทียบเคียงกับงานประเมิน downstream

5.1.2 การกระจายตามโดเมน

เมื่อรวมตามฐานโดเมน พบว่าเอกสารถูกกระจายดังนี้

- it.chula.ac.th จำนวน 937 เอกสาร คิดเป็น 37.89% ของเอกสารทั้งหมด (ขนาดรวม ~2.18M อักขระ)
- sa.chula.ac.th จำนวน 840 เอกสาร คิดเป็น 33.97% (ขนาดรวม ~3.32M อักขระ)
- reg.chula.ac.th จำนวน 690 เอกสาร คิดเป็น 27.90% แต่มี ขนาดข้อความรวม ~29.99M อักขระ หรือ 84.5% ของขนาดทั้งหมด
- อื่น ๆ รวม 6 เอกสาร (0.24%)

ข้อสังเกตเชิงคุณภาพ: แม้ reg.chula.ac.th จะมีสัดส่วน “จำนวนเอกสาร” ต่ำกว่าโดเมนอื่น แต่เป็นแหล่งเอกสารเนื้อหาเยอะมาก (เช่น หน้าสรุปรายวิชา/หมวดวิชา) จึงมีผลอย่างยิ่งต่อภาระการจัดทำดัชนี การค้นคืน และการจัดอันดับ (จะเชื่อมโยงในการอภิปรายผล 5.4)

5.1.3 การกระจายตามชนิดเอกสาร

เอกสารถูกจำแนกตามชนิดเป็น 3 กลุ่มหลัก ได้แก่

- HTML จำนวน 1,087 เอกสาร (43.95%)
- PDF จำนวน 623 เอกสาร (25.19%)
- IMAGE OCR จำนวน 763 เอกสาร (30.86%)

กลุ่ม IMAGE OCR มีความเสี่ยงด้านคุณภาพอักขระและเลย์เอาต์สูงกว่ากลุ่มอื่น จึงเป็นตัวแปรที่ต้องเฝ้าระวังในขั้นตอนจัดทำชิ้นส่วนความรู้ (chunking) และการดึงบริบทสำหรับการตอบ

5.1.4 การกระจายตามภาษา

ประเมินภาษาโดยใช้อัตราส่วนตัวอักษรไทย (threshold ≥ 0.20) พบว่า

- ภาษาไทย (TH) จำนวน 1,797 เอกสาร (72.66%)
- ภาษาอังกฤษ (EN) จำนวน 676 เอกสาร (27.34%)

สัดส่วนภาษาไทยที่มาก สอดคล้องกับขอบเขตการใช้งานจริงและเนื้อหาทางการของหน่วยงานภายใน ทำให้ระบบจำเป็นต้องคุมคุณภาพ OCR/การแบ่งชิ้นส่วนสำหรับภาษาไทยเป็นพิเศษ

5.1.5 เอกสารที่มี “ตาราง” และผลกระทบต่อการสกัด

จากการตรวจจับพื้เจอร์ “มีตาราง” พบเอกสารชนิดนี้รวม 174 รายการ (7.04% ของทั้งหมด) โดยภายในกลุ่ม PDF พบอัตรา “มีตาราง” สูงราว 27.93% ของเอกสาร PDF ทั้งหมด ลักษณะข้อมูลแบบตารางส่งผลต่อความถูกต้องของการสกัดและการจัดวางบริบท (layout-aware) หากไม่จัดการอย่างเหมาะสมอาจนำไปสู่การดึงบริบทผิด/ขาดหายและกระทบต่อคะแนนความสอดคล้อง (faithfulness) ในบทประเมินคำตอบภายหลัง

5.1.6 หมายเหตุวิธีนับและข้อจำกัดของชุดข้อมูลนี้

1. การจำแนกภาษาใช้เกณฑ์สัดส่วนตัวอักษรไทย ≥ 0.20 เพื่อสะท้อน “ภาษาเด่น” ของเอกสาร ซึ่งเพียงพอต่อการวิเคราะห์ภาพรวม แต่ไม่ใช่ตัวชี้วัดภาษาศาสตร์เชิงลึก
2. การประมาณจำนวนโทเคนเป็นค่าประมาณเพื่อใช้ในการวางแผนประสิทธิภาพ downstream (retrieval/rerank/answering) มิใช่มาตรวัดต้นทุนโมเดลที่แน่นอน
3. เอกสารขนาดใหญ่มาจาก reg.chula.ac.th ควรถูกสุ่มตรวจเชิงคุณภาพ (ตัวอย่าง 20 รายการขนาดใหญ่สุด) เพื่อสร้างกฎการแบ่งชิ้นส่วนเฉพาะกรณี (เช่น segment ตามหัวข้อ/ตาราง) ก่อนประเมินผลจริงใน 5.2–5.3

5.2 ผลลัพธ์ของการตัดชิ้นข้อความ (Chunking Results)

ระบบตัดชิ้นข้อความจากไฟล์กลาง documents.jsonl โดยใช้สองโปรไฟล์หลัก—structured (size=512, overlap=0.12) และ ocr_raw (size=256, overlap=0.20)—เพื่อให้เหมาะกับเนื้อหาที่เป็นหน้าเว็บ/เอกสารที่มีโครงสร้าง และเนื้อหาที่ผ่าน OCR ตามลำดับ ผลและข้อวิเคราะห์สรุปได้ดังนี้

5.2.1 ภาพรวมเชิงปริมาณ

จากเอกสาร 2,473 รายการ ถูกตัดเป็นชิ้นรวม 108,893 ชิ้น มีขนาดข้อความรวมประมาณ 45.67 ล้านอักขระ (≈ 11.46 ล้านโทเคน*) เฉลี่ย 419.4 อักขระ/ชิ้น (≈ 105.2 โทเคน/ชิ้น) ขนาดเฉลี่ยราว 100 โทเคน/ชิ้นถือว่าเล็กสำหรับการค้นคืนและการจัดอันดับซ้ำ (rerank) เพราะละเอียดเกินไป อาจทำให้เกิดภาวะ K สูงเกินจำเป็นในการรีเทรฟและรีแรนก์

5.2.2 การกระจายตามโปรไฟล์

- ocr_raw = 91,744 ชิ้น (84.25%)
- structured = 17,149 ชิ้น (15.75%)

สัดส่วน ocr_raw ที่สูงสะท้อนภาระงานฝั่ง OCR/เลย์เอาต์ค่อนข้างมาก จึงควรให้ความสำคัญกับการ layout-aware/heading-aware chunking และคุณภาพ OCR (ภาษาไทย) เพื่อคงโครงสร้างหัวข้อ ตาราง และหัวเรื่อง ให้บริบทที่ดึงมา “ตรงส่วน” มากขึ้น

5.2.3 การกระจายตามชนิดเอกสาร (ของชั้นที่ได้)

- HTML = 91,678 ชั้น (84.19%)
- PDF → Markdown = 15,733 ชั้น (14.45%)
- IMAGE → OCR = 1,482 ชั้น (1.36%)

วิเคราะห์: แม้ชั้นส่วนส่วนใหญ่มาจาก HTML แต่สัดส่วน PDF/ภาพ ก็มีนัยสำคัญพอที่จะกระทบคุณภาพบริบท—โดยเฉพาะหน้าที่มี ตาราง/แบบฟอร์ม และ ประกาศยาว จึงควรเพิ่มตัวสกัดตาราง/ยัดหัวเรื่องก่อนแบ่งชั้นในรอบถัดไป

5.2.4 ภาษาโดยประมาณของชั้น

ใช้เกณฑ์สัดส่วนอักขระไทย ≥ 0.20 (แบบหยาบ) พบว่า

- TH ~96.56% (105,142 ชั้น)
- EN ~3.44% (3,751 ชั้น)

วิเคราะห์: ชี้ว่าการใช้งานหลักเป็น ภาษาไทย จึงต้องเน้น คุณภาพ OCR-TH, การกรอง noise ภาษาไทย และ คีย์เวิร์ด/พจนานุกรมสองภาษาเพื่อครอบคลุมกรณีที่ใช้ภาษาม EN แต่บริบทหลักเป็น TH

5.2.5 โดเมนต้นทางของชั้น

- www.reg.chula.ac.th $\approx 44.90\%$ (48,898 ชั้น)
- web.reg.chula.ac.th $\approx 42.42\%$ (46,194 ชั้น)
- www.sa.chula.ac.th $\approx 7.52\%$ (8,193 ชั้น)
- www.it.chula.ac.th $\approx 5.13\%$ (5,584 ชั้น)

วิเคราะห์: ชั้นกว่า 87% มาจากกลุ่มโดเมน reg.chula.ac.th ซึ่งเป็นแหล่ง “ข้อบังคับ/คู่มือ/ระเบียบ” ขนาดยาว ส่งผลบวกต่อ “ความเป็นหลักฐาน” แต่เพิ่มความเสี่ยงเรื่อง ความยาว-เวอร์ชันเอกสารหลายปี จึงควรมี freshness/version gating ดันฉบับล่าสุดขึ้นก่อน และปรับ chunking ให้ยัดหัวข้อเฉพาะส่วน “ขั้นตอน/กำหนดเวลา/ค่าปรับ”

5.2.6 หมายเหตุเรื่องเมทาดาตา (สำคัญต่อชั้นฝังเวกเตอร์)

ระหว่างชั้น chunking พิลด์ metadata ของหลายชั้น ว่าง/ไม่มีข้อมูล (ใน chunks.jsonl พบว่า metadata ส่วนใหญ่เป็น None ขณะที่ source_url อยู่ระดับบนของระเบียบ) เพื่อให้เข้ากับข้อจำกัดของตัวฝังเวกเตอร์ BGE-M3 (BGE3) ซึ่ง ไม่ยอมรับเมทาดาตาที่ว่าง, ในขั้นเตรียมฝังเวกเตอร์จึงใช้แนวทาง มินิมอลเมทาดาตา โดย ใส่เฉพาะคีย์ url (แม้ปมาจาก source_url) และ ละเว้นคีย์อื่น ที่ส่วนใหญ่ไม่มีข้อมูลจริง

แนวทางนี้ทำให้ ผ่านข้อจำกัดของโมเดล และยังคง ความสามารถในการอ้างอิง/ลบ/อัปเดตตาม URL ได้ครบถ้วนใน UC-04/UC-05

5.3 การประเมินผลการค้นคืน (Retrieval)

การประเมินในส่วนนี้ใช้หลักฐานเชิงภาพจากผลค้นคืนของคำถามตัวอย่าง “ขั้นตอนลงทะเบียนเรียน จุฬาลงกรณ์มหาวิทยาลัย” โดยเปรียบเทียบ ก่อนทำการจัดอันดับซ้ำ (re-rank) กับ หลังทำ re-rank เพื่อพิจารณาคุณภาพเชิงบริบท แหล่งอ้างอิง และความพร้อมต่อการสังเคราะห์คำตอบแบบอ้างอิงได้

```
[QUERY] ขั้นตอนลงทะเบียนเรียน จุฬาลงกรณ์มหาวิทยาลัย

#1 id=69b323cd66c7ef85edf091e2dcfad839a155daf9:23 score=0.8810 url=https://www.reg.chula.ac.th/admissions/
สิดจุฬาฯ และทำบัตรประจำตัวนิสิต ที่ระบบลงทะเบียนแรกเข้า
- ติดตามกำหนดการ และอ่านรายละเอียดเพิ่มเติมเกี่ยวกับเอกสารที่ต้องใช้ ที่เว็บไซต์
**สำนักงานการทะเบียน**

**5. ลงทะเบียนเรียน**- ลงทะเบียนเรียนรายวิชาตามที่หลักสูตรกำหนดผ่านเว็บไซต์
**สำนักงานการทะเบียน** - ติดตามกำหนดการการ...

#2 id=09c20a559a141d8f86803b0debe069283c022bd4:22 score=0.8810 url=https://www.reg.chula.ac.th/admissions/
สิดจุฬาฯ และทำบัตรประจำตัวนิสิต ที่ระบบลงทะเบียนแรกเข้า
- ติดตามกำหนดการ และอ่านรายละเอียดเพิ่มเติมเกี่ยวกับเอกสารที่ต้องใช้ ที่เว็บไซต์
**สำนักงานการทะเบียน**

**5. ลงทะเบียนเรียน**- ลงทะเบียนเรียนรายวิชาตามที่หลักสูตรกำหนดผ่านเว็บไซต์
**สำนักงานการทะเบียน** - ติดตามกำหนดการการ...

#3 id=95dd4cc26b9f87fe41d9315463698c63abce5a8a:21 score=0.8810 url=https://www.reg.chula.ac.th/admissions/
สิดจุฬาฯ และทำบัตรประจำตัวนิสิต ที่ระบบลงทะเบียนแรกเข้า
- ติดตามกำหนดการ และอ่านรายละเอียดเพิ่มเติมเกี่ยวกับเอกสารที่ต้องใช้ ที่เว็บไซต์
**สำนักงานการทะเบียน**

**5. ลงทะเบียนเรียน**- ลงทะเบียนเรียนรายวิชาตามที่หลักสูตรกำหนดผ่านเว็บไซต์
**สำนักงานการทะเบียน** - ติดตามกำหนดการการ...

...
- รับรหัสผ่าน CU-NET จาก
**สำนักบริหารเทคโนโลยีสารสนเทศ**เพื่อใช้บริการต่างๆ ของมหาวิทยาลัย และเข้าระบบลงทะเบียนเรียน

**4. ลงทะเบียนแรกเข้า**- กรอกประวัติ อัปโหลดเอกสาร และรูปถ่ายนิสิต เพื่อขึ้นทะเบียนเป็นนิสิตจุฬาฯ และทำบัตรประจำตัวนิสิต...
```

รูปที่ 5.1 ผลลัพธ์ก่อน re-rank

ผลลัพธ์อันดับต้น ๆ ส่วนใหญ่ชี้ไปที่หน้าแลนดิง ../admissions/ หลายรายการซ้ำกัน เนื้อหาที่ปรากฏในสรุป (snippet) เป็นถ้อยความกว้าง ๆ เกี่ยวกับการลงทะเบียน/ติดตามกำหนดการ มากกว่าจะเป็น “ขั้นตอนปฏิบัติ” ที่อ่านแล้วนำไปใช้ได้ทันที ภาพรวมจึงมีลักษณะ (1) ความซ้ำซ้อนสูง (รายการหลายลิงก์จากหน้าเดียวกัน) (2) ความเฉพาะเจาะจงของหลักฐานต่ำ และ (3) ความพร้อมต่อการอ้างอิงเชิงข้อกำหนดยังไม่เด่นชัด เพราะผู้อ่านต้องกดต่อเพื่อค้นหาส่วน “ขั้นตอน” ที่แท้จริงในหน้าเดียวกัน


```
[QUERY] ขั้นตอนลงทะเบียนเรียน จุฬาฯ ทำอย่างไร
You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `__call__` method is

#1 id=e320441db30bba64db0390ca44825f06add115b9:80 rerank_score=0.9964 url=https://web.reg.chula.ac.th/CR99_2566_T_S.pdf
◆ายในหอสมุดแรกของภาคการศึกษา หรือภายในสองสัปดาห์แรกของภาค
ฤดูร้อน โดยค าเนินการที่เว็บไซต์ www.reg.chula.ac.th (นัสดสามารถศึกษารายละเอียดจากคู่มือการลงทะเบียนเรียน
ที่ปรากฏในเว็บไซต์ของส านักงานการทะเบียน) ตามขั้นตอนต่อไปนี้
1. เข้าสู่ระบบการลงทะเบียนเรียน โดยเลือกรายการ **เข...

#2 id=861da5daa97094d2bc666d94501cda7d5c05d7a8:62 rerank_score=0.9907 url=https://www.reg.chula.ac.th/CR99_2025_Th.pdf
ทะเบียนเรียนปกติ
1. การลงทะเบียนเรียนปกติในภาคการศึกษาแรกสำหรับนิสิตใหม่
งานของลงทะเบียนเรียนผ่านทางอินเทอร์เน็ตที่เว็บไซต์ www.reg.chula.ac.th (นัสดสามารถศึกษาขั้นตอนจา...ลงความ

#3 id=861da5daa97094d2bc666d94501cda7d5c05d7a8:78 rerank_score=0.9871 url=https://www.reg.chula.ac.th/CR99_2025_Th.pdf
ไม่ได้แสดงความงานของลงทะเบียนเรียนตามช่วงเวลาที่ยากหนด ต้องลงทะเบียนเรียนสาย (พร้อมชำระ
ร่อนทหารได้ภายในสัปดาห์แรกของภาคฤดูร้อน โดยลงทะเบียนเรียนที่ เว็บไซต์ www.reg.chula.ac.th (นัสด...

#4 id=69b323cd66c7ef85edf091e2dcfad839a155daf9:22 rerank_score=0.9701 url=https://www.reg.chula.ac.th/admissions/
การทะเบียน**

**3. รับรหัสผ่าน CU-NET**
- รับรหัสผ่าน CU-NET จาก
**สำนักบริหารเทคโนโลยีสารสนเทศ**เพื่อใช้บริการต่างๆ ของมหาวิทยาลัย และเข้าระบบลงทะเบียนเรียน

**4. ลงทะเบียนแรกเข้า**- กรอกรประวัติ อัปโหลดเอกสาร และรูปถ่ายนัสด เพื่อขึ้นทะเบียนเป็นนัสดจุฬาฯ และทำบัตรประจำตัวนัสด...
...
3. ขอบังคับจุฬาลงกรณ์มหาวิทยาลัย ว่าด้วยการศึกษาชั้นปริญญาตรี พ.ศ. 2556 และ พ.ศ. 2565
5. ขอบังคับจุฬาลงกรณ์มหาวิทยาลัย ว่าด้วยการศึกษาในหลักสูตรระดับอุดมศึกษา พ.ศ. 2566

การลง...
```

รูปที่ 5.2 ผลลัพธ์หลัง re-rank

อันดับต้นเปลี่ยนไปชี้ เอกสารการประเภทข้อบังคับ/คู่มือ ในโดเมน reg.chula.ac.th (เช่น CR99_2025_Th.pdf, CR99_2566_T_S.pdf) พร้อมคงหน้า .../admissions/ ไว้เป็นตัวเสริม ผลที่ตามมา คือ 1. ประเภทหลักฐานยกระดับจากหน้าแลนดิงสรุป ไปสู่ “ตัวบททางการ” ที่มีโครงสร้างเป็นข้อ ๆ อ่านแล้ว รู้ขั้นตอน 2. ลดรายการซ้ำ เพิ่มความหลากหลายของชนิดเอกสาร (PDF ทางการ + HTML สรุป) และ 3. ความพร้อมต่อการอ้างอิงสูงขึ้น เพราะ URL ปลายทางเป็นเอกสารที่ตรวจสอบย้อนกลับได้ตรงประเด็นกับ คำถาม

ผลเชิงคุณภาพที่สังเกตได้

1. ความตรงประเด็นของแหล่งอ้างอิง ดีขึ้นอย่างชัดเจน: จาก “หน้าแฉรวม” สู่ “เอกสารขั้นตอน/ข้อบังคับ” ทำให้บริบทที่ป้อนเข้าส่วนสังเคราะห์คำตอบมีความเป็น “ขั้นตอนใช้งานได้จริง” มากขึ้น
2. ความพร้อมต่อการอ้างอิง สูงขึ้น: ลิงก์ปลายทางเป็น PDF ทางการ ช่วยให้การอ้างอิงในคำตอบมีน้ำหนัก และเปิดตรวจสอบได้ทันที
3. ความซ้ำซ้อนลดลง: รายการหน้าเดียวกันที่ขึ้นหลายลิงก์ถูกดันลง ส่งผลให้ Top-K มีความหลากหลายของหลักฐานมากขึ้น
4. ประสบการณ์นำทางของผู้ใช้ ดีขึ้นในเชิงอ้อม: เมื่ออันดับต้นเป็นเอกสารที่ “บอกขั้นตอน” โดยตรง ผู้ใช้ต้องคลิกน้อยครั้งลงเพื่อไปถึงสาระที่ต้องการ

ข้อสรุป

จากหลักฐานเชิงภาพที่เปรียบเทียบกัน การทำ re-rank ส่งผลเชิงคุณภาพที่เป็นรูปธรรม: แหล่งอ้างอิง “เฉพาะเจาะจงและตรวจสอบได้” ขึ้น ความซ้ำซ้อนลดลง และบริบทพร้อมต่อการสังเคราะห์คำตอบแบบมีอ้างอิงมากกว่าเดิม อย่างไรก็ตาม อย่างไรก็ดี เพื่อให้ผลสะท้อนการใช้งานจริงทั้งระบบ ควรขยายการวิเคราะห์ไปยังคำถาม หัวข้อใกล้เคียง (เช่น กำหนดเวลา ช่องทาง ระบบลงทะเบียน และเอกสารประกอบ) พร้อมบังคับใช้นโยบาย เวอร์ชันล่าสุดและ chunking ตามหัวเรื่องในขั้นตอนถัดไปของงาน

5.4 ผลการทดลองตาม Use Case

5.4.1 การถาม-ตอบภาษาธรรมชาติ (TH/EN)

```
[QUERY] ขั้นตอนลงทะเบียนเรียน จุฬาฯ ทำอย่างไร
You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `__call__` method is faster than us

[LLM ANSWER]
สรุปขั้นตอนการลงทะเบียนเรียน จุฬาฯ (แบบกระชับ)

1. เตรียมบัญชีเข้าใช้
- รับรหัสผ่าน CU-NET จากสำนักบริหารเทคโนโลยีสารสนเทศ เพื่อเข้าสู่ระบบลงทะเบียนเรียน (CU-NET) ก่อนดำเนินการอื่นๆ (ดูรายละเอียดการรับรหัส)
อ้างอิง: https://www.reg.chula.ac.th/admissions/

2. นิสิตใหม่ (ลงทะเบียนแรกเข้า)
- ดำเนินการลงทะเบียนแรกเข้าออนไลน์ (กรอกประวัติ อัปโหลดเอกสารและรูปถ่าย) ผ่านระบบลงทะเบียนแรกเข้า (adm.reg.chula.ac.th) ตามกำหนดของนิสิตใหม่
อ้างอิง: https://www.reg.chula.ac.th/admissions/ , https://adm.reg.chula.ac.th

3. การลงทะเบียนเรียนปกติ (ทั่วไป)
- เข้าเว็บระบบลงทะเบียนที่ www.reg.chula.ac.th
- เลือกเมนู "เข้าสู่ระบบ / ลงทะเบียนเรียน / ข้อมูลส่วนบุคคล" แล้วเลือก "ลงทะเบียนเรียน"
- เลือกประเภทการลงทะเบียน เช่น "ลงปกติ / เปลี่ยนตอนเรียน / ละครายวิชา"
- บันทึกรายวิชาและตอนเรียน แล้วกดบันทึก
- การลดรายวิชาต้องกดบันทึกและยืนยัน
- ตรวจสอบผลการลงทะเบียนได้ที่เมนู "สอบถามข้อมูลส่วนบุคคล"
อ้างอิง: https://web.reg.chula.ac.th/CR99\_2566\_T\_S.pdf , https://www.reg.chula.ac.th/CR99\_2025\_Th.pdf

4. กรณีลงทะเบียนสาย
- หากไม่ได้แสดงความจำนงตามกำหนด ต้องลงทะเบียนสายผ่านอินเทอร์เน็ตภายในสองสัปดาห์แรกของภาคการศึกษา (ภาคฤดูร้อนภายในสัปดาห์แรก) และมีค่าปรับตามประกาศ
- วิธีเข้าสู่ระบบ: ป้อนเลขประจำตัวนิสิต, รหัสผ่าน และ Captcha แล้วดำเนินการลงทะเบียนทางเว็บ
อ้างอิง: https://web.reg.chula.ac.th/CR99\_2566\_T\_S.pdf , https://www.reg.chula.ac.th/CR99\_2025\_Th.pdf
...
- ชำระค่าเล่าเรียนผ่าน CU NEX Application ตามวันเวลาที่กำหนดในปฏิทินการศึกษา
อ้างอิง: https://web.reg.chula.ac.th/document/student\_doc1.pdf

(เอกสารอธิบายขั้นตอนฉบับเต็มและคู่มือการลงทะเบียนอยู่ที่เว็บไซต์สำนักงานการทะเบียน www.reg.chula.ac.th)
```

รูปที่ 5.3 ผลลัพธ์คำตอบสรุปขั้นตอนลงทะเบียนเรียน

```
[QUERY] How to register for courses at Chulalongkorn University
You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `__call__` method is faster than using a method to encode the text followed by a call to the 'pad' method

[LLM ANSWER]
Short answer - main steps

1) New / first-semester entry (initial enrollment)
- Complete online first-entry registration (fill student survey JTH19 and record JTH20, upload documents/photo) at https://adm.reg.chula.ac.th (see CR99_2025_Th.pdf).
(Reference: https://www.reg.chula.ac.th/CR99\_2025\_Th.pdf)

2) Get account/password (CU-NET)
- Obtain CU-NET password from the Office of IT to log into university systems (info at www.it.chula.ac.th / contact in CR99_2025_Th.pdf).
(Reference: https://www.reg.chula.ac.th/CR99\_2025\_Th.pdf)

3) Regular course registration (online)
- Go to the registration website www.reg.chula.ac.th, sign in via the menu: "เข้าสู่ระบบ / ลงทะเบียนเรียน / ข้อมูลส่วนบุคคล" (enter student ID, password, captcha), then choose "ลงทะเบียนเรียน" and select registration
(References: https://web.reg.chula.ac.th/CR99\_2566\_T\_S.pdf ; https://www.reg.chula.ac.th/CR99\_2025\_Th.pdf)

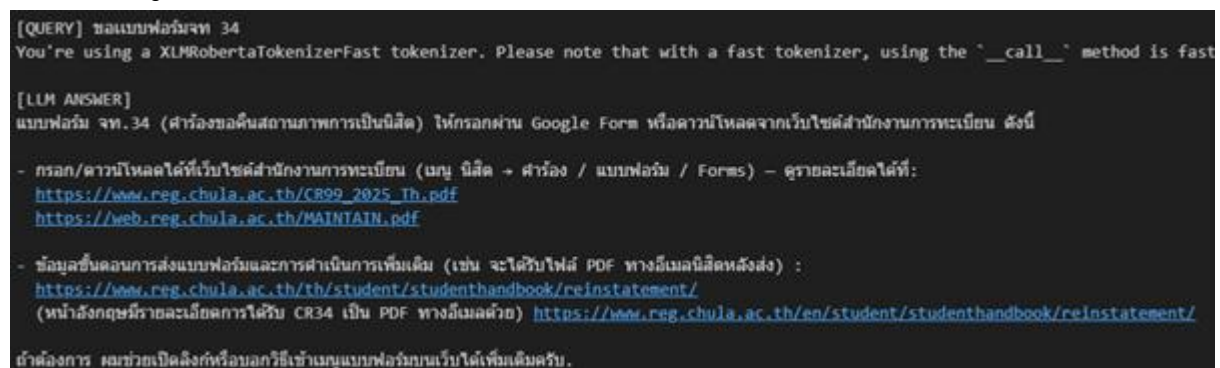
4) Late registration
- If you miss the declared registration period you must do late registration online (within the permitted grace period; summer has a shorter window) and pay any applicable fines, via www.reg.chula.ac.th.
(References: https://www.reg.chula.ac.th/CR99\_2025\_Th.pdf ; https://web.reg.chula.ac.th/CR99\_2566\_T\_S.pdf)

5) Payment and advisor approval
- Pay tuition via CU NEX Application as scheduled. Advisors provide registration advice/approval and (for undergraduates) give the registration code when approving.
(References: https://web.reg.chula.ac.th/document/student\_doc1.pdf ; https://www.reg.chula.ac.th/CR99\_2566\_T\_S.pdf)
```

รูปที่ 5.4 ผลลัพธ์คำตอบภาษาอังกฤษ

จากรูปประกอบภาษาไทยและภาษาอังกฤษ ระบบตรวจจับภาษาได้ถูกต้องและสรุปคำตอบเชิงลำดับชั้นอย่างครบถ้วน โดยยึดหลักว่าต้องตอบคำถามจากภาษาธรรมชาติที่ผู้ใช้งานเท่านั้น เช่นไฟล์ข้อบังคับและคู่มือภายใต้ reg.chula.ac.th ควบคู่กับหน้าสำหรับการนำทางเพิ่มเติม ผลลัพธ์สองภาษามีโครงสร้างและสาระเทียบเท่ากัน สะท้อนความเสถียรของกระบวนการค้นคืน คัดเลือก และสังเคราะห์คำตอบตามเงื่อนไขความสำเร็จของ UC-01 ตั้งแต่ผู้ส่งคำถาม ระบบค้นคืนรายการที่เกี่ยวข้องและจัดลำดับซ้ำ ก่อนสังเคราะห์คำตอบจากบริบทและแสดงที่อยู่ต้นฉบับที่ตรวจสอบได้ ชุดรูปนี้ยืนยันว่าเคสสำเร็จตามนิยามของยูสเคส

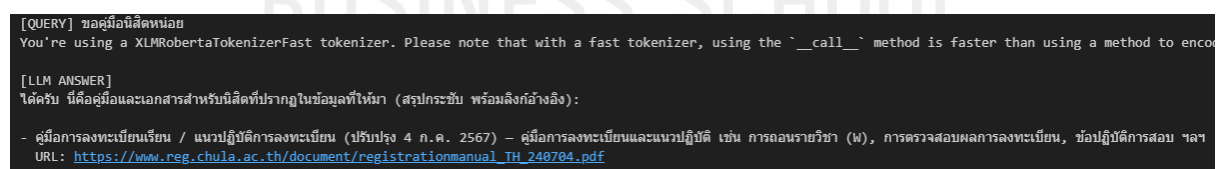
5.4.2 เปิดดูเอกสารต้นฉบับจาก URL



รูปที่ 5.5 ตัวอย่างการทดสอบขอเอกสารต้นทาง

จากรูปที่ 5.5 ระบบแสดงคำตอบพร้อมรายการที่อยู่ต้นฉบับซึ่งชี้ไปยังไฟล์ข้อบังคับหรือคู่มือในโดเมนทางการของมหาวิทยาลัยโดยตรง รวมถึงหน้าอ้างอิงที่เกี่ยวข้อง ผู้ใช้สามารถเปิดเอกสารต้นฉบับเพื่อยืนยันข้อมูลได้ทันทีโดยไม่ต้องผ่านหน้าอื่นหลายชั้น คุณภาพของลิงก์อยู่ในมาตรฐานการอ้างอิงที่น่าเชื่อถือ ตรงตามเงื่อนไขความสำเร็จของ UC-02 ได้แก่ ผู้ใช้เลือกชิ้นผลลัพธ์ ระบบแสดงหรือส่งออกที่อยู่ต้นฉบับ และผู้ใช้เปิดเอกสารต้นฉบับได้จริง จึงนับว่าบรรลุวัตถุประสงค์ของยูสเคสนี้

5.4.3 ค้นหาเอกสารงานทะเบียนด้วยคีย์เวิร์ด



รูปที่ 5.6 ตัวอย่างการค้นหาเอกสารงานทะเบียนด้วยคีย์เวิร์ด

จากรูปที่ 5.6 ระบบตีความคำค้นแบบสั้นในภาษาผู้ใช้ไปยังเอกสารทางการที่เกี่ยวข้องโดยตรง และคืนผลเป็นลิงก์ไฟล์ PDF ต้นฉบับในโดเมนมหาวิทยาลัย ทำให้ผู้ใช้เข้าถึงต้นฉบับเพื่อยืนยันข้อมูลได้ทันที คำตอบสรุปสาระสำคัญของเอกสารก่อนแนบลิงก์ ช่วยให้เข้าใจบริบทและตัดสินใจเปิดไฟล์ได้รวดเร็ว ลำดับเหตุการณ์สอดคล้องกับเงื่อนไขความสำเร็จของ UC-03 อย่างครบถ้วน ตั้งแต่ผู้ใช้ป้อนคีย์เวิร์ด ระบบค้นคืนรายการที่ตรงประเด็น และแสดงขึ้นข้อมูลพร้อมที่อยู่ต้นฉบับที่เปิดได้จริง จึงยืนยันผลสำเร็จของเคส

5.4.4 ลบข้อมูลจากดัชนี

```
url_to_delete = "https://www.it.chula.ac.th/wp-content/uploads/2020/01/Path_Academic_2551-2555.pdf"

# ลองดึง id มาก่อนเพื่อนับ
hits = col.get(where={"url": url_to_delete}, include=[]) # ขอเฉพาะ id
ids = hits.get("ids", [])
if not ids:
    print("no url match")
else:
    col.delete(where={"url": url_to_delete})
    print(f"deleted {len(ids)} vectors with url = {url_to_delete}")

✓ 0.1s
deleted 5 vectors with url = https://www.it.chula.ac.th/wp-content/uploads/2020/01/Path_Academic_2551-2555.pdf
```

รูปที่ 5.7 รูปตัวอย่างการลบข้อมูล

จากเอาต์พุตโค้ด ระบบรายงานจำนวนรายการที่ลบได้ตามที่อยู่ต้นฉบับเป้าหมาย และยืนยันผลการลบด้วยค่าจำนวนที่ลบและจำนวนคงเหลือ หลังเสร็จสิ้นได้ทำการค้นซ้ำด้วยคีย์เวิร์ดเดิมแล้วไม่ปรากฏขึ้นข้อมูลจากที่อยู่นั้นอีก ขั้นตอนตั้งแต่การระบุที่อยู่ต้นฉบับ การลบ การยืนยันผลลัพธ์ ไปจนถึงการตรวจสอบด้วยการค้นซ้ำ สอดคล้องกับเงื่อนไขความสำเร็จของ UC-04 จึงถือว่าสำเร็จตามวัตถุประสงค์ในการบำรุงรักษาลังเวกเตอร์

5.4.5 อัปเดตดัชนี

```
from .autonotebook import tqdm as notebook_tqdm
[INFO] input=output/stage/embeddable.jsonl
[INFO] chroma=chroma | collection=rag_chunks | space=cosine
[INFO] model=BAAI/bge-m3 | batch=256
c:\Users\gpaki\Desktop\RAG_openai\.venv312\Lib\site-packages\transformers\
attn_output = torch.nn.functional.scaled_dot_product_attention(
[ADD] +256 (total=256)
[ADD] +256 (total=512)
[ADD] +256 (total=768)
[ADD] +256 (total=1024)
[ADD] +256 (total=1280)
```

รูปที่ 5.8 การอัปเดตดัชนี

จากเอาต์พุตโค้ด ระบบดัชนีข้อความแล้วได้ชุดขึ้นรวม 1,280 ขึ้น พร้อมไฟล์สำหรับฝังเวกเตอร์ที่ผ่านการกรองและเตรียมใช้งาน จากนั้นอัปเดตคลังเวกเตอร์ตามขั้นตอนที่กำหนด และตรวจสอบด้วยการค้นคำถามที่เกี่ยวข้อง พบว่าเอกสารฉบับล่าสุดถูกค้นคืนและจัดแสดงในอันดับต้นตามที่ต้องการ ลำดับการทำงาน chunking ฝังเวกเตอร์ บันทึกคลัง และทดสอบหลังอัปเดต สอดคล้องกับเงื่อนไขความสำเร็จของ UC-05 เคสนี้จึงถือว่าสำเร็จ

5.5 อภิปรายผล

ผลการทดลองจากรูปและเอาต์พุตโค้ดยืนยันว่ากระบวนการของระบบตั้งแต่ดึงข้อมูล แบ่งชิ้น คั่นคืน คัดเลือก และสังเคราะห์คำตอบ ทำงานสอดคล้องตามที่ออกแบบไว้ในสภาพการใช้งานจริงของโดเมนทางการของมหาวิทยาลัย จุดแข็งที่เด่นชัดคือความสามารถในการค้นแหล่งอ้างอิงที่ตรวจสอบได้ทันทีในรูปแบบของไฟล์ ข้อบังคับและคู่มือ รวมถึงการคงรูปแบบคำตอบสองภาษาที่มีโครงสร้างเทียบเท่ากัน ซึ่งช่วยให้ผู้ใช้เข้าถึงสารสนเทศเชิงขั้นตอนได้น่าเชื่อถือและรวดเร็ว ทั้งนี้ การจัดลำดับซ้ำมีบทบาทสำคัญต่อการยกระดับคุณภาพเชิงอ้างอิง โดยผลักดันเอกสารทางการขึ้นสู่อันดับต้นและลดความซ้ำซ้อนของรายการจากหน้าแลนดังกล่าวไป

ในมิติข้อมูล ชุดที่ดึงได้มีสัดส่วนภาษาไทยเป็นส่วนใหญ่และประกอบด้วยเอกสารจำนวนมากจากโดเมนงานทะเบียน ส่งผลให้การแบ่งชิ้นแบบเน้นรองรับ OCR และหน้า HTML ยาวเป็นเรื่องจำเป็น ตัวเลขขึ้นรวม 108,893 ชิ้นและค่าเฉลี่ยประมาณหนึ่งร้อยโทเคนต่อชิ้น ช่วยสมดุลความละเอียดในการค้นคืนกับภาระของตัวจัดลำดับซ้ำ อย่างไรก็ตาม เอกสารทางการบางประเภทมีหลายฉบับในคนละปี การค้นผลพร้อมกันหลายเวอร์ชันเป็นผลดีต่อความครอบคลุม แต่ควรกำกับให้เห็นฉบับล่าสุดชัดเจนเพื่อเลี่ยงความสับสนของผู้ใช้ นอกจากนี้ เอกสาร PDF ที่ยาวและมีตารางจำเป็นต้องพิจารณาการแบ่งชิ้นโดยยึดหัวข้อหรือโครงสร้างสารบัญเพื่อเพิ่มความแม่นยำในการดึงบริบท

ในทางเทคนิค การเตรียมเมตาดาตาสำหรับการฝังเวกเตอร์ด้วยโมเดล BGE-M3 ต้องไม่มีคีย์ว่าง จึงกำหนดให้ใช้เมตาดาตาแบบมินิมอลเฉพาะคีย์ที่อยู่ต้นฉบับ ซึ่งเพียงพอสำหรับการอ้างอิง การลบ และการอัปเดตตามที่อยู่ต้นฉบับในขั้นตอนบำรุงรักษา ในด้านประสบการณ์ผู้ใช้ คำตอบที่สรุปใจความก่อนลิงก์ต้นฉบับลดภาระการเปิดอ่านทั้งฉบับล่วงหน้า และใช้ประโยชน์จากลิงก์ไฟล์ตรงเพื่อยืนยันข้อเท็จจริงได้รวดเร็ว

โดยสรุป การทดสอบทั้งห้ากรณีบ่งชี้ว่าระบบบรรลุผลตามนิยามของยูสเคสที่กำหนดไว้ ความท้าทายหลักที่ควรพัฒนาในรอบถัดไปได้แก่การกำกับฉบับล่าสุดให้เด่นชัด การแบ่งชิ้นแบบยึดหัวเรื่องเพื่อเอกสารยาวหรือมีตาราง และการติดตามคุณภาพเชิงเวลา เพื่อให้คุณภาพการค้นคืนและความน่าเชื่อถือของคำตอบคงที่เมื่อข้อมูลมีการปรับปรุงอย่างต่อเนื่อง

FLAGSHIP FOR LIFE

บทที่ 6

สรุปและข้อเสนอแนะ

6.1 สรุปผลตามวัตถุประสงค์

โครงการได้บรรลุวัตถุประสงค์ด้านสถาปัตยกรรมระบบถาม-ตอบบนเอกสาร (RAG) ตามที่กำหนด โดยวางสายพานตั้งแต่การดึงข้อมูล การแบ่งชิ้นข้อความ การฝังเวกเตอร์ การค้นคืนสองชั้นด้วยตัวค้นและการจัดลำดับซ้ำ ไปจนถึงการสังเคราะห์คำตอบที่ยึดหลักฐานและแสดงแหล่งอ้างอิงต้นฉบับอย่างโปร่งใส ระบบจึงสามารถตอบคำถามจากผู้ใช้ด้วยถ้อยคำกระชับ ตรวจสอบย้อนกลับได้ และไม่คาดเดาเกินบริบท พร้อมทั้งทำงานสองภาษาไทย-อังกฤษได้อย่างสม่ำเสมอ โดยให้โครงคำตอบและแหล่งอ้างอิงเทียบเท่ากัน

ในด้านข้อมูล ระบบรวบรวมและเตรียมชุดข้อมูลจากโดเมนทางการที่กำหนด ครอบคลุมทั้งหน้าเว็บ HTML เอกสาร PDF และภาพที่ผ่าน OCR ชุดที่รายงานมีเอกสาร 2,473 รายการ รวมประมาณ 35.50 ล้านอักขระ คิดเป็นราว 8.88 ล้านโทเคน โดยสัดส่วนภาษาไทยในระดับเอกสารอยู่ที่ประมาณ 72.66 เปอร์เซ็นต์ เมื่อแบ่งชิ้นข้อความเพื่อใช้งานค้นคืน ได้ขึ้นรวม 108,893 ชิ้นส่วนส่วนใหญ่เกิดจากเนื้อหาที่ต้องรองรับ OCR และหน้า HTML ยาว จึงสะท้อนการเตรียมข้อมูลที่ใกล้เคียงการใช้งานจริงในงานทะเบียน

ผลด้านคุณภาพการอ้างอิงและความน่าเชื่อถือบรรลุเป้าหมายที่ตั้งไว้ โดยการจัดลำดับซ้ำช่วยดันเอกสารทางการประเภทข้อบังคับและคู่มือในโดเมนมหาวิทยาลัยขึ้นสู่อันดับต้น ลดความซ้ำซ้อนจากหน้าแลนดิง และทำให้คำตอบพร้อมสำหรับการอ้างอิงต้นฉบับทันที ประสพการณ์ผู้ใช้ดีขึ้นอย่างเป็นรูปธรรม ผู้ใช้ถามครั้งเดียวแล้วเห็นคำตอบสั้นที่สรุปใจความก่อนลิงก์ไฟล์ตรง ช่วยลดเวลาและจำนวนการคลิก ทั้งยังรองรับผู้ใช้สองภาษาได้ในมาตรฐานเดียวกัน

ความพร้อมด้านการบำรุงรักษาข้อมูลก็เป็นไปตามวัตถุประสงค์ ระบบสามารถลบและอัปเดตดัชนีตามที่อยู่ต้นฉบับได้จริง พร้อมหลักฐานจากแฮชพุดโค้ดและการค้นซ้ำยืนยันผล หลังอัปเดตพบว่าเอกสารฉบับล่าสุดถูกค้นคืนในอันดับที่เหมาะสม สอดคล้องกับการดูแลความทันสมัยของคลังเวกเตอร์ นอกจากนี้ได้กำหนดนโยบายเมทาเดตาแบบมินิมอลให้เก็บเฉพาะคีย์ url เนื่องจากฟิลด์อื่นส่วนใหญ่ไม่มีข้อมูล และโมเดลฝังเวกเตอร์ BGE-M3 ไม่ยอมรับเมทาเดตาว่าง แนวทางนี้ยังคงความสามารถในการอ้างอิง ลบ และอัปเดตตามที่อยู่ต้นฉบับครบถ้วน

ท้ายที่สุด การประเมินตามยูสเคสหลักที่กำหนดไว้ในบทที่ 5 ได้แก่ ถาม-ตอบภาษาธรรมชาติ เปิดดูต้นฉบับจาก URL ค้นหาเอกสารด้วยคีย์เวิร์ด รวมถึงการลบและอัปเดตดัชนี แสดงให้เห็นว่าแต่ละกรณีทำงานได้สำเร็จตามเงื่อนไขและวัตถุประสงค์ของระบบ โดยมีหลักฐานจากรูปและแฮชพุดโค้ดยืนยันตลอดสายพานการทำงาน ตั้งแต่การค้นคืนที่ตรงเจตนาผู้ใช้ไปจนถึงการเปิดตรวจสอบเอกสารทางการได้จริงในขั้นสุดท้าย

6.2 ปัญหาและแนวทางแก้ไข

ตลอดการพัฒนา ระบบ RAG ได้พบปัญหาเชิงระบบหลายประเด็นซึ่งส่งผลกระทบต่อความสะอาดของข้อมูล ประสิทธิภาพการดึง-สกัดเอกสาร และความน่าเชื่อถือของผลลัพธ์ ในส่วนนี้จึงสรุปเฉพาะ “ปัญหาใหญ่” และ “แนวทางแก้ไขที่ดำเนินการแล้ว” โดยอ้างอิงจากสคริปต์ดึงข้อมูลดิบ ควบคู่กับมาตรการเสริมในเวิร์กโฟลว์จริง เริ่มจากต้นน้ำ พบความซ้ำซ้อนของหน้าเว็บจาก URL หลายรูปแบบ (เช่นพารามิเตอร์ติดตาม, fragment, และ /index.html) รวมถึงการกระเด็นออกนอกโดเมนที่ตั้งใจ ส่งผลให้ดัชนีปนเปื้อนและสิ้นเปลืองทรัพยากร แนวทางแก้ไขคือเพิ่มกระบวนการ canonicalization บังคับใช้ https ตัดพารามิเตอร์ที่ไม่จำเป็นและ fragment รวมถึงอ่านค่าจาก <link rel="canonical"> หากมี พร้อมทั้งบังคับขอบเขตด้วย allowlist ของ โดเมนก่อนเข้าคิวทุกจุด ผลลัพธ์คือจำนวนเอกสารซ้ำลดลงอย่างมีนัยสำคัญและขอบเขตข้อมูลถูกต้องตาม วัตถุประสงค์มากขึ้น

ถัดมาในมิติความเสถียรของการครอว์ล เดิมระบบขยายคิวโดยไม่คุมความลึกและไม่มีการจำกัดอัตราเรียกต่อ โฮสต์ ทำให้เกิดการรวนรูปและเสียงถูกระบบปลายทางจำกัดการเข้าถึง ได้ปรับปรุงโดยกำหนด MAX_DEPTH/MAX_PAGES ควบคู่กับชุด seen สำหรับกันซ้ำ และเพิ่มตัวจำกัดความเร็ว (Rate Limiter) ราย โฮสต์ก่อนทุกคำขอ ส่งผลให้การครอว์ลปิดงานได้ภายในเวลาที่คาดหวัง ลดความผิดพลาดระดับ p95 และ ลดโอกาสชน rate limit ของเว็บไซต์ต้นทาง

ด้านคุณภาพของ HTML ที่ดึงได้ เดิมเพจที่พึ่งพาการเรนเดอร์ฝั่งไคลเอนต์ (JS-heavy) มักให้ DOM ไม่ครบ หรือบางครั้งตอบกลับด้วยชนิดเนื้อหาที่ไม่ใช่ HTML ระบบจึงเปลี่ยนมาเปิดหน้าเว็บผ่าน Playwright (Chromium) กำหนด timeout และรอให้ตัวเลือก (selector) ที่สำคัญพร้อมก่อนอ่าน DOM ควบคู่กับการ ตรวจสอบ content-type เพื่อข้ามเนื้อหาที่ไม่เกี่ยวข้อง นอกจากนี้ยังเพิ่มตัวแยกลิงก์ขึ้นสูงเพื่อค้นหา PDF/รูปภาพที่ถูกฝังในหลายรูปแบบ (เช่น srcset, <picture>, background-image, และ data-*) ทำให้การเก็บ ไฟล์สาระสำคัญครบถ้วนขึ้น ลดอัตราการตกหล่นของเอกสารต้นทางที่จำเป็นต่อการตอบคำถาม

ในประเด็นข้อมูลซ้ำ ระบบเคยดาวน์โหลดไฟล์ PDF/รูปเดิมซ้ำหลายครั้งและตั้งชื่อไม่สอดคล้องกัน ขณะนี้ แก้ไขโดยใช้ชุดกันซ้ำสำหรับ asset และตั้งชื่อไฟล์ตามแฮชของ URL เพื่อให้เสถียรข้ามรอบรัน พร้อมลงบันทึก ใน manifest ทุกครั้ง สำหรับระดับ “เนื้อหา” (เกินกว่าไฟล์) ได้กำหนดยุทธศาสตร์สองชั้นในขั้นตอนถัดไป ของ pipeline คือทำ exact-hash หลังการปรับสภาพข้อความ (normalize/NFKC)

ยังมีปัญหาที่เกิดจากไฟล์ต้นทางประเภท PDF ซึ่งเสียโครงสร้าง (เช่น cross-reference/stream ผิด) ทำให้ ขึ้นสกัดข้อความล้มเหลว ระบบจึงเพิ่ม “ด่านซ่อม PDF” ก่อนการสกัด โดยใช้เครื่องมือ qpdf เพื่อซ่อมหรือทำให้ไฟล์เป็นเชิงเส้น (linearize) แล้วจึงส่งเข้ากระบวนการถัดไป ผลคืออัตราความล้มเหลวของการแปลง PDF-to-text/OCR ลดลงและขึ้นสกัดมีความต่อเนื่อง

สุดท้าย ในการทดสอบ เดิมเคยทดลองสร้างชุดทดสอบอัตโนมัติด้วย RAGAS เวอร์ชัน 0.3.6 บนคอร์ปัสขนาดใหญ่ แต่พบทั้งความล่าช้า ต้นทุนสูง และข้อจำกัดเชิงโลบารรี (เช่นการเติม default transforms อัตโนมัติ การสร้าง persona ที่ผิดพลาด และข้อกำหนดของ multi-hop ที่ต้องพึ่งโครงสร้างความสัมพันธ์) จึงตัดสินใจยุติแนวทางดังกล่าว และหันมาใช้ “smoke test” ที่กำหนดกรณีคำถามตามการใช้งานจริง (ไทย/อังกฤษ/สองภาษา) สามารถควบคุมเวลาและงบประมาณได้ดีกว่า

โดยสรุป ชุดการแก้ไขข้างต้นช่วยให้ข้อมูลต้นน้ำสะอาดและอยู่ในขอบเขตที่ตั้งใจ การครอว์ลมีเสถียรภาพและติดตามได้ตามแผน ขั้นถัดมามีความพร้อมรับไฟล์ที่ซับซ้อน (ทั้ง HTML เชิงไดนามิกและ PDF ที่ซ่อมแล้ว) ขณะที่การจัดการ Unicode และการบันทึกแฟ้มกลางอย่างมีมาตรฐานทำให้การตรวจทานและติดตามปัญหาทำได้รวดเร็วขึ้น ภาพรวมส่งผลให้ความแม่นยำและความน่าเชื่อถือของคำตอบจากระบบ RAG ดีขึ้นอย่างเป็นรูปธรรม พร้อมพื้นฐานที่เอื้อต่อการปรับปรุงขั้น retrieval/re-ranking ในระยะต่อไป

6.3 ข้อเสนอแนะ

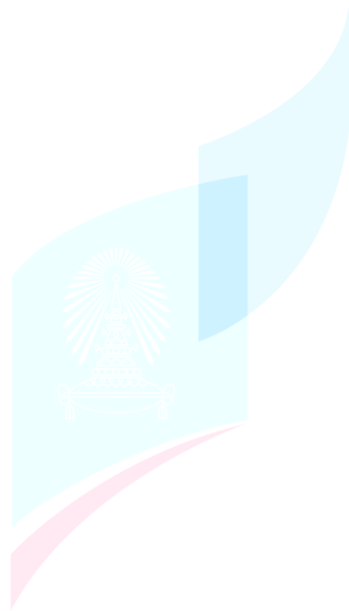
เพื่อคงคุณภาพคำตอบให้ “ทันสมัย น่าเชื่อถือ และตรวจสอบได้” ควรกำกับฉบับล่าสุดของเอกสารอย่างเป็นระบบ โดยกำหนดกติกาเลือกเวอร์ชันตามวันที่เผยแพร่หรือปีการศึกษาให้เด่นชัดในผลค้นคืนและคำตอบ พร้อมระบุสถานะฉบับล่าสุดไว้ที่แหล่งอ้างอิง และลดน้ำหนักฉบับเก่าที่อยู่หน้าเดียวกันโดยไม่ตัดความสามารถในการเปิดตรวจย้อนหลัง แนวทางนี้ช่วยหลีกเลี่ยงความสับสนเมื่อมีหลายเวอร์ชันปรากฏพร้อมกัน และทำให้การใช้งานในรอบปีการศึกษาปัจจุบันมีความถูกต้องสอดคล้องยิ่งขึ้น

ในขั้นการค้นคืน แนะนำให้ปรับเป็นแบบผสมผสานระหว่างการค้นเชิงคำและการค้นเชิงความหมาย แล้วใช้ตัวจัดลำดับซ้ำแบบ cross-encoder เพื่อคัดความเกี่ยวข้องในขั้นท้าย โดยวางตัวกรองเมทาเดตาที่สำคัญ เช่น โดเมน ภาษา ช่วงเวลา และชนิดเอกสาร ควบคู่การลดผลซ้ำตามที่อยู่ต้นฉบับ และรองรับการเขียนคำถามสองภาษาอย่างสม่ำเสมอ แนวปฏิบัตินี้จะช่วยผลักเอกสารทางการขึ้นสู่อันดับต้น ลดสัญญาณรบกวน และคงคุณภาพเชิงอ้างอิงเมื่อสภาวะข้อมูลเปลี่ยนแปลง

ด้านข้อมูล แนะนำเพิ่มเมทาเดตาแบบค่อยเป็นค่อยไป จากนโยบายปัจจุบันที่เก็บเพียงที่อยู่ต้นฉบับ ให้เพิ่มฟิลด์ที่จำเป็นต่อการคัดเลือกและตรวจสอบย้อนหลัง เช่น วันที่เผยแพร่ ชนิดเอกสาร ภาษา ปีการศึกษา เวอร์ชัน เวลาที่ดึงข้อมูล และที่อยู่ต้นฉบับแบบคาโนนิคัล พร้อมกำหนดกฎตรวจสอบไม่ให้ค่าว่างในฟิลด์ที่ใช้ตัดสินใจ การยกระดับดังกล่าวจะช่วยทั้งการคัดฉบับล่าสุด การกรองผล การลบหรืออัปเดตตามที่อยู่ต้นฉบับ และการตรวจทานย้อนรอยในภายหลัง

ท้ายที่สุด ควรสร้างชุดทดสอบโดเมนจริงแบบสองภาษาที่สะท้อนงานของนิสิตและพนักงานทะเบียน ครอบคลุมกรณีมีคำตอบและไม่มีคำตอบ ระบุคำตอบอ้างอิง ที่อยู่ต้นฉบับ ส่วนหัวข้อในเอกสาร และกรณีทำหายอย่างเอกสารหลายเวอร์ชัน ชุดทดสอบนี้จะถูกใช้ตั้งเกณฑ์คุณภาพก่อนปล่อยใช้งาน ตรวจการถดถอยเมื่อ

มีการปรับพารามิเตอร์หรืออัปเดตดัชนี ปรับจนค่าที่เหมาะสมระหว่างคุณภาพและเวลา ติดตามคุณภาพแยกตามภาษา โดเมน และชนิดเอกสาร รวมถึงทำรายงานแนวโน้มคุณภาพรายสัปดาห์หรือรายเดือน เพื่อให้การพัฒนาในรอบถัดไปมีหลักฐานเชิงประจักษ์และทำซ้ำผลได้อย่างโปร่งใส



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

บรรณานุกรม

- สำนักงานการทะเบียน จุฬาลงกรณ์มหาวิทยาลัย. เว็บไซต์สำนักงานการทะเบียน [ออนไลน์]. ไม่ปรากฏปี.
แหล่งที่มา: <https://www.reg.chula.ac.th/> [สืบค้นเมื่อ: 18 กุมภาพันธ์ 2568].
- สำนักบริหารเทคโนโลยีสารสนเทศ จุฬาลงกรณ์มหาวิทยาลัย. เว็บไซต์ IT Chula [ออนไลน์]. ไม่ปรากฏปี.
แหล่งที่มา: <https://it.chula.ac.th/> [สืบค้นเมื่อ: 20 กุมภาพันธ์ 2568].
- สำนักบริหารกิจการนิสิต จุฬาลงกรณ์มหาวิทยาลัย. เว็บไซต์สำนักบริหารกิจการนิสิต [ออนไลน์]. ไม่ปรากฏปี. แหล่งที่มา: <https://sa.chula.ac.th/> [สืบค้นเมื่อ: 22 กุมภาพันธ์ 2568].
- Chulalongkorn University. Main Site (EN) [ออนไลน์]. n.d. แหล่งที่มา: <https://www.chula.ac.th/en/> [สืบค้นเมื่อ: 24 กุมภาพันธ์ 2568].
- Microsoft. Playwright for Python — Documentation [ออนไลน์]. n.d. แหล่งที่มา: <https://playwright.dev/python/> [สืบค้นเมื่อ: 3 มีนาคม 2568].
- aio-lib. aiohttp — Documentation [ออนไลน์]. n.d. แหล่งที่มา: <https://docs.aiohttp.org/en/stable/> [สืบค้นเมื่อ: 5 มีนาคม 2568].
- Richardson, L. Beautiful Soup 4 — Documentation [ออนไลน์]. n.d. แหล่งที่มา: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> [สืบค้นเมื่อ: 6 มีนาคม 2568].
- Scrapy Project. w3lib — Documentation [ออนไลน์]. n.d. แหล่งที่มา: <https://w3lib.readthedocs.io/> [สืบค้นเมื่อ: 8 มีนาคม 2568].
- Scrapy Project. w3lib (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา: <https://github.com/scrapy/w3lib> [สืบค้นเมื่อ: 9 มีนาคม 2568].
- Barbaresi, A. Trafilatura — Documentation [ออนไลน์]. n.d. แหล่งที่มา: <https://trafilatura.readthedocs.io/en/latest/> [สืบค้นเมื่อ: 12 มีนาคม 2568].
- Barbaresi, A. Trafilatura (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา: <https://github.com/adbar/trafilatura> [สืบค้นเมื่อ: 14 มีนาคม 2568].
- PyMuPDF Developers. PyMuPDF (fitz) — Documentation [ออนไลน์]. n.d. แหล่งที่มา: <https://pymupdf.readthedocs.io/> [สืบค้นเมื่อ: 2 เมษายน 2568].
- PyMuPDF Team. PyMuPDF4LLM (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา: <https://github.com/pymupdf/pymupdf4llm> [สืบค้นเมื่อ: 4 เมษายน 2568].
- PyPI. pymupdf4llm (Python Package Index) [ออนไลน์]. n.d. แหล่งที่มา: <https://pypi.org/project/pymupdf4llm/> [สืบค้นเมื่อ: 5 เมษายน 2568].

Belval, J.-D. pdf2image — Documentation [ออนไลน์]. n.d. แหล่งที่มา:
<https://pdf2image.readthedocs.io/en/latest/> [สืบค้นเมื่อ: 7 เมษายน 2568].

Belval, J.-D. pdf2image (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/Belval/pdf2image> [สืบค้นเมื่อ: 8 เมษายน 2568].

Freedesktop.org. Poppler — PDF rendering library (official site) [ออนไลน์]. n.d. แหล่งที่มา:
<https://poppler.freedesktop.org/> [สืบค้นเมื่อ: 9 เมษายน 2568].

oschwartz10612. poppler-windows (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/oschwartz10612/poppler-windows> [สืบค้นเมื่อ: 10 เมษายน 2568].

JaidedAI. EasyOCR (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/JaidedAI/EasyOCR> [สืบค้นเมื่อ: 12 เมษายน 2568].

Saffsd. langid.py (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/saffsd/langid.py> [สืบค้นเมื่อ: 2 พฤษภาคม 2568].

Barbaresi, A. py3langid (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/adbar/py3langid> [สืบค้นเมื่อ: 5 พฤษภาคม 2568].

OpenCV.org. OpenCV — Documentation [ออนไลน์]. n.d. แหล่งที่มา:
<https://docs.opencv.org/> [สืบค้นเมื่อ: 10 พฤษภาคม 2568].

NumPy Developers. NumPy — Official site [ออนไลน์]. n.d. แหล่งที่มา: <https://numpy.org/>
[สืบค้นเมื่อ: 12 พฤษภาคม 2568].

PaddlePaddle. PaddleOCR (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/PaddlePaddle/PaddleOCR> [สืบค้นเมื่อ: 18 พฤษภาคม 2568].

OpenAI. tiktoken (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/openai/tiktoken> [สืบค้นเมื่อ: 2 มิถุนายน 2568].

OpenAI. Tokenizer Tool [ออนไลน์]. n.d. แหล่งที่มา: <https://platform.openai.com/tokenizer>
[สืบค้นเมื่อ: 3 มิถุนายน 2568].

Leon, S. simhash (Python) — GitHub repository [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/leonsim/simhash> [สืบค้นเมื่อ: 8 มิถุนายน 2568].

Moz. simhash-py (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/seomoz/simhash-py> [สืบค้นเมื่อ: 9 มิถุนายน 2568].

Heafield, K. KenLM (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/kpu/kenlm> [สืบค้นเมื่อ: 15 มิถุนายน 2568].

Heafield, K. KenLM — Language Model Toolkit (official page) [ออนไลน์]. n.d. แหล่งที่มา:
<https://kheafield.com/code/kenlm/> [สืบค้นเมื่อ: 16 มิถุนายน 2568].

- Chroma Core. Chroma (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/chroma-core/chroma> [สืบค้นเมื่อ: 1 กรกฎาคม 2568].
- LangChain AI. LangChain (Python docs) [ออนไลน์]. n.d. แหล่งที่มา:
<https://python.langchain.com/> [สืบค้นเมื่อ: 4 กรกฎาคม 2568].
- BAAI / FlagOpen. FlagEmbedding (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา:
<https://github.com/FlagOpen/FlagEmbedding> [สืบค้นเมื่อ: 5 สิงหาคม 2568].
- BAAI. BGE-M3 (Hugging Face model card) [ออนไลน์]. 2023. แหล่งที่มา:
<https://huggingface.co/BAAI/bge-m3> [สืบค้นเมื่อ: 12 สิงหาคม 2568].
- BAAI. BGE-Reranker-v2-m3 (Hugging Face model card) [ออนไลน์]. 2024. แหล่งที่มา:
<https://huggingface.co/BAAI/bge-reranker-v2-m3> [สืบค้นเมื่อ: 20 สิงหาคม 2568].
- UKP Lab. SentenceTransformers — Official docs [ออนไลน์]. n.d. แหล่งที่มา:
<https://www.sbert.net/> [สืบค้นเมื่อ: 25 สิงหาคม 2568].
- BEIR Project. BEIR (GitHub repository) [ออนไลน์]. n.d. แหล่งที่มา: <https://github.com/beir-cellar/beir> [สืบค้นเมื่อ: 5 กันยายน 2568].
- Thakur, N.; Reimers, N.; et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. arXiv [ออนไลน์]. 2021. แหล่งที่มา:
<https://arxiv.org/abs/2104.08663> [สืบค้นเมื่อ: 6 กันยายน 2568].
- Exploding Gradients. RAGAS — Documentation [ออนไลน์]. n.d. แหล่งที่มา:
<https://docs.ragas.io/> [สืบค้นเมื่อ: 15 กันยายน 2568].
- LangChain. LangSmith — Evaluate RAG Tutorial [ออนไลน์]. n.d. แหล่งที่มา:
<https://docs.langchain.com/langsmith/evaluate-rag-tutorial> [สืบค้นเมื่อ: 20 กันยายน 2568].

ภาคผนวก ก

รายการซอร์สโค้ด

ก.1 โค้ดการดึงข้อมูลดิบ

```
from __future__ import annotations
import asyncio, hashlib, io, json, os, re, sys, time
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any, Dict, List, Optional, Tuple
from urllib.parse import urljoin, urlparse, urlsplit, urlunsplit

import aiohttp
from bs4 import BeautifulSoup
import numpy as np
import cv2

import trafilatura
from trafilatura.settings import Extractor

import fitz # PyMuPDF
import pymupdf4llm # PDF -> Markdown
from pdf2image import convert_from_path
import easyocr
from langid.langid import LanguageIdentifier, model as LANGID_MODEL
_LANGID_ID = LanguageIdentifier.from_modelstring(LANGID_MODEL, norm_probs=True)

from playwright.async_api import async_playwright
from w3lib.url import canonicalize_url as w3_canonicalize_url, url_query_cleaner
# ===== Logging =====
LOG_LVL = 20
LEVELS = {"DEBUG":10, "INFO":20, "WARNING":30, "ERROR":40, "CRITICAL":50}

def now() -> str: return datetime.now(timezone.utc).isoformat()
```

```

def log(level: str, msg: str, **kw):
    if LEVELS.get(level, 100) < LOG_LVL: return
    extra = " ".join(f"{k}={kw[k]}" for k in kw)
    print(f"[{now()}] {level} {msg} {extra}")

# ===== Config =====
class CFG:
    ALLOWED_HOSTS = (
        "it.chula.ac.th",
        "reg.chula.ac.th", "www.reg.chula.ac.th",
        "sa.chula.ac.th",
    )
    START_URLS = (
        "https://www.it.chula.ac.th/en/",
        "https://www.it.chula.ac.th/th/",
        "https://www.reg.chula.ac.th/",
    )
    OUT_HTML = "output/raw/html"
    OUT_PDF = "output/raw/pdfs"
    OUT_IMG = "output/raw/images"
    OUT_STAGE = "output/stage"
    MANIFEST = "output/stage/manifest.jsonl"
    DOCS = "output/stage/documents.jsonl"
    MAX_PAGES = 1000000
    MAX_DEPTH = 10
    PER_HOST_RPS = 2.0
    HEADLESS = True
    NAV_TIMEOUT_MS = 30000
    WAIT_SELECTOR = "body"
    POPPLER_PATH = r"C:\Users\gpaki\Downloads\Release-24.08.0-0\poppler-24.08.0\Library\bin"
    OCR_LANGS = ("th", "en")

# ===== Shared helpers =====
SEEN_SHA1: set[str] = set()

```



```

def ensure_dirs():
    for p in (CFG.OUT_HTML, CFG.OUT_PDF, CFG.OUT_IMG, CFG.OUT_STAGE):
        os.makedirs(p, exist_ok=True)

def sha1_text(s: str) -> str: return hashlib.sha1(s.encode("utf-8")).hexdigest()

def host_allowed(u: str) -> bool:
    try:
        host = urlparse(u).hostname or ""
        return any(host == h or host.endswith("." + h) for h in CFG.ALLOWED_HOSTS)
    except Exception:
        return False

def safe_urljoin(base: str, href: Optional[str]) -> Optional[str]:
    if not href: return None
    href = href.strip()
    if not href or href.startswith(("#", "mailto:", "tel:", "javascript:", "data:")):
        return None
    try:
        u = urljoin(base, href)
        urlparse(u)
        return u
    except Exception:
        return None

_TRACKING_PARAMS =
{"utm_source", "utm_medium", "utm_campaign", "utm_term", "utm_content", "utm_id", "fbclid", "gclid", "_gl", "ref"}

```

```

def canonicalize_url(url: str, keep_params: set[str] | None = None, drop_params: set[str] | None =
None, force_https: bool = True) -> str:
    if not url: return url
    try: u = urlsplit(url)
    except Exception: return url
    scheme = (u.scheme or "").lower()
    if force_https and scheme in ("http", "https"):
        url = urlunsplit(("https", u.netloc, u.path, u.query, ""))
    url2 = w3_canonicalize_url(url, keep_fragments=False)
    if keep_params:
        cleaned = url_query_cleaner(url2, parameterlist=list(keep_params), remove=False)
    else:
        drops = _TRACKING_PARAMS | set(drop_params or [])
        cleaned = url_query_cleaner(url2, parameterlist=list(drops), remove=True)
    try:
        s = urlsplit(cleaned)
        path = s.path or "/"
        if path.endswith(("index.html", "index.htm")):
            path = path.rsplit("/", 1)[0] + "/"
        cleaned = urlunsplit((s.scheme, s.netloc, path, s.query, ""))
    except Exception: pass
    return cleaned

def canonical_from_html(html: str, base: str) -> Optional[str]:
    soup = BeautifulSoup(html, "lxml")
    link = soup.find("link", attrs={"rel": re.compile("^canonical$", re.I)})
    if link and link.get("href"): return safe_urljoin(base, link.get("href"))
    return None

def append_jsonl(path: str, obj: dict):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "a", encoding="utf-8") as f:
        f.write(json.dumps(obj, ensure_ascii=False) + "\n")

```

```

class RateLimiter:
    def __init__(self, rps: float):
        self.delay = 1.0 / max(0.1, rps)
        self._last: dict[str, float] = {}
        self._lock = asyncio.Lock()
    async def wait(self, host: str):
        async with self._lock:
            now_t = time.monotonic(); t = self._last.get(host, 0.0)
            delta = now_t - t
            if delta < self.delay: await asyncio.sleep(self.delay - delta)
            self._last[host] = time.monotonic()

RATE = RateLimiter(CFG.PER_HOST_RPS)

async def fetch_html_playwright(page, url: str) -> Optional[str]:
    try:
        await RATE.wait(urlparse(url).hostname or "")
        resp = await page.goto(url, wait_until="domcontentloaded", timeout=CFG.NAV_TIMEOUT_MS)
        if not resp or resp.status >= 400:
            log("WARNING", "page_non200", url=url, status=(resp.status if resp else None)); return None
        raw_ct = await resp.header_value("content-type")
        ct = (raw_ct or "").split("; ", 1)[0].strip().lower()
        if (not ct) or (ct in {"text/html", "application/xhtml+xml", "text/plain"}):
            try:
                await page.wait_for_selector(CFG.WAIT_SELECTOR, state="attached",
                timeout=CFG.NAV_TIMEOUT_MS)
            except Exception as e:
                log("WARNING", "wait_selector_timeout", url=url, selector=CFG.WAIT_SELECTOR,
                err=str(e)); return None
            return await page.content()
        log("INFO", "skip_non_html", url=url, content_type=(ct or "<none>")); return None
    except Exception as e:
        log("ERROR", "goto_error", url=url, err=str(e)); return None

```

```

async def fetch_text(session: aiohttp.ClientSession, url: str, timeout: int = 20) -> Optional[str]:
    try:
        async with session.get(url, timeout=timeout) as r:
            if r.status == 200: return await r.text()
        return None
    except Exception: return None

async def discover_sitemaps(session: aiohttp.ClientSession, origin: str) -> List[str]:
    robots_url = urljoin(origin, "/robots.txt")
    text = await fetch_text(session, robots_url)
    maps: set[str] = set()
    if text:
        for line in text.splitlines():
            if line.lower().startswith("sitemap:"):
                maps.add(line.split(":", 1)[1].strip())
    for path in ("/sitemap.xml", "/sitemap_index.xml", "/wp-sitemap.xml"):
        maps.add(urljoin(origin, path))
    return list(maps)

async def parse_sitemap(session: aiohttp.ClientSession, sm_url: str) -> List[str]:
    xml = await fetch_text(session, sm_url)
    if not xml or "<" not in xml: return []
    urls: List[str] = []
    for m in re.finditer(r"<loc>(.*?)</loc>", xml, re.I | re.S):
        u = m.group(1).strip()
        if host_allowed(u): urls.append(u)
    return urls[:20000]

```

```

@dataclass
class PageItem:
    url: str
    depth: int

async def gather_seeds(origins: List[str]) -> List[str]:
    urls: List[str] = []
    async with aiohttp.ClientSession(headers={"User-Agent": "RAG-Ingest/1.0"}) as s:
        for origin in origins:
            maps = await discover_sitemaps(s, origin)
            for sm in maps:
                urls += await parse_sitemap(s, sm)
    urls += list(CFG.START_URLS)
    out, seen = [], set()
    for u in urls:
        cu = canonicalize_url(u)
        if host_allowed(cu) and cu not in seen:
            seen.add(cu); out.append(cu)
    return out

# ---- link/pdf/image discovery
PDF_URL_PAT = re.compile(r"\.pdf(?:[?#].*)?$", re.I)
IMG_URL_PAT = re.compile(r"\.(?:png|jpe?g|webp|gif|bmp|tiff?)(?:[?#].*)?$", re.I)

```

```

def extract_links_pdfs_imgs(html: str, base: str) -> Tuple[List[str], List[str], List[Tuple[str,
Dict[str,str]]]]:
    soup = BeautifulSoup(html, "lxml")
    links: List[str] = []; pdfs: List[str] = []; images: List[Tuple[str, Dict[str,str]]] = []
    def _add(u: Optional[str]) -> Optional[str]:
        if u and host_allowed(u): return u
        return None
    for a in soup.find_all("a"):
        u = safe_urljoin(base, a.get("href"));
        if not u: continue
        if host_allowed(u):
            links.append(u)
            if PDF_URL_PAT.search(u): pdfs.append(u)
    for img in soup.find_all("img"):
        alt = (img.get("alt") or "").strip()
        src = _add(safe_urljoin(base, img.get("src")))
        if src and IMG_URL_PAT.search(src):
            images.append((src, {"alt": alt, "src_hint": img.get("src") or ""}))
        ss = img.get("srcset")
        if ss:
            for part in ss.split(","):
                cand = part.strip().split(" ")[0]
                u2 = _add(safe_urljoin(base, cand))
                if u2 and IMG_URL_PAT.search(u2):
                    images.append((u2, {"alt": alt, "src_hint": cand}))
    for s in soup.select("picture source[srcset]"):
        ss = s.get("srcset") or ""
        for part in ss.split(","):
            cand = part.strip().split(" ")[0]
            u3 = _add(safe_urljoin(base, cand))
            if u3 and IMG_URL_PAT.search(u3): images.append((u3, {"alt": "", "src_hint": cand}))

```

```

for el in soup.select("[style*='background-image']"):
    st = el.get("style") or ""; m = re.search(r"url\(((^\\))+)", st)
    if m:
        cand = m.group(1).strip(" ' \""); u4 = _add(safe_urljoin(base, cand))
        if u4 and IMG_URL_PAT.search(u4): images.append((u4, {"alt":"","src_hint":cand}))
for el in soup.find_all(True):
    for a in ("data-href","data-src"):
        u = safe_urljoin(base, el.get(a))
        if u and host_allowed(u) and PDF_URL_PAT.search(u): pdfs.append(u)
def _strip_frag(u: str) -> str:
    try: p = urlparse(u); return p._replace(fragment="").geturl()
    except Exception: return u
links = list(dict.fromkeys(_strip_frag(u) for u in links))
pdfs = list(dict.fromkeys(pdfs))
seen: set[str] = set(); imgs: List[Tuple[str, Dict[str,str]]] = []
for u, meta in images:
    u2 = _strip_frag(u)
    if u2 in seen: continue
    seen.add(u2); imgs.append((u2, meta))
return links, pdfs, imgs

# ===== Main=====
MAX_PAGES = None
MAX_DEPTH = None

async def step1_fetch_html(max_pages: int | None = None, max_depth: int | None = None):
    ensure_dirs()
    origins = sorted({f"https://{h}" for h in CFG.ALLOWED_HOSTS})
    seeds = await gather_seeds(origins)
    log("INFO","seeds_ready", count=len(seeds))

    async with async_playwright() as pw:
        browser = await pw.chromium.launch(headless=CFG.HEADLESS)
        ctx = await browser.new_context(accept_downloads=True, ignore_https_errors=True)
        page = await ctx.new_page()
        q: asyncio.Queue[PageItem] = asyncio.Queue()

```



```

seen: set[str] = set()
assets_seen: set[str] = set()
# Seed queue (expand XML sitemaps into concrete page URLs first)
async with aiohttp.ClientSession(headers={"User-Agent": "RAG-Ingest/1.0"}) as sx:
    for u in seeds:
        cu = canonicalize_url(u)
        if cu.lower().endswith(".xml"):
            try:
                pending = [cu]
                visited_xml: set[str] = set()
                while pending:
                    x = pending.pop()
                    if x in visited_xml:
                        continue
                    visited_xml.add(x)
                try:
                    child_urls = await parse_sitemap(sx, x)
                except Exception:
                    child_urls = []
                for c in child_urls:
                    cc = canonicalize_url(c)
                    if cc.lower().endswith(".xml"):
                        pending.append(cc)
                    elif host_allowed(cc):
                        await q.put(Pageltem(url=cc, depth=0))
            except Exception as e:
                log("WARNING", "seed_xml_expand_failed", url=cu, err=str(e))
        else:
            await q.put(Pageltem(url=cu, depth=0))
    pages_done = 0
    depth_limit = CFG.MAX_DEPTH if max_depth is None else max_depth
    page_limit = CFG.MAX_PAGES if max_pages is None else max_pages
    while not q.empty() and pages_done < page_limit:
        item = await q.get()
        if item.url in seen or not host_allowed(item.url):
            continue

```

```

assets_seen: set[str] = set()

# Seed queue (expand XML sitemaps into concrete page URLs first)
async with aiohttp.ClientSession(headers={"User-Agent": "RAG-Ingest/1.0"}) as sx:
    for u in seeds:
        cu = canonicalize_url(u)
        if cu.lower().endswith(".xml"):
            try:
                pending = [cu]
                visited_xml: set[str] = set()
                while pending:
                    x = pending.pop()
                    if x in visited_xml:
                        continue
                    visited_xml.add(x)
                try:
                    child_urls = await parse_sitemap(sx, x)
                except Exception:
                    child_urls = []
                for c in child_urls:
                    cc = canonicalize_url(c)
                    if cc.lower().endswith(".xml"):
                        pending.append(cc)
                    elif host_allowed(cc):
                        await q.put(Pageltem(url=cc, depth=0))
            except Exception as e:
                log("WARNING", "seed_xml_expand_failed", url=cu, err=str(e))
        else:
            await q.put(Pageltem(url=cu, depth=0))
pages_done = 0
depth_limit = CFG.MAX_DEPTH if max_depth is None else max_depth
page_limit = CFG.MAX_PAGES if max_pages is None else max_pages
while not q.empty() and pages_done < page_limit:
    item = await q.get()
    if item.url in seen or not host_allowed(item.url):
        continue
    seen.add(item.url)

```

```

html = await fetch_html_playwright(page, item.url)
if not html:
    continue

canon = canonicalize_url(canonical_from_html(html, item.url) or item.url)
html_path = os.path.join(CFG.OUT_HTML, f"{sha1_text(canon)}.html")
with open(html_path, "w", encoding="utf-8") as f:
    f.write(html)

append_jsonl(CFG.MANIFEST, {"record": "page", "url": item.url, "canonical": canon,
"html_path": html_path, "html_path_abs": os.path.abspath(html_path), "ts": now()})

# extract links and assets from this page
links, pdfs, images = extract_links_pdfs_imgs(html, canon)

# download & index assets immediately (RAW-first design)
async with aiohttp.ClientSession(headers={"User-Agent": "RAG-Ingest/1.0"}) as dl:
    # PDFs
    for pu in pdfs:
        pu_c = canonicalize_url(pu)
        if pu_c in assets_seen:
            continue
        try:
            await RATE.wait(urlparse(pu_c).hostname or "")
            async with dl.get(pu_c, headers={"Referer": canon}, timeout=30) as r:
                if r.status != 200:
                    log("WARNING", "pdf_download_non200", url=pu_c, status=r.status)
                    continue
                data = await r.read()
        except Exception as e:
            log("ERROR", "pdf_download_failed", url=pu_c, err=str(e))
            continue
        pdf_path = os.path.join(CFG.OUT_PDF, f"{sha1_text(pu_c)}.pdf")
        with open(pdf_path, "wb") as pf:
            pf.write(data)
        append_jsonl(CFG.MANIFEST,
{"record": "asset", "asset_type": "pdf", "url": pu_c, "path": pdf_path, "page": canon, "ts": now()})
        assets_seen.add(pu_c)

```

```

# Images
for iu, meta in images:
    iu_c = canonicalize_url(iu)
    if iu_c in assets_seen:
        continue
    try:
        await RATE.wait(urlparse(iu_c).hostname or "")
        async with dl.get(iu_c, headers={"Referer": canon}, timeout=30) as r:
            if r.status != 200:
                log("WARNING", "img_download_non200", url=iu_c, status=r.status)
                continue
            data = await r.read()
        except Exception as e:
            log("ERROR", "img_download_failed", url=iu_c, err=str(e))
            continue
        ext = os.path.splitext(urlparse(iu_c).path)[1].lower() or ".bin"
        img_path = os.path.join(CFG.OUT_IMG, f"{sha1_text(iu_c)}{ext}")
        with open(img_path, "wb") as fimg:
            fimg.write(data)
        append_jsonl(CFG.MANIFEST,
{"record": "asset", "asset_type": "image", "url": iu_c, "path": img_path, "alt": meta.get("alt", ""), "page": canon, "ts":
:now()})

        assets_seen.add(iu_c)

# Enqueue new links if within depth limit
if item.depth < depth_limit:
    for lk in links:
        lk_c = canonicalize_url(lk)
        if lk_c not in seen and host_allowed(lk_c):
            await q.put(Pageltem(url=lk_c, depth=item.depth+1))
    pages_done += 1
    if pages_done % 10 == 0:
        log("INFO", "progress", pages=pages_done, queue=q.qsize())
    await browser.close()
    log("INFO", "step1_done", pages=pages_done)

# ===== Run Step 1 =====
_asyncio.run(step1_fetch_html(max_pages=MAX_PAGES, max_depth=MAX_DEPTH))

```

ก.2 โค้ดการสกัดข้อมูล

ก.2.1 การตั้งค่า config

```
# - Config (CFG): MANIFEST, DOCS, POPPLER_PATH, OCR_LANGS
# - Helpers: ensure_dirs, hydrate_seen_sha1, content_sha1, sha1_text,
#           append_jsonl, now, log, get_easyocr_reader
# - Globals: SEEN_SHA1, LOG_LEVEL

from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from functools import lru_cache
from typing import Any, Dict, Iterable, Optional, Set, Union
from datetime import datetime
from zoneinfo import ZoneInfo
import hashlib
import json
import os
import sys
import easyocr

# ----- CONFIG -----

@dataclass
class Config:
    OUT_HTML: str = os.environ.get("OUT_HTML", "output/raw/html")
    OUT_PDF: str = os.environ.get("OUT_PDF", "output/raw/pdfs")
    OUT_IMG: str = os.environ.get("OUT_IMG", "output/raw/images")
    OUT_STAGE: str = os.environ.get("OUT_STAGE", "output/stage")
    MANIFEST: str = os.environ.get("MANIFEST", "output/stage/manifest.jsonl")
    DOCS: str = os.environ.get("DOCS", "output/stage/documents.jsonl")
    POPPLER_PATH: Optional[Path] = None
    OCR_LANGS: tuple[str, ...] = ("en", "th")
```

```

# ค่าตั้งต้น ปรับตามโปรเจกต์ได้
CFG = Config(
    MANIFEST=Path("data/manifest.jsonl"),
    DOCS=Path("output/documents.jsonl"),
    POPPLER_PATH=Path(os.getenv("POPPLER_PATH", r"C:\Users\gpaki\Downloads\Release-
24.08.0-0\poppler-24.08.0\Library\bin")).resolve() if os.getenv("POPPLER_PATH") else None,
    OCR_LANGS=tuple([s.strip() for s in os.getenv("OCR_LANGS", "en,th").split(",") if s.strip()]),
)

# ----- LOGGING -----

LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO").upper()
_LEVELS = {"DEBUG": 10, "INFO": 20, "WARN": 30, "ERROR": 40}
_TZ = ZoneInfo(os.getenv("TZ", "Asia/Bangkok"))

def now() -> str:
    """คืนค่าเวลาปัจจุบันในเขตเวลา Asia/Bangkok เป็น ISO8601 (ตัด milliseconds)"""
    return datetime.now(_TZ).replace(microsecond=0).isoformat()

def log(level: str, message: str, **fields: Any) -> None:
    lvl = level.upper()
    if _LEVELS.get(lvl, 999) < _LEVELS.get(LOG_LEVEL, 20):
        return
    rec = {"ts": now(), "level": lvl, "msg": message}
    if fields:
        rec.update(fields)
    print(json.dumps(rec, ensure_ascii=False), file=sys.stderr)

```

```

# ----- I/O & PATHS -----

def ensure_dirs(*paths: Path) -> None:
    targets = list(paths) or [CFG.DOCS.parent]
    for p in targets:
        p = Path(p)
        if p.suffix: # ถ้าเป็นไฟล์ให้ใช้โฟลเดอร์แม่
            p = p.parent
        p.mkdir(parents=True, exist_ok=True)

def append_jsonl(path: Union[str, Path], record: Dict[str, Any]) -> None:
    """
    เขียน 1 ระเบียบลงไฟล์ .jsonl (append) — ใช้ทุกสัปดาห์เวลาบันทึกเอกสาร
    """
    path = Path(path)
    ensure_dirs(path)
    with path.open("a", encoding="utf-8") as f:
        f.write(json.dumps(record, ensure_ascii=False) + "\n")

# ----- HASH/ID -----

def _to_bytes(x: Union[str, bytes]) -> bytes:
    if isinstance(x, bytes):
        return x
    return x.encode("utf-8", "ignore")

def content_sha1(content: Union[str, bytes]) -> str:
    return hashlib.sha1(_to_bytes(content)).hexdigest()

def sha1_text(text: str) -> str:
    return hashlib.sha1(text.encode("utf-8", "ignore")).hexdigest()

```

```

# ----- DEDUP STATE -----
SEEN_SHA1: Set[str] = set()
def hydrate_seen_sha1(docs_path: Union[str, Path] = None) -> Set[str]:
    docs = Path(docs_path) if docs_path else CFG.DOCS
    if not docs.exists():
        SEEN_SHA1.clear()
        return SEEN_SHA1
    loaded: Set[str] = set()
    with docs.open("r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            try:
                obj = json.loads(line)
            except json.JSONDecodeError:
                continue
            for key in ("sha1", "content_sha1", "hash"):
                val = obj.get(key)
                if isinstance(val, str) and len(val) >= 8:
                    loaded.add(val)
                    break
    SEEN_SHA1.clear()
    SEEN_SHA1.update(loaded)
    log("DEBUG", "hydrate_seen_sha1", count=len(SEEN_SHA1), path=str(docs))
    return SEEN_SHA1

```



```
# ----- OCR READER -----
def get_easyocr_reader():
    # ตรวจ GPU อัตโนมัติ (ถ้าไม่มี torch ให้ทำงานแบบ CPU)
    use_gpu = False
    try:
        import torch # type: ignore
        use_gpu = bool(torch.cuda.is_available())
    except Exception:
        use_gpu = False
    langs = list(CFG.OCR_LANGS) if CFG.OCR_LANGS else ["en"]
    reader = easyocr.Reader(langs, gpu=use_gpu)
    log("INFO", "easyocr.Reader ready", langs=langs, gpu=use_gpu)
    return reader
```



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

ก.2.2 การดึงข้อมูล HTML

```
import hashlib
import json
import trafilatura
from bs4 import BeautifulSoup

# ===== HTML extraction (Trafilatura) =====
# No Extractor needed, use trafilatura.extract directly

def extract_meta_min(html: str, default_url: Optional[str] = None) -> Dict[str, Any]:
    try:
        meta_json = trafilatura.extract(html, output_format="json", with_metadata=True)
        if not meta_json: return {}
        m = json.loads(meta_json)
        return {"title": m.get("title"), "published_at": m.get("date"), "updated_at": m.get("modified") or
m.get("date_modified")}
    except Exception as e:
        log("WARNING", "trafilatura_meta_failed", err=str(e)); return {}

def html_to_text(html: str) -> str:
    if trafilatura is not None:
        txt = trafilatura.extract(
            html,
            include_tables=True,
            include_formatting=True,
            favor_recall=True,
            deduplicate=True,)
        if txt:
            return txt
    soup = BeautifulSoup(html, "xml")
    return soup.get_text("\n", strip=True)
```

===== Step 2: แปลง HTML → ข้อความ แล้วเขียนลง documents.jsonl =====

```
ensure_dirs(); hydrate_seen_sha1(); added = 0
```

```
if not os.path.exists(CFG.MANIFEST):
```

```
    log("ERROR","manifest_missing", path=CFG.MANIFEST)
```

```
else:
```

```
    with open(CFG.MANIFEST, encoding="utf-8") as f:
```

```
        for line in f:
```

```
            try: m = json.loads(line)
```

```
            except Exception: continue
```

```
            if m.get("record") != "page":
```

```
                continue
```

```
            html_path = m.get("html_path"); src = m.get("canonical")
```

```
            if (not html_path) or (not os.path.exists(html_path)) or (not src):
```

```
                log("ERROR","manifest_row_invalid"); continue
```

```
            with open(html_path, encoding="utf-8") as hf:
```

```
                html = hf.read()
```

```
            clean = html_to_text(html)
```

```
            if not clean: continue
```

```
            sha1_val = content_sha1(clean)
```

```
            if sha1_val in SEEN_SHA1: continue
```

```
            SEEN_SHA1.add(sha1_val)
```

```
            rec = {"id": sha1_text(src), "source_url": src, "source_type": "html", "text_full": clean,
```

```
"content_hash": sha1_val, "captured_at": now()}}
```

```
            meta = extract_meta_min(html, src)
```

```
            if meta: rec.update(meta)
```

```
            append_jsonl(CFG.DOCS, rec); added += 1
```

```
log("INFO","step2_done", html_docs=added)
```

ก.2.3 การดึงข้อมูล Images

```
import cv2
import numpy as np
import easyocr

def _poly_area(poly: List[List[float]]) -> float:
    x = np.array([p[0] for p in poly], dtype=float); y = np.array([p[1] for p in poly], dtype=float)
    return 0.5 * abs(np.dot(x, np.roll(y, 1)) - np.dot(y, np.roll(x, 1)))

def is_infographic(img_np: np.ndarray, alt_text: str = "", src_hint: str = "") -> bool:
    h, w = img_np.shape[:2]
    if min(h, w) < 320: return False
    long_side = max(h, w); scale = 800/long_side
    img_small = cv2.resize(img_np, (int(w*scale), int(h*scale)), interpolation=cv2.INTER_AREA) if
scale < 1.0 else img_np
    reader = get_easyocr_reader(); res = reader.readtext(img_small, detail=1, paragraph=False)
    if not res:
        at = f"{alt_text} {src_hint}".lower();
        for k in ("กำหนดการ", "ประกาศ", "ขั้นตอน", "รายชื่อ", "ลงทะเบียน", "ชำระเงิน", "reg", "chula", "office of the
registrar", "student journey"):
            if k in at: return True
        return False
    img_area = img_small.shape[0]*img_small.shape[1]
    boxes_area = sum(_poly_area(r[0]) for r in res)
    text_area_ratio = boxes_area / max(1, img_area)
    confs = [float(r[2]) for r in res if isinstance(r[2], (float,int))]
    avg_conf = (sum(confs)/len(confs)) if confs else 0.0
    texts_join = " ".join([r[1] for r in res if isinstance(r[1], str)].lower())
    if len(res) >= 6: return True
    if text_area_ratio >= 0.08 and avg_conf >= 0.5: return True
    for k in ("กำหนดการ", "ประกาศ", "ขั้นตอน", "รายชื่อ", "ลงทะเบียน", "ชำระเงิน", "reg", "chula", "office of the
registrar", "student journey"):
        if k in texts_join: return True
    return False
```

```

def ocr_image_highres(img_np: np.ndarray) -> str:
    H, W = img_np.shape[:2]; scale = 1.8 if max(H, W) < 1600 else 1.2
    img_big = cv2.resize(img_np, (int(W*scale), int(H*scale)), interpolation=cv2.INTER_CUBIC)
    reader = get_easyocr_reader(); out = reader.readtext(img_big, detail=0)
    return "".join(out).strip()

# ===== Step 3: ดาวน์โหลดจาก manifest → OCR เฉพาะ infographic =====
import aiohttp as _aiohttp

async def _http_get_bytes(url: str, referer: str | None = None, timeout: int = 30) -> bytes | None:
    headers = {"User-Agent": "RAG-Ingest/1.0"};
    if referer: headers["Referer"] = referer
    try:
        async with _aiohttp.ClientSession(headers=headers) as s:
            async with s.get(url, timeout=timeout) as r:
                if r.status == 200: return await r.read()
    except Exception as e:
        log("WARNING", "http_get_bytes_failed", url=url, err=str(e))
    return None

# ===== Run Step 3 =====
ensure_dirs(); hydrate_seen_sha1(); added = 0

if not os.path.exists(CFG.MANIFEST):
    log("ERROR", "manifest_missing", path=CFG.MANIFEST)
else:
    with open(CFG.MANIFEST, encoding="utf-8") as f:
        for line in f:
            try: m = json.loads(line)
            except Exception: continue
            if m.get("record") != "asset" or m.get("asset_type") != "image":
                continue
            img_path = m.get("path"); iu = m.get("url"); alt = m.get("alt", ""); canon = m.get("page")
            if (not img_path) or (not os.path.exists(img_path)) or (not iu):
                log("ERROR", "asset_image_missing", path=img_path, url=iu);
                continue

```

```

try:
    with open(img_path, "rb") as rf:
        data = rf.read()

        buf = np.frombuffer(data, np.uint8); img_np = cv2.imdecode(buf, cv2.IMREAD_COLOR)
        if img_np is None:
            log("ERROR", "asset_image_decode_failed", path=img_path);
            continue
except Exception as e:
    log("ERROR", "asset_image_read_failed", path=img_path, err=str(e));
    continue

if not is_infographic(img_np, alt_text=alt, src_hint=m.get("src_hint", "")):
    continue

# High-res OCR + QC metrics (เก็บเฉพาะ 2 ค่า)
H, W = img_np.shape[:2]
scale = 1.8 if max(H, W) < 1600 else 1.2
img_big = cv2.resize(img_np, (int(W*scale), int(H*scale)), interpolation=cv2.INTER_CUBIC)

reader = get_easyocr_reader()
res = reader.readtext(img_big, detail=1, paragraph=False)

text_img = " ".join([r[1] for r in res]).strip()
if not text_img:
    continue

confs = [float(r[2]) for r in res if isinstance(r[2], (int, float))]
ocr_avg_conf = (sum(confs)/len(confs)) if confs else 0.0

img_area = img_big.shape[0] * img_big.shape[1]
boxes_area = sum(_poly_area(r[0]) for r in res) if res else 0.0
ocr_text_area_ratio = float(boxes_area / img_area) if img_area else 0.0
sha1_val = content_sha1(text_img)
if sha1_val in SEEN_SHA1:
    continue

```

```
SEEN_SHA1.add(sha1_val)
```

```
rec = {  
    "id": sha1_val,  
    "source_url": iu,  
    "source_type": "image",  
    "is_infographic": True,  
    "text_full": text_img,  
    "alt": alt,  
    "content_hash": sha1_val,  
    "captured_at": now(),  
    "page": canon,  
    "ocr_avg_conf": round(ocr_avg_conf, 3),  
    "ocr_text_area_ratio": round(ocr_text_area_ratio, 4),  
}  
append_jsonl(CFG.DOCS, rec); added += 1
```

```
log("INFO","step3_done", image_docs=added)
```

CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

ก.2.4 การดึงข้อมูล PDFs

```
import fitz # ถ้าใช้ร่วม
import pymupdf4llm
import hashlib
import numpy as np
import cv2
import subprocess, shutil, unicodedata, re, numpy as np, cv2, fitz, pymupdf4llm

# ใช้ pattern เดิม (อนุโลม TH/EN + punctuation ทั่วไป)
_ALLOWED_RE = re.compile(r"[A-Za-z0-9\u0E00-\u0E7F\s\.,:;!\"'()?\\[\]{}|\-_\@#%&*+|=<>~`°—
...•]")

from langid.langid import LanguageIdentifier, model
_LANGID_ID = LanguageIdentifier.from_modelstring(model, norm_probs=True)

def md_has_table(md: str) -> bool:
    if not md:
        return False
    low = md.lower()
    if "<table" in low:
        return True
    lines = [ln.strip() for ln in md.splitlines()]
    for i in range(len(lines) - 1):
        a, b = lines[i], lines[i + 1]
        if a.startswith("|") and a.endswith("|") and "|" in a:
            parts = [p.strip() for p in b.split("|") if p.strip()]
            if parts and all((set(p).issubset(set("-: ")) and len(p.replace(":", "").replace(" ", "")) >= 3 for p in
parts):
                return True
    return False
```



```
def is_usable_th_en(text: str, min_len: int = 150, min_prob: float = 0.70) -> bool:
```

```
    if not text or len(text) < min_len:
```

```
        return False
```

```
    try:
```

```
        lang, prob = _LANGID_ID.classify(text)
```

```
        return (lang in ("th", "en")) and (prob >= min_prob)
```

```
    except Exception:
```

```
        return False
```

```
def _qc_ok(t: str, min_chars: int) -> bool:
```

```
    if not t or len(t) < min_chars:
```

```
        return False
```

```
    if "□" in t:                # replacement char -> treat as failed
```

```
        return False
```

```
    total = len(t)
```

```
    bad = sum(1 for ch in t if not _ALLOWED_RE.match(ch))
```

```
    bad_ratio = (bad / total) if total else 1.0
```

```
    if bad_ratio >= 0.05:        # เข้มข้นเพี้ยน noise
```

```
        return False
```

```
    return True
```

```
def _pdftotext_whole(pdf_path: str) -> str:
```

```
    exe = "pdftotext.exe" if os.name == "nt" else "pdftotext"
```

```
    cmd = [exe, "-layout", "-enc", "UTF-8", pdf_path, "-"]
```

```
    env = os.environ.copy()
```

```
    if getattr(CFG, "POPPLER_PATH", None):
```

```
        env["PATH"] = str(CFG.POPPLER_PATH) + os.pathsep + env.get("PATH", "")
```

```
    out = subprocess.run(cmd, env=env, capture_output=True, check=True)
```

```
    return out.stdout.decode("utf-8", errors="strict").strip()
```

```
def _pdftotext_page(pdf_path: str, pno: int) -> str:
```

```
    exe = "pdftotext.exe" if os.name == "nt" else "pdftotext"
```

```
    cmd = [exe, "-layout", "-enc", "UTF-8", "-f", str(pno), "-l", str(pno), pdf_path, "-"]
```

```
    env = os.environ.copy()
```

```
    if getattr(CFG, "POPPLER_PATH", None):
```

```
        env["PATH"] = str(CFG.POPPLER_PATH) + os.pathsep + env.get("PATH", "")
```

```
    out = subprocess.run(cmd, env=env, capture_output=True, check=True)
```

```
    return out.stdout.decode("utf-8", errors="strict").strip()
```

```

def pdf_text_per_page_with_langdetect_fallback(pdf_path: str, ocr_dpi: int = 220) -> str:
# เรียบง่าย: ต่อหน้า → PyMuPDF text → ถ้าไม่โอเคลอง pdftotext หน้าเดียว → ถ้ายังว่าง/เสีย → OCR
เฉพาะหน้านั้น

    out_pages = []
    reader = get_easyocr_reader()
    with fitz.open(pdf_path) as doc:
        for p in doc:
            t = (p.get_text("text", sort=True) or "").strip()
            if t and _qc_ok(t, min_chars=30) and is_usable_th_en(t, min_len=30, min_prob=0.55):
                out_pages.append(t); continue
            # ลอง Poppler เฉพาะหน้านั้นก่อนแทน OCR
            try:
                t2 = _pdftotext_page(pdf_path, p.number+1) # pdftotext หน้าฐาน 1
            except Exception:
                t2 = ""
            if t2 and _qc_ok(t2, min_chars=30) and is_usable_th_en(t2, min_len=30, min_prob=0.55):
                out_pages.append(t2); continue
            # สุดท้ายค่อย OCR เฉพาะหน้านี่
            pix = p.get_pixmap(dpi=ocr_dpi)
            img_bytes = pix.tobytes("png")
            arr = np.frombuffer(img_bytes, np.uint8)
            img = cv2.imdecode(arr, cv2.IMREAD_COLOR)
            if img is None:
                out_pages.append("")
            else:
                ocr_lines = reader.readtext(img, detail=0)
                out_pages.append(" ".join(ocr_lines).strip())
    return "".join([x for x in out_pages if x]).strip()

```

```

def pdf_to_markdown_with_ocr_fallback(pdf_path: str, min_chars: int = 150, ocr_dpi: int = 220) ->
str:
    # ลำดับพยายาม: PyMuPDF4LLM → pdftotext ทั้งไฟล์ → per-page salvage (เฉพาะหน้า) + OCR
หน้า
    # 1) PyMuPDF4LLM
    md = ""
    try:
        md = (pymupdf4llm.to_markdown(pdf_path) or "").strip()
    except Exception as e:
        log("WARNING", "pymupdf4llm_failed", path=pdf_path, err=str(e)); md = ""
        if md and _qc_ok(md, min_chars) and (is_usable_th_en(md, min_len=min_chars, min_prob=0.65)
or len(md) >= (min_chars*2)):
            return md

    # 2) Poppler (ทั้งไฟล์) – ยัง “ไม่ต้อง OCR”
    try:
        txt = _pdftotext_whole(pdf_path)
    except Exception as e:
        txt = ""
        log("WARNING", "pdftotext_whole_failed", path=pdf_path, err=str(e))
        if txt and _qc_ok(txt, min_chars) and (is_usable_th_en(txt, min_len=min_chars, min_prob=0.65) or
len(txt) >= (min_chars*2)):
            return txt

    # 3) Per-page salvage + OCR เฉพาะหน้า
    txt2 = pdf_text_per_page_with_langdetect_fallback(pdf_path, ocr_dpi=ocr_dpi)
    if not txt2 or not txt2.strip():
        raise RuntimeError("pdf_markdown_and_ocr_fallback_empty")
    return txt2

```

```

# ===== ดาวน์โหลด/แปลง PDF จาก manifest → documents.jsonl =====
import aiohttp as _aiohttp

# ===== Run Step 4 (inline, local-only) =====
ensure_dirs(); hydrate_seen_sha1(); added = 0

if not os.path.exists(CFG.MANIFEST):
    log("ERROR", "manifest_missing", path=CFG.MANIFEST)
else:
    # Read only pdf assets from manifest (local-only, no network)
    with open(CFG.MANIFEST, encoding="utf-8") as f:
        for line in f:
            try: m = json.loads(line)
            except Exception: continue

            if m.get("record") != "asset" or m.get("asset_type") != "pdf":
                continue

            pdf_path = m.get("path"); pu = m.get("url"); canon = m.get("page")
            if (not pdf_path) or (not os.path.exists(pdf_path)) or (not pu):
                log("ERROR", "asset_pdf_missing", path=pdf_path, url=pu);
                continue

            try:
                md = pdf_to_markdown_with_ocr_fallback(pdf_path)
                has_tbl = md_has_table(md)
            except Exception as e:
                log("ERROR", "pdf_to_markdown_failed", path=pdf_path, err=str(e));
                continue

            if not md:
                continue

            sha1_val = content_sha1(md)
            if sha1_val in SEEN_SHA1:
                continue

            SEEN_SHA1.add(sha1_val)

            rec = {"id": sha1_val, "source_url": pu, "source_type": "pdf_md", "format": "markdown",
                "has_tables": has_tbl, "text_full": md, "content_hash": sha1_val, "captured_at": now(), "page": canon}
            append_jsonl(CFG.DOCS, rec); added += 1

log("INFO", "step4_done", pdf_docs=added)

```

ก.3 การเตรียมข้อมูล

ก.3.1 การทำ chunking

```
from __future__ import annotations

import os, sys, json, hashlib, re

from typing import Dict, Any, List

# ----- Config -----

DOCS_JSONL = os.getenv("DOCS_JSONL", "output/stage/documents.jsonl")
CHUNKS_JSONL = os.getenv("CHUNKS_JSONL", "output/stage/chunks.jsonl")

CHUNK_SIZE_STRUCT = int(os.getenv("CHUNK_SIZE_STRUCT", "512")) # tokens
CHUNK_OVERLAP_STRUCT = float(os.getenv("CHUNK_OVERLAP_STRUCT", "0.12")) # 12%

CHUNK_SIZE_OCR = int(os.getenv("CHUNK_SIZE_OCR", "256")) # tokens
CHUNK_OVERLAP_OCR = float(os.getenv("CHUNK_OVERLAP_OCR", "0.20")) # 20%

SEPARATORS = ["\n\n", "\n", ". ", " ", ""]

# ----- Tokenization utils -----

import tiktoken # type: ignore
_ENC = tiktoken.get_encoding("cl100k_base")

def count_tokens(text: str) -> int:
    return len(_ENC.encode(text))

def trim_to_tokens(text: str, max_tokens: int) -> str:
    ids = _ENC.encode(text)
    return text if len(ids) <= max_tokens else _ENC.decode(ids[:max_tokens])

TOKEN_MODE = "tiktoken"

# ----- Structure heuristics -----

MD_TABLE_HDR = re.compile(r"^\s*\.\s*$", re.M)
MD_TABLE_SEP = re.compile(r"^\s*\s*?:-{3,}.\s*$", re.M)
HTML_TABLE = re.compile(r"<\s*table\s>.*?>", re.I | re.S)
MD_HEADER = re.compile(r"^\s*#{1,6}\s+.", re.M)
```

```

def looks_structured(text: str) -> bool:
    if not text:
        return False
    if MD_HEADER.search(text) or HTML_TABLE.search(text):
        return True
    if MD_TABLE_HDR.search(text) and MD_TABLE_SEP.search(text):
        return True
    nl = text.count("\n")
    return (nl >= 3 and (nl / max(1, len(text))) > 0.002)

# ----- Minimal helpers for splitting -----
def _hard_chunk(text: str, size: int, overlap: int) -> List[str]:
    if size <= 0:
        return [text]
    chunks: List[str] = []
    ids = _ENC.encode(text) # type: ignore # only if tiktoken available
    n = len(ids)
    step = size - overlap if size > overlap else size
    i = 0
    while i < n:
        end = min(n, i + size)
        chunk = _ENC.decode(ids[i:end]) # type: ignore
        chunks.append(chunk)
        if end >= n:
            break
        i += max(1, step)
    return chunks

```

```

def _chunk_once_at_sep(text: str, size: int, overlap: int, sep: str) -> List[str] | None:
    if sep == "":
        return _hard_chunk(text, size, overlap)
    if sep not in text:
        return None
    parts = text.split(sep)
    chunks: List[str] = []
    buf = ""
    for part in parts:
        piece = part if not buf else (sep + part)
        if count_tokens((buf + piece) if buf else part) <= size:
            buf = (buf + piece) if buf else part
            continue
        if buf:
            chunks.append(buf)
            if overlap > 0:
                approx_chars = overlap * (4 if TOKEN_MODE == "approx_chars" else 3)
                tail = buf[-int(approx_chars):]
                buf = tail + part
            else:
                buf = part
            if count_tokens(buf) > size:
                sub = _hard_chunk(buf, size, overlap)
                if sub:
                    chunks.extend(sub[:-1])
                    buf = sub[-1]
                else:
                    buf = ""
            else:
                sub = _hard_chunk(part, size, overlap)
                chunks.extend(sub)
                buf = ""
        if buf:
            chunks.append(buf)
    return chunks

```

```

# ----- Main (inline pipeline) -----
print(f"[INFO] Reading: {DOCS_JSONL}")
print(f"[INFO] Writing: {CHUNKS_JSONL}")
print(f"[INFO] Profiles: structured(size={CHUNK_SIZE_STRUCT},
overlap={CHUNK_OVERLAP_STRUCT}), "
      f"ocr_raw(size={CHUNK_SIZE_OCR}, overlap={CHUNK_OVERLAP_OCR})")

if not os.path.exists(DOCS_JSONL):
    raise FileNotFoundError(f"Input not found: {DOCS_JSONL}")

os.makedirs(os.path.dirname(CHUNKS_JSONL) or ".", exist_ok=True)

total_docs = 0
total_chunks = 0
seen_ids: set[str] = set()
with open(DOCS_JSONL, "r", encoding="utf-8") as fin, \
    open(CHUNKS_JSONL, "w", encoding="utf-8") as fout:
    for raw in fin:
        line = raw.strip()
        if not line:
            continue
        try:
            rec: Dict[str, Any] = json.loads(line)
        except Exception:
            print(f"[WARN] Bad JSONL line skipped: {line[:200]}...", file=sys.stderr)
            continue
        total_docs += 1
        # ----- Derive parent_id (same as original behavior)
        content_hash = rec.get("content_hash")
        if content_hash:
            parent_id = content_hash
        else:
            src_url = rec.get("source_url") or ""
            parent_id = hashlib.sha1(src_url.encode("utf-8", errors="ignore")).hexdigest()

```



```

# ----- Text field
text = (rec.get("text_full") or rec.get("text") or "")
text = text.strip()
if not text:
    continue

# ----- Choose profile (same heuristics)
st = (rec.get("source_type") or "").lower()
fmt = (rec.get("format") or "").lower()
if st == "image":
    profile = "ocr_raw"
elif st in {"pdf_md", "html", "text"}:
    if fmt == "markdown" and looks_structured(text):
        profile = "structured"
    else:
        profile = "structured" if looks_structured(text) else "ocr_raw"
else:
    profile = "ocr_raw"
if profile == "structured":
    size = CHUNK_SIZE_STRUCT
    ov = int(round(size * CHUNK_OVERLAP_STRUCT))
else:
    size = CHUNK_SIZE_OCR
    ov = int(round(size * CHUNK_OVERLAP_OCR))
# ----- Chunking (iterative, same as original)
if count_tokens(text) <= size:
    chunks = [text]
else:
    stack: List[str] = [text]
    for sep in SEPARATORS:
        next_stack: List[str] = []
        for seg in stack:
            if count_tokens(seg) <= size:
                next_stack.append(seg)
                continue
        out = _chunk_once_at_sep(seg, size, ov, sep)

```

```

        if out is None:
            next_stack.append(seg)
        else:
            next_stack.extend(out)
        stack = next_stack
    # finalize: any leftover > size => hard chunk
    final: List[str] = []
    for seg in stack:
        if count_tokens(seg) <= size:
            final.append(seg)
        else:
            final.extend(_hard_chunk(seg, size, ov))
    chunks = final
    # ----- Write out chunks
    total = len(chunks)
    for idx, ch in enumerate(chunks):
        cid = f"{{parent_id}}:{{idx}}"
        if cid in seen_ids:
            # preserve original behavior: skip duplicates silently
            continue
        seen_ids.add(cid)
        out_rec = {
            "id": cid,
            "parent_id": parent_id,
            "source_url": rec.get("source_url"),
            "source_type": rec.get("source_type"),
            "profile": profile,
            "chunk_index": idx,
            "chunk_total": total,
            "n_tokens": count_tokens(ch),
            "text": ch,
            "metadata": None, # unchanged
        }
        fout.write(json.dumps(out_rec, ensure_ascii=False) + "\n")
        total_chunks += 1

```

```

print(f"[DONE] Chunked {total_docs} documents into {total_chunks} chunks → {CHUNKS_JSONL}")

```

ก.3.2 การปรับรูปแบบข้อมูลให้ตรงกับโมเดล

```
# Prepare embeddable chunks (normalize fields → id,text,metadata)

from __future__ import annotations

import os, json, sys, hashlib

IN_JSONL = os.getenv("IN_JSONL", "output/stage/chunks.jsonl")
OUT_JSONL = os.getenv("OUT_JSONL", "output/stage/embeddable.jsonl")
TEXT_KEY = os.getenv("TEXT_KEY", "")
KEEP_META = os.getenv("KEEP_META", "1")
DEDUP = os.getenv("DEDUP", "none")
TOKEN_MODE = os.getenv("TOKEN_MODE", "approx_chars")
CHAR_PER_TOKEN = float(os.getenv("CHAR_PER_TOKEN", "3.6"))
MIN_TOKENS = int(os.getenv("MIN_TOKENS", "0"))
MIN_CHARS = int(os.getenv("MIN_CHARS", "0"))

def approx_tokens(s: str) -> int:
    return int(len(s) / CHAR_PER_TOKEN)

def token_count(s: str) -> int:
    if TOKEN_MODE == "approx_chars":
        return approx_tokens(s)
    else:
        try:
            import tiktoken
            enc = tiktoken.get_encoding("cl100k_base")
            return len(enc.encode(s))
        except Exception:
            return approx_tokens(s)

def choose_text(rec: dict) -> str:
    if TEXT_KEY:
        return rec.get(TEXT_KEY, "")
    for k in ("text", "content", "page_content"):
        v = rec.get(k)
        if isinstance(v, str):
            return v
    return
```

```

with open(IN_JSONL, "r", encoding="utf-8") as fi, \
    open(OUT_JSONL, "w", encoding="utf-8") as fo:
    for line in fi:
        line = line.strip()
        if not line:
            continue
        total += 1
        try:
            rec = json.loads(line)
        except Exception:
            continue

        text = choose_text(rec)
        if not isinstance(text, str) or not text.strip():
            filtered += 1; continue
        if MIN_CHARS and len(text) < MIN_CHARS:
            filtered += 1; continue
        if MIN_TOKENS and token_count(text) < MIN_TOKENS:
            filtered += 1; continue

        if DEDUP == "sha1_text":
            h = hashlib.sha1(text.encode("utf-8")).hexdigest()
            if h in seen:
                deduped += 1; continue
            seen.add(h)

        url = (rec.get("metadata") or {}).get("source_url") or rec.get("source_url")
        out = {
            "id": str(rec.get("id", total)),
            "text": text,
            "metadata": {"url": url} # บังคับให้มีคีย์ url ทุกชิ้น
        }
        fo.write(json.dumps(out, ensure_ascii=False) + "\n")
        kept += 1

    print(f"[DONE] in={total} kept={kept} filtered={filtered} deduped={deduped} -> {OUT_JSONL}")

```

ก.4 การสร้างดัชนี

ก.4.1 การสร้าง vector database

```
#Step 2 — Embed → Chroma (with metadata:url)
# - IN_JSONL: แต่ละบรรทัดต้องมี id, text, metadata.url

import os, json
import chromadb

from chromadb.api.types import EmbeddingFunction, Documents, Embeddings
from sentence_transformers import SentenceTransformer

import numpy as np

IN_JSONL = os.getenv("IN_JSONL", "output/stage/embeddable.jsonl")
CHROMA_PATH = os.getenv("CHROMA_PATH", "chroma")
COLLECTION_NAME = os.getenv("COLLECTION_NAME", "rag_chunks")
EMBEDDING_MODEL = os.getenv("EMBEDDING_MODEL", "BAAI/bge-m3")
HNSW_SPACE = os.getenv("HNSW_SPACE", "cosine")
BATCH_SIZE = int(os.getenv("BATCH_SIZE", "256"))

class BGEM3EmbeddingFunction(EmbeddingFunction):
    def __init__(self, model_name: str):
        self.model = SentenceTransformer(model_name)

    def __call__(self, docs: Documents) -> Embeddings:
        embs = self.model.encode(
            docs, convert_to_numpy=True, normalize_embeddings=True, show_progress_bar=False
        ).astype("float32")
        return [e.tolist() for e in embs]
```

```

def main():
    print(f"[INFO] input={IN_JSONL}")
    print(f"[INFO] chroma={CHROMA_PATH} | collection={COLLECTION_NAME} |
space={HNSW_SPACE}")
    print(f"[INFO] model={EMBEDDING_MODEL} | batch={BATCH_SIZE}")

    ef = BGEM3EmbeddingFunction(EMBEDDING_MODEL)
    client = chromadb.PersistentClient(path=CHROMA_PATH)
    col = client.get_or_create_collection(
        name=COLLECTION_NAME,
        embedding_function=ef,
        metadata={"hnsw:space": HNSW_SPACE},
    )
    ids, docs, metas = [], [], []
    added = 0 # ← ประกาศตัวนับก่อนใช้
    with open(IN_JSONL, "r", encoding="utf-8") as f:
        for line in f:
            rec = json.loads(line)
            ids.append(str(rec["id"]))
            docs.append(rec["text"])
            metas.append(rec["metadata"]) # ต้องมี {"url": ...}

            if len(ids) >= BATCH_SIZE:
                n = len(ids)
                col.add(ids=ids, documents=docs, metadatas=metas)
                added += n
                print(f"[ADD] +{n} (total={added})")
                ids, docs, metas = [], [], []

    if ids:
        n = len(ids)
        col.add(ids=ids, documents=docs, metadatas=metas)
        added += n
        print(f"[ADD] +{n} (total={added})")

    print(f"[DONE] collection.count() = {col.count()}")

```

ก.4.2 การลบข้อมูลออกจากดัชนี

```
url_to_delete = "https://www.it.chula.ac.th/wp-content/uploads/2020/01/Path_Academic_2551-2555.pdf"
```

```
# ลองดึง id มาก่อนเพื่อนับ
```

```
hits = col.get(where={"url": url_to_delete}, include=[]) # ขอเฉพาะ id
```

```
ids = hits.get("ids", [])
```

```
if not ids:
```

```
    print("no url match")
```

```
else:
```

```
    col.delete(where={"url": url_to_delete})
```

```
    print(f"deleted {len(ids)} vectors with url = {url_to_delete}")
```



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

ก.5 การดึงข้อมูลเพื่อใช้ตอบคำถาม

ก.5.1 การตั้งค่า config

```
OPENAI_API_KEY = ""

from __future__ import annotations
import os
import chromadb

from chromadb.api.types import EmbeddingFunction, Documents, Embeddings
from sentence_transformers import SentenceTransformer
from FlagEmbedding import FlagReranker
from openai import OpenAI
import numpy as np

# Define the embedding function class (reuse from cell 5)
class BGEM3EmbeddingFunction(EmbeddingFunction):
    def __init__(self, model_name: str):
        self.model = SentenceTransformer(model_name)

    def __call__(self, docs: Documents) -> Embeddings:
        embs = self.model.encode(
            docs, convert_to_numpy=True, normalize_embeddings=True, show_progress_bar=False
        ).astype("float32")
        return [e.tolist() for e in embs]

# ----- Config (reuse same ENV names as earlier) -----
CHROMA_PATH = os.getenv("CHROMA_PATH", "chroma") # Match the default from cell 5
COLLECTION_NAME = os.getenv("COLLECTION_NAME", "rag_chunks")
EMBEDDING_MODEL = os.getenv("EMBEDDING_MODEL", "BAAI/bge-m3")
HNSW_SPACE = os.getenv("HNSW_SPACE", "cosine")
TOP_K = int(os.getenv("TOP_K", "5")) # final K for plain, or for display

# Rerank knobs
RETRIEVAL_MODE = os.getenv("RETRIEVAL_MODE", "auto") # "plain" | "rerank" | "auto"
INITIAL_K = int(os.getenv("INITIAL_K", "50")) # used if rerank
FINAL_K = int(os.getenv("FINAL_K", "10")) # top-K after rerank
RERANKER_MODEL = os.getenv("RERANKER_MODEL", "BAAI/bge-reranker-v2-m3")
```



```
# LLM
OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-5-mini")
client_llm = OpenAI(api_key=OPENAI_API_KEY) # Use the variable directly

# ----- Open collection (same embedding fn as ingest) -----
ef = BGEM3EmbeddingFunction(EMBEDDING_MODEL)
client = chromadb.PersistentClient(path=CHROMA_PATH)
col = client.get_or_create_collection(
    name=COLLECTION_NAME,
    embedding_function=ef,
    metadata={"hnsw:space": HNSW_SPACE},
)
```

ก.5.2 Retrieval แบบ no rerank

```
# Retrieval (no rerank)
# Use the same test query as in the next cell
query = "ขั้นตอนลงทะเลเป็นเรียน จุฬาลงกรณ์ฯ ทำอย่างไร"
print(f"[QUERY] {query}")

# --- plain retrieval (show scores) ---
res = col.query(query_texts=[query], n_results=TOP_K,
include=["documents", "metadatas", "distances"])
ids = res["ids"][0]
docs = res["documents"][0]
metas = res["metadatas"][0]
dists = res["distances"][0]

for i, (doc_id, doc, meta, dist) in enumerate(zip(ids, docs, metas, dists), 1):
    cos_sim = 1.0 - dist
    score01 = (cos_sim + 1.0) / 2.0
    print(f"\n#{i} id={doc_id} score={score01:.4f} url={meta['url']}")
    print(doc)

contexts = [f"URL: {m['url']}\n{d}" for m, d in zip(metas, docs)]
```

5.2 Retrieval แบบ rerank

```
# --- Rerank path (with URL) -----
query = "ขั้นตอนลงทะเบียนเรียน จุฬาฯ ทำอย่างไร"
res = col.query(query_texts=[query], n_results=INITIAL_K, include=["documents", "metadatas"])
ids = res["ids"][0]
docs = res["documents"][0]
metas = res["metadatas"][0] # ต้องมี {"url": ...} จากขั้น embed
print(f"[QUERY] {query}")

reranker = FlagReranker(RERANKER_MODEL, use_fp16=True)
scores = reranker.compute_score([[query, d] for d in docs], normalize=True)

# sort ตามคะแนน reranker แล้วตัดเหลือ FINAL_K
ranked = sorted(zip(ids, docs, metas, scores), key=lambda x: x[3], reverse=True)[:FINAL_K]

for i, (doc_id, doc_text, meta, score) in enumerate(ranked, start=1):
    url = meta["url"] # ใช้คีย์ 'url' ตามที่เตรียมไว้ใน metadata
    preview = (doc_text[:280] + "...") if len(doc_text) > 280 else doc_text
    print(f"\n#{i} id={doc_id} rerank_score={score:.4f} url={url}")
    print(preview)

# contexts สำหรับส่งเข้า LLM: แแนบ URL ไว้กับแต่ละชิ้น
contexts = [f"URL: {meta['url']}\n{doc}" for (_, doc, meta, _) in ranked]
```

BUSINESS SCHOOL

FLAGSHIP FOR LIFE

5.4 การตอบคำถาม

```
contexts = [f"URL: {meta['url']}\n{doc}" for (_, doc, meta, _) in ranked]
context_blob = "\n\n---\n\n".join(contexts)
prompt = (
    "คุณเป็นผู้ช่วยตอบคำถามจากเอกสาร (RAG). "
    "ตอบโดยอ้างอิงเฉพาะจาก Context ด้านล่างเท่านั้น หากไม่พบข้อมูลให้บอกว่าไม่พบ.\n\n"
    f"Context:\n{context_blob}\n\n"
    f"คำถาม: {query}\n\n"
    "รูปแบบคำตอบ: กระชับ ชัดเจน (ตอบตามภาษาที่ใช้ถาม)ให้อ้างอิง URL ที่นำมาตอบด้วย"
)
resp = client_llm.chat.completions.create(
    model=OPENAI_MODEL,
    messages=[{"role": "user", "content": prompt}],
)
print("\n[LLM ANSWER]\n" + resp.choices[0].message.content)
```



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE

ภาคผนวก ข

ผลลัพธ์การทำงาน

ข.1 ผลลัพธ์การดึงข้อมูล

ข.1.1 ตัวอย่างผลลัพธ์การดึงข้อมูล html

[html] total=1087 sample=5

1. id=a11ed74474ef93a7bc8790ab90da392a278c5134

url:

https://www.sa.chula.ac.th/act_gallery/%E0%B8%9E%E0%B8%B4%E0%B8%98%E0%B8%B5%E0%B8%9B%E0%B8%90%E0%B8%A1%E0%B8%99%E0%B8%B4%E0%B9%80%E0%B8%97%E0%B8%A8%E0%B8%99%E0%B8%B4%E0%B8%AA%E0%B8%B4%E0%B8%95%E0%B9%83%E0%B8%AB%E0%B8%A1%E0%B9%88%E0%B9%81-2/

title: พิธีปฐมนิเทศนิสิตใหม่และพิธีถวายสัตย์ปฏิญาณตนเป็นนิสิตจุฬาลงกรณ์มหาวิทยาลัยสำหรับนิสิตใหม่ระดับปริญญาตรี ประจำปีการศึกษา 2560 วันที่ 3 สิงหาคม 2560

text: พิธีปฐมนิเทศนิสิตใหม่และพิธีถวายสัตย์ปฏิญาณตนเป็นนิสิตจุฬาลงกรณ์มหาวิทยาลัยสำหรับนิสิตใหม่ระดับปริญญาตรี ประจำปีการศึกษา 2560 วันที่ 3 สิงหาคม 2560 – สำนักบริหารกิจการนิสิต ชั้น 2 อาคาร จุลจักรพงษ์ จุฬาลงกรณ์มหาวิทยาลัย ถนนพญาไท แขวงวังใหม่ เขตปทุมวัน กรุงเทพฯ 10330 โทรศัพท์: 02-218-7052 โทรสาร: 02-218-7053 หน้าแรก เกี่ยวกับสำนัก รู้จักสำนักบริหารกิจการนิสิต ฝ่ายพัฒนานิสิต ฝ่ายทุนการศึกษาและสวัสดิการนิสิต ฝ่ายส่งเสริมสุขภาพ
นิตินิต ฝ่ายประสานงานและเครือข่ายกิจการนิสิต นโยบาย วิสัยทัศน์ พันธกิจ ผู้บริหาร เจ้าหน้าที่ e.chula.ac.th/ แล้วมาดูกันว่าพี่เจอรี่ใหม่จะช่วยให้ชีวิตคุณ “ง่ายขึ้น” ได้แค่ไหน!

2. id=b1d4607251e7815c60779ee9a16d98d0044e7c79

url: https://www.it.chula.ac.th/asked_questions_category/application/

title: Application Archives - IT Chulalongkorn University

text: Application Archives - IT Chulalongkorn University หน้าแรก บริการสำนักฯ ข้อมูลสำนักฯ วิสัยทัศน์ และพันธกิจ ผู้บริหารและบุคลากร รายงานประจำปี แผนยุทธศาสตร์ฯ ตราสัญลักษณ์สำนักฯ ความมั่นคงปลอดภัยทางไซเบอร์ นโยบายคุ้มครองข้อมูลส่วนบุคคล ติดต่อเรา ประกาศและข่าวสาร ข่าวสารสำนักฯ ประกาศสำนักฯ ประกาศจัดซื้อจัดจ้าง ข่าวทั่วไป ข่าวรับสมัครงาน แจ้งปัญหาและขอใช้บริการ แจ้งปัญหา คู่มือการแจ้งปัญหา ขอใช้บริการ คู่มือการสร้างคำร้องขอใช้บริการ คำถามที่พบบ่อย สารแนะนำรู้ด้านไอที ความปลอดภัยด้านไอที บทความด้านไอที

3. id=fc16d24dff54e0cd890019a356c6f1324e64e470

url: <https://www.it.chula.ac.th/chulagenie-upgrade/>

title: ChulaGENIE เพิ่มความสามารถใหม่ - IT Chulalongkorn University

text: ChulaGENIE เพิ่มความสามารถใหม่ ที่จะช่วยให้ชีวิตคุณง่ายขึ้น...กว่าเดิม เพราะเราอยากให้คุณการใช้งานของคุณ ง่ายขึ้น รวดเร็วขึ้น และตอบโต้มากขึ้น ตอนนี้ ChulaGENIE ได้อัปเดตฟีเจอร์ใหม่แบบจัดเต็ม! ใน ใช้งานสะดวกกว่าเดิม รองรับความต้องการที่หลากหลาย ช่วยให้การเรียน การสอน การทำวิจัย และการทำงานมีประสิทธิภาพยิ่งขึ้น มาค้นพบความสามารถใหม่ ๆ ของ ChulaGENIE ที่ ออกแบบมาให้คุณ ลองใช้เลยตอนนี้ที่ <https://genie.e.chula.ac.th/> แล้วมาดูกันว่า ฟีเจอร์ใหม่จะช่วยให้ชีวิตคุณ “ง่ายขึ้น” ได้แค่ไหน!

4. id=500f9a629de9d6e70622ff7c26674ca3ccdf7ae4

url: <https://www.reg.chula.ac.th/document/courselist/68/39miscu-0109-CRpt.txt>

text: 03/09/2568 สำนักงานการทะเบียน จุฬาลงกรณ์มหาวิทยาลัย หน้า 1 09:51:25 ทะเบียนรายวิชาที่เปิด (ระดับปริญญาบัณฑิตและระดับบัณฑิตศึกษา) คณะวิทยาศาสตร์การกีฬา ไม่สังกัดภาควิชา/เทียบเท่า -----

----- ลำดับที่ รหัส ชื่อรายวิชา หน่วยกิต วิธีการสอน ชม/สัปดาห์ วิธีการวัดผล ประเภท
ภาค การศึกษา/ปีการศึกษา รายวิชา บรรยาย ไม่บรรยาย รายวิชา เปิดรายวิชา ปิดรายวิชา -----

5. id=53a88c520b9bae0193ab4d356b894b4bf2be120b

url:

<https://www.sa.chula.ac.th/announcement/%E0%B8%9B%E0%B8%A3%E0%B8%B0%E0%B8%81%E0%B8%B2%E0%B8%A8%E0%B8%9C%E0%B8%A5-%E0%B8%82%E0%B8%AD%E0%B9%81%E0%B8%AA%E0%B8%94%E0%B8%87%E0%B8%84%E0%B8%A7%E0%B8%B2%E0%B8%A1%E0%B8%A2%E0%B8%B4%E0%B8%99%E0%B8%94/>

title: ประกาศผล ขอแสดงความยินดีกับผู้ชนะการประกวดคลิปสั้น “สื่อ สาน ใจ” ทุกท่าน

text: **ประกาศผล ขอแสดงความยินดีกับ ผู้ชนะการประกวดคลิปสั้น “สื่อ สาน ใจ” ทุกท่าน** ขอขอบคุณทุกเรื่องราว “กระทบใจ” ทั้งใจฟูและฝั่งใจที่ร่วมส่งเข้าประกวด เพราะทุกคลิปคือพลังที่สะท้อนถึงความสำคัญของการสื่อสาร ที่จะช่วยสร้างความเข้าใจในเรื่องสุขภาพจิตให้ลึกซึ้งยิ่งขึ้น * รางวัลชนะเลิศ * > Track โหมเมนต์ที่ทำให้ใจฟู: **วันนี้ที่อบอุ่น** > Track โหมเมนต์ที่ยังฝั่งใจ: **Shutter** * รางวัลรองชนะเลิศ อันดับ 1 * > Tra ack โหมเมนต์ที่ทำให้ใจฟู: **ห้องใจปลอดภัย** > Track โหมเมนต์ที่ยังฝั่งใจ: **TP Production*

ข.1.2 ตัวอย่างผลลัพธ์การดึงข้อมูล html

[image] total=763 sample=5

1. id=3a769de213a92afb93ca23ee679764dcc4488bf7

url: <https://www.it.chula.ac.th/wp-content/uploads/2022/06/cer02.jpg>

alt: " is_infographic=True

text: จุฬาลงกรณ์มหาวิทยาลัย ขอขอบประกาศเกียรติคุณฉบับนี้ เพื่อแสดงว่า สำนักบริหารเทคโนโลยีสารสนเทศ ได้รับรางวัลยกย่องเชิดชูเกียรติบุคลากรสายปฏิบัติการ (คนดี จ) กลุ่มบุคลากรยอดเยี่ยมขนาดใหญ่ กลุ่มคณะทำงาน "ศูนย์บริการวัคซีน ุฬาลงกรณ์มหาวิทยาลัย ให้ไว้ ณ วันที่ ๓๑ เดือน พฤษภาคม พุทธศักราช ๒๕๖๕ >a7e av. อธิการบดี ประธานคณะกรรมการสรรหาบุคลากรดีเด่น ศรีจุฬาฯ

2. id=e664267fc5bcc79779efd6b300a77c335cba9c5d

url: <https://www.it.chula.ac.th/wp-content/uploads/2020/01/Capturedns3.jpg>

alt: " is_infographic=True

text: rootkali virtualbox dig 0161 .200 _ www .kapook .com c di6 9.9.5-12.1 debian a 0161 .200 www .kapook coil (1 serter found) gtobal options: +cid got answen: >>header<<- opcode.: ouery status: refused id: 13013 flaas: rd.: query: l. allswer: n. authority: 0. additional: warnii6: recursion requested but not available opt pseudosection: edns: version: 0. flags: ; udp: 4096 ouestion sectionw: พืฟพ kaapook com iii a query time 36 isec server: 161 200 #53(161 .200 when: wed llov 16 11 :

3. id=2b5f614e8de7d95be102f9846cecf1a4bbbca8c5

url: https://www.sa.chula.ac.th/wp-content/uploads/2020/09/BB-Hackathon-News_001.jpg

alt: " is_infographic=True

text: chulalongkorn university 24 hour beyond biodiversity sdgs hackathon office of student affairs, chulalongkorn university incorporation with faculty of architecture, rom chulalongkorn university., waseda university and aeon foundation organized a 24- beyond biodiversity sdgs hackathon during feb 3'd 4th 2020. beyond biodiversity bb) symposium., beyond biodiversity sdgs hackathon which is sponsored by aeon foundation and has micro finance and ecosystem for the solution of environmental crisi

4. id=c98444486a20b274b6146f50cc9247b1d9e445b4

url: <https://www.it.chula.ac.th/wp-content/uploads/2020/09/t1.4.png>

alt: " is_infographic=True

text: welcome to backup and sync (step 2 of 3) x sign in my computer choose folders to continuously back up to google drive my computer desktop 3 kb documents 15 mb google drive pictures 502 kb choose folder backing up all files and folders change learn more about photo and video uploads photo and video upload size learn more high quality (free unlimited storage) great visual quality at reduced file size original quality (3.3 gb storage left) full resolution that counts against your duo

5. id=8ce33e3ef109dd26259e4f72d2e168846992c24b

url: https://www.sa.chula.ac.th/wp-content/uploads/2025/04/o10_Chula-E-Service_CUSW-Inhouse_page-0001.jpg

alt: " is_infographic=True

text: ระบบบริการที่มีประสิทธิภาพมีความสำคัญ chulalongkorn chula university chula student ๑ wellness ช่วยให้เรามีข้อมูลครอบคลุมในการดูแลจิตใจ target group goal action for all sustained good mental health lth promote accessibility and positive self cares promote accessibility and fostering self awareness and self care practices all students (40,000 develop self help tools and direct channel (line @cusw) students) promoted personal growth promote communication that supports and cares for mental

ข.1.3 ตัวอย่างผลลัพธ์การดึงข้อมูล pdf

[pdf_md] total=623 sample=5

1. id=7811435d0710caff60031718cbf01fbce9954420

url: https://www.reg.chula.ac.th/FacultyScheduleCeremony_2024_Eng.pdf

has_tables=True

text: ## **Faculty Schedule for Graduation Convocation Ceremony** **Academic Year 2024** #
Chulalongkorn Auditorium |September 29, 2025|Col2|Col3| ---|---|---| |**Morning
Session**|**Afternoon Session**|**1. Faculty of Commerce and
Accountancy**
(Undergraduate Level)|**1. Faculty of Commerce and
Accountancy**
(Undergraduate Level)|**1. Honorary Doctoral Degree**|**1. Faculty of
Commerce and Accountancy**
(Undergraduate Level)|**1. Faculty of Commerce and
Accountancy**

2. id=187b1cd30e7fa1dc5c7ddd5aae17bd586329ab0a

url: <https://web.reg.chula.ac.th/LEAVE.pdf>

has_tables=False

text: **Page | 1** # **การลาพักการศึกษา** นิสิตที่มีความจำเป็นต้องลาพักการศึกษาให้ศึกษาข้อบังคับจุฬาลงกรณ์มหาวิทยาลัย ว่าด้วยการศึกษาชั้นปริญญาตรี พ.ศ. 2556 ข้อ 14 และ พ.ศ. 2565 หมวด 7 หรือข้อบังคับ บจุฬาลงกรณ์มหาวิทยาลัยว่าด้วย การศึกษาในระดับบัณฑิตศึกษา พ.ศ. 2561 หมวด 9 และให้ปฏิบัติดังนี้ 1. นิสิตดำเนินการท วร้อง **จท49** (ค วร้องขอลาพักการศึกษา) ผ่าน google form จากเว็บไซต์ของ ส านักงานการทะเบียน ยน หัวข้อแบบฟอร์มต่างๆ ของส านักงานการทะเบียน / Forms 2. ยื่น **จท49** พร้อมหลักฐานประกอบ

3. id=845e52696cd524ab09e6bd9216c51ff0c5df8c19

url: <https://www.sa.chula.ac.th/wp-content/uploads/2018/10/9.%E0%B8%9A%E0%B8%B1%E0%B8%99%E0%B8%97%E0%B8%B6%E0%B8%81%E0%B8%84%E0%B8%93%E0%B8%B0%E0%B9%81%E0%B8%9E%E0%B8%97%E0%B8%A2%E0%B9%8C.pdf>

has_tables=False

text: # **บันทึกข้อความ** **ส่วนงาน** ส านักบริหารกิจการนิสิต อำนาจจุฬจักรพงษ์ ชั้น ๒ โทร ๘๗๐๕๒ ต่อ ๓๐๒ **ที่** วันที่ **เรื่อง** การจัดสรรงบประมาณเพื่อสนับสนุนกิจกรรมระดับบัณฑิตศึกษา -----
เรียน คณะบดีคณะแพทยศาสตร์ ด้วยมหาวิทยาลัยได้จัดสรรงบประมาณเพื่อสนับสนุนกิจกรรมระดับบัณฑิตศึกษาในปีงบประมาณ ๒ ๕๖๕ ซึ่งส านักบริหารกิจการนิสิตได้น ำเข้าที่ประชุมคณะกรรมการประสานงานกิจการนิสิตครั้งที่

4. id=7f9caf7d662641ccbd0fae7a5f8b5c395ce74c63

url: https://www.reg.chula.ac.th/en/wp-content/uploads/sites/8/2025/07/CR90_Tri2025_Eng.pdf

has_tables=True

text: # **CR90** ## **Calendar** ### **Academic Year 2025** ##### First Semester Second Semester Third Semester **For Programs using the calendar of Trimester System**
|Faculty|Program|Major| ---|---|---| |**Graduate School**|Doctor of Philosophy|- European Studies|
|**Science**|Master of Science|- Energy Geosciences| |**Political Science**|Master of Arts|-
International Development Studies| |Doctor of Philosophy|- International Development Studies|
|**Commerce and Accountancy**|Master of Scie

5. id=826aab39b43ec63cc525939688eb1e166c0b86b2

url: <https://www.it.chula.ac.th/wp-content/uploads/2020/01/AnyconnectWin8-2.pdf>

has_tables=False

text: คู่มือการติดตั้ง Cisco AnyConnect VPN Client สำหรับระบบปฏิบัติการ Windows 8 1. เปิด Web Browser แล้วเข้าไปที่ URL: <https://www.it.chula.ac.th> 2. คลิกเลือกเมนู "บริการสำนักฯ" แล้วเลือกบริการ "โปรแกรม VPN" คู่มือการติดตั้ง Cisco AnyConnect VPN Client สำหรับระบบปฏิบัติการ Windows 8 3. คลิกลิงก์ "Download for Windows" 4. ป้อน Username และ Password แล้วคลิกเลือกปุ่ม "Ok" เพื่อทำการล็อกอินเข้าดาวน์โหลดซอฟต์แวร์คู่มือการติดตั้ง Cisco AnyConnect VPN Client สำหรับระบบปฏิบัติการ Windows

ข.1.4 ตัวอย่างข้อมูลที่มีความเสียหาย

output/stage/documents.jsonl total=2473

output/stage/chunks.jsonl total=108893

pdf_md has-□ = 766 (0.70%)

html has-□ = 8998 (8.26%)

image has-□ = 24 (0.02%)

Samples of pdf_md with □:

1. id=d2382c31fe4c0b269085641ed53554431c9a2562:4 url=<https://www.it.chula.ac.th/wp-content/uploads/2020/01/Example.pdf>

□อนไปดูประสบการณ์□ในอดีตมากนัก ฯลฯ ดังนั้น□น□น□กัน□าเชื่อได้□ว□าปรากฏการณ์□
วงจรอุบาทว์□ทางการเมืองจะยังอยู่□□กับสังคมเรา

2. id=d2382c31fe4c0b269085641ed53554431c9a2562:9 url=<https://www.it.chula.ac.th/wp-content/uploads/2020/01/Example.pdf>

□□ไปพร□อมๆ กับตัวแสดงทางการเมืองอื่นๆ พร□อมทั้งปรับ ประเด็นการวิจัย กระบวนการและ
กิจกรรมการวิจัยไปตามสถานการณ์□ทางการเมือง

3. id=c32940f61f249983aeee6ce8fa61b1f359f839b6:15 url=<https://www.it.chula.ac.th/wp-content/uploads/2020/01/cyberlaw.pdf>

□หรือ บุตรของผู้เสียหายร้องทุกข์ได้ และให้ถือว่าเป็นผู้เสียหาย จล มาตรา ๑๗ ผู้ใดกระทำความผิดตาม
พระราชบัญญัตินี้นอกราชอาณาจักร

4. id=c32940f61f249983aeee6ce8fa61b1f359f839b6:19 url=<https://www.it.chula.ac.th/wp-content/uploads/2020/01/cyberlaw.pdf>

□ี่ เลม ๑๒๔ ตอนที่ ๒๗ ก. ราชกิจจานุเบกษา มิถุนายน ๒๕๕๐ (๗ ถอดรหัสลับของข้อมูลคอมพิวเตอร์
ของบุคคลใด หรือสั่งให้บุคคลที่เก

5. id=c32940f61f249983aeee6ce8fa61b1f359f839b6:25 url=<https://www.it.chula.ac.th/wp->

ข.1.5 ผลลัพธ์การทำ chunking

```
[INFO] Reading: output/stage/documents.jsonl
[INFO] Writing: output/stage/chunks.jsonl
[INFO] Profiles: structured(size=512, overlap=0.12), ocr_raw(size=256, overlap=0.2)
[DONE] Chunked 2473 documents into 108893 chunks → output/stage/chunks.jsonl
```

ข.2 ตัวอย่างการตอบคำถาม

ข.2.1 ผลการทำงานจาก Retrieval node

no rerank

[QUERY] ขึ้นตอนลงทะเบียนเรียน จุฬาลงกรณ์มหาวิทยาลัย อย่างไร

#1 id=69b323cd66c7ef85edf091e2dcfad839a155daf9:23 score=0.8810

url=<https://www.reg.chula.ac.th/admissions/>

ิสิตรจุฬาลงกรณ์มหาวิทยาลัย และทำบัตรประจำตัวนิสิต ที่ระบบลงทะเบียนแรกเข้า

- ติดตามกำหนดการ และอ่านรายละเอียดเพิ่มเติมเกี่ยวกับเอกสารที่ต้องใช้ ที่เว็บไซต์

****สำนักงานการทะเบียน****

****5. ลงทะเบียนเรียน****- ลงทะเบียนเรียนรายวิชาตามที่หลักสูตรกำหนดผ่านเว็บไซต์

****สำนักงานการทะเบียน**** - ติดตามกำหนดการการลงทะเบียนเรียนของนิสิตใหม่ได้จากเว็บไซต์

****สำนักงานการทะเบียน****

****6. ชำระค่าเล่าเรียน****

- ชำระค่าเล่าเรียนผ่าน Application CU-NEX ภายในช่วงเวลาที่กำหนด (อ่านวิธีชำระค่าเล่าเรียน:

****คลิก****) - ติดตามกำหนดการชำระเงินค่าเล่าเรียนได้จากเว็บไซต์

****สำนักงานการทะเบียน****

รวบรวมเนื้อหาโดย: ****สำนักงานการทะเบียน****

#2 id=09c20a559a141d8f86803b0debe069283c022bd4:22 score=0.8810

url=<https://www.reg.chula.ac.th/admissions/>

ิสิตรจุฬาลงกรณ์มหาวิทยาลัย และทำบัตรประจำตัวนิสิต ที่ระบบลงทะเบียนแรกเข้า

- ติดตามกำหนดการ และอ่านรายละเอียดเพิ่มเติมเกี่ยวกับเอกสารที่ต้องใช้ ที่เว็บไซต์

****สำนักงานการทะเบียน****

with rerank

[QUERY] ขั้นตอนลงทะเบียนเรียน จุฬาลงกรณ์มหาวิทยาลัย

#1 id=e320441db30bba64db0390ca44825f06add115b9:80 rerank_score=0.9964

url=https://web.reg.chula.ac.th/CR99_2566_T_S.pdf

☐ ยานในหอสมุดแรกของภาคการศึกษา หรือภายในสองสัปดาห์แรกของภาค

ฤดูร้อน โดยด าเนินการที่เว็บไซต์ www.reg.chula.ac.th (นิสิตสามารถศึกษารายละเอียดจากคู่มือการลงทะเบียนเรียน

ที่ปรากฏในเว็บไซต์ของส านักงานการทะเบียน) ตามขั้นตอนต่อไปนี้

1. เข้าสู่ระบบการลงทะเบียนเรียน โดยเลือกรายการ **เข...
- 2.

#2 id=861da5daa97094d2bc666d94501cda7d5c05d7a8:62 rerank_score=0.9907

url=https://www.reg.chula.ac.th/CR99_2025_Th.pdf

ทะเบียนเรียนปกติ

1. การลงทะเบียนเรียนปกติในภาคการศึกษาแรกสำหรับนิสิตใหม่

งานขอลงทะเบียนเรียนผ่านทางอินเทอร์เน็ตที่เว็บไซต์ www.reg.chula.ac.th (นิสิตสามารถศึกษาขั้นตอนจา...ดงความ

#3 id=861da5daa97094d2bc666d94501cda7d5c05d7a8:78 rerank_score=0.9871

url=https://www.reg.chula.ac.th/CR99_2025_Th.pdf

ไม่ได้แสดงความจำนงขอลงทะเบียนเรียนตามช่วงเวลาที่กำหนด ต้องลงทะเบียนเรียนสาย (พร้อมชำระ
ร้อนทางการได้ภายในสัปดาห์แรกของภาคฤดูร้อน โดยลงทะเบียนเรียนที่ เว็บไซต์ www.reg.chula.ac.th
(นิสิต...

#4 id=e320441db30bba64db0390ca44825f06add115b9:16 rerank_score=0.9807

url=https://web.reg.chula.ac.th/CR99_2566_T_S.pdf

ฉบับบัณฑิตศึกษา พ.ศ. 2561

5. ก าหนดการลงทะเบียนเรียน (ติดตามในเว็บไซต์ส านักงานการทะเบียน www.reg.chula.ac.th)

การลงทะเบียนแรกเข้าออนไลน์

การลงทะเบียนแรกเข้าออนไลน์...

#5 id=69b323cd66c7ef85edf091e2dcfad839a155daf9:22 rerank_score=0.9701

url=https://www.reg.chula.ac.th/admissions/

การทะเบียน**

3. รับรหัสผ่าน CU-NET

- รับรหัสผ่าน CU-NET จาก

ข.2.2 การตอบคำถามทั่วไป

[QUERY] ขั้นตอนลงทะเบียนเรียน จุฬาฯ ทำอย่างไร

[LLM ANSWER]

สรุปขั้นตอนการลงทะเบียนเรียน (อ้างอิงจากเอกสารราชการของสำนักทะเบียน จุฬาฯ):

สำหรับนิสิตใหม่ (ลงทะเบียนแรกเข้า)

- รับรหัสผ่าน CU-NET จากสำนักบริหารเทคโนโลยีสารสนเทศ ก่อนใช้งานระบบลงทะเบียน (เพื่อใช้บริการต่างๆ ของมหาวิทยาลัย) <https://www.reg.chula.ac.th/admissions/>

- ทำการลงทะเบียนแรกเข้าออนไลน์ที่ <https://adm.reg.chula.ac.th> : กรอกแบบสำรวจ/ประวัติ (จท19, จท20) และอัปโหลดเอกสาร/รูปถ่าย ตามที่ระบบกำหนด https://web.reg.chula.ac.th/CR99_2566_T_S.pdf และ

<https://www.reg.chula.ac.th/admissions/>

การลงทะเบียนเรียนปกติ (ผ่านระบบออนไลน์)

1. เข้าเว็บไซต์สำนักทะเบียน <https://www.reg.chula.ac.th> แล้วเลือกเมนู เข้าสู่ระบบ / ลงทะเบียนเรียน / ข้อมูลส่วนบุคคล เพื่อเข้าสู่ระบบลงทะเบียน https://web.reg.chula.ac.th/document/student_doc1.pdf

2. ป้อนเลขประจำตัวนิสิต, รหัสผ่าน (และกรอก Captcha หากระบบขอ) แล้วกดตกลง

https://web.reg.chula.ac.th/document/student_doc1.pdf และ https://www.reg.chula.ac.th/CR99_2025_Th.pdf

3. เลือกเมนู “ลงทะเบียนเรียน” แล้วเลือกรูปแบบการลงทะเบียน เช่น ลงปกติ / เปลี่ยนตอนเรียน / ลดรายวิชา จากนั้นบันทึกรายวิชาและตอนเรียน https://web.reg.chula.ac.th/CR99_2566_T_S.pdf

4. บันทึกการลงทะเบียนเรียน (และสำหรับการลดรายวิชา ต้องกดบันทึกและยืนยัน)

https://web.reg.chula.ac.th/CR99_2566_T_S.pdf

5. ตรวจสอบผลการลงทะเบียนได้ที่เมนู สอบถามข้อมูลส่วนบุคคล และชำระค่าเล่าเรียนผ่าน CU NEX ตามกำหนดในปฏิทินการศึกษา https://web.reg.chula.ac.th/document/student_doc1.pdf

กรณีลงทะเบียนสาย

- หากไม่ได้แสดงความจำนงตามเวลาที่กำหนด ต้องลงทะเบียนสายผ่านอินเทอร์เน็ตภายในสองสัปดาห์แรกของภาคการศึกษา (ภาคฤดูร้อนภายในสัปดาห์แรก) และอาจมีค่าปรับตามประกาศ

https://www.reg.chula.ac.th/CR99_2025_Th.pdf

(อ้างอิงเอกสาร: https://web.reg.chula.ac.th/CR99_2566_T_S.pdf ,

ข.2.3 การให้คำแนะนำ

[QUERY] ขั้นตอนถอนรายวิชาต้องทำอะไร

You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `\_\_call\_\_` method is faster than using a method to encode the text followed by a call to the `pad` method to get a padded encoding.

[LLM ANSWER]

สรุปขั้นตอนการขอถอนรายวิชา (จากเอกสารของสำนักทะเบียน)

1. ตรวจสอบช่วงเวลาที่สามารถถอนได้ (สัปดาห์ที่ 7-12 ของภาคการศึกษา หรือ สัปดาห์ที่ 3-4 ของภาคฤดูร้อน)
(web.reg.chula.ac.th/CR99_2566_T_S.pdf, www.reg.chula.ac.th/CR99_2025_Th.pdf)
2. เข้าสู่ระบบลงทะเบียนที่ www.reg.chula.ac.th (ใส่รหัสนิสิต, รหัสผ่าน, captcha แล้วกด “ตกลง”)
(web.reg.chula.ac.th/CR99_2566_T_S.pdf, www.reg.chula.ac.th/CR99_2025_Th.pdf)
3. เลือกเมนู “แสดงความจำนงขอถอนรายวิชา” → ตี๊กเลือกรายวิชาที่จะถอน → กด “save” → กด “Print CR48” และกด “Print” เพื่อพิมพ์แบบฟอร์ม จท48 (CR48) (web.reg.chula.ac.th/CR99_2566_T_S.pdf, www.reg.chula.ac.th/CR99_2025_Th.pdf, web.reg.chula.ac.th/document/registrationmanual_TH_240704.pdf)
4. กรอกคำร้อง จท48 (ระบุเหตุผลการขอถอน) (web.reg.chula.ac.th/CR99_2566_T_S.pdf, www.reg.chula.ac.th/CR99_2025_Th.pdf)
5. ขอความเห็นผู้สอน/ผู้รับผิดชอบรายวิชา (และสำหรับนิสิตระดับปริญญาตรี ให้ขอความเห็นอาจารย์ที่ปรึกษา)
(web.reg.chula.ac.th/CR99_2566_T_S.pdf, web.reg.chula.ac.th/document/registrationmanual_TH_240704.pdf)
6. ยื่นคำร้องและเอกสารที่ทะเบียนคณะ เพื่อให้นายทะเบียนคณะเสนอคณบดีพิจารณา/ลงนาม
(web.reg.chula.ac.th/CR99_2566_T_S.pdf, web.reg.chula.ac.th/document/registrationmanual_TH_240704.pdf)
7. ตรวจสอบผลการขอถอน (สถานะ “W” หากอนุมัติ) ผ่านเมนูสอบถามข้อมูลส่วนบุคคล → “สอบถามสถานะขอถอนรายวิชา” ที่ www.reg.chula.ac.th (web.reg.chula.ac.th/CR99_2566_T_S.pdf, www.reg.chula.ac.th/CR99_2025_Th.pdf)

อ้างอิง:

- https://web.reg.chula.ac.th/CR99_2566_T_S.pdf
- https://www.reg.chula.ac.th/CR99_2025_Th.pdf
- https://web.reg.chula.ac.th/document/registrationmanual_TH_240704.pdf

(หากต้องการรายละเอียดการกรอกแบบฟอร์มหรือเอกสารประกอบเพิ่มเติม ให้แจ้งได้)

ข.2.4 การตอบคำถามภาษาอังกฤษ

[QUERY] how to withdraw course?

You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `__call__` method is faster than using a method to encode the text followed by a call to the `pad` method to get a padded encoding.

[LLM ANSWER]

Here's how to withdraw a course at Chulalongkorn University:

When you can withdraw

- Normal period: Week 7–12 of the semester, or Week 3–4 of the summer session (W will show on transcript) (https://www.reg.chula.ac.th/document/registrationmanual_ENG_240704.pdf, https://web.reg.chula.ac.th/CR99_2023_EngS.pdf)

Steps (normal withdrawal)

1) Log in at www.reg.chula.ac.th > Request for Course Withdrawal. Select the course and click “Save” (cannot cancel after Save). Click “Print CR48” and print the form (<https://web.reg.chula.ac.th/Manual-W-Eng.pdf>).

2) Complete and sign CR48, provide the reason.

- Undergraduates: obtain instructor and advisor approvals

(https://www.reg.chula.ac.th/document/registrationmanual_ENG_240704.pdf; also stated generally in https://web.reg.chula.ac.th/CR99_2023_EngS.pdf).

- Graduates: obtain advisor approval (per manual;

https://www.reg.chula.ac.th/document/registrationmanual_ENG_240704.pdf).

3) Submit the signed CR48 to your Faculty's Registrar Office; it will be forwarded to the Dean for approval (https://www.reg.chula.ac.th/document/registrationmanual_ENG_240704.pdf).

4) Check the result at Registration Result (CR54) / Personal Information Inquiry > Inquiry about Course Withdrawal Status. If approved, “W” appears; you can print the result as proof (https://web.reg.chula.ac.th/CR99_2023_EngS.pdf).

If you miss the period (late withdrawal)

- Download/print the “Request Form for Late Course Withdrawal (CR48)” from Student > Forms, get required approvals (UG: instructor + advisor; Grad: advisor), pay the late withdrawal fine (300 THB per course; method announced each semester), then check the result at CR54 (https://www.reg.chula.ac.th/document/registrationmanual_ENG_240704.pdf).

ข.2.5 การตอบคำถามที่ไม่มีข้อมูลในดัชนี

[QUERY] วันสอบปลายภาคเรียนปีการศึกษา 2569 คือวันไหน

You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the `__call__` method is faster than using a method to encode the text followed by a call to the `'pad'` method to get a padded encoding.

[LLM ANSWER]

ไม่พบข้อมูล “วันสอบปลายภาคเรียน ปีการศึกษา 2569” ในเอกสารที่ให้

อย่างไรก็ตาม เอกสารระบุช่วงสอบปลายภาคที่เกิดขึ้นในปี 2569 ดังนี้ (อ้างอิงตามเอกสาร):

- ระบบภาคเรียน ภาคการศึกษาปลาย (ปีการศึกษา 2568): 27 เม.ย. – 12 พ.ค. 2569

URL: https://www.reg.chula.ac.th/th/wp-content/uploads/sites/4/2025/07/CR90_Sem2025_Thai.pdf

- ระบบไตรภาค ภาคการศึกษาที่สาม (ปีการศึกษา 2568): 13 – 17 ก.ค. 2569

URL: https://www.reg.chula.ac.th/th/wp-content/uploads/sites/4/2025/07/CR90_Tri2025_Thai.pdf

- ภาคฤดูร้อน (ระบบภาคเรียน): วันสุดท้ายของการเรียนและการสอบ 17 ก.ค. 2569

URL: https://www.reg.chula.ac.th/th/wp-content/uploads/sites/4/2025/07/CR90_Sem2025_Thai.pdf



CHULALONGKORN
BUSINESS SCHOOL

FLAGSHIP FOR LIFE